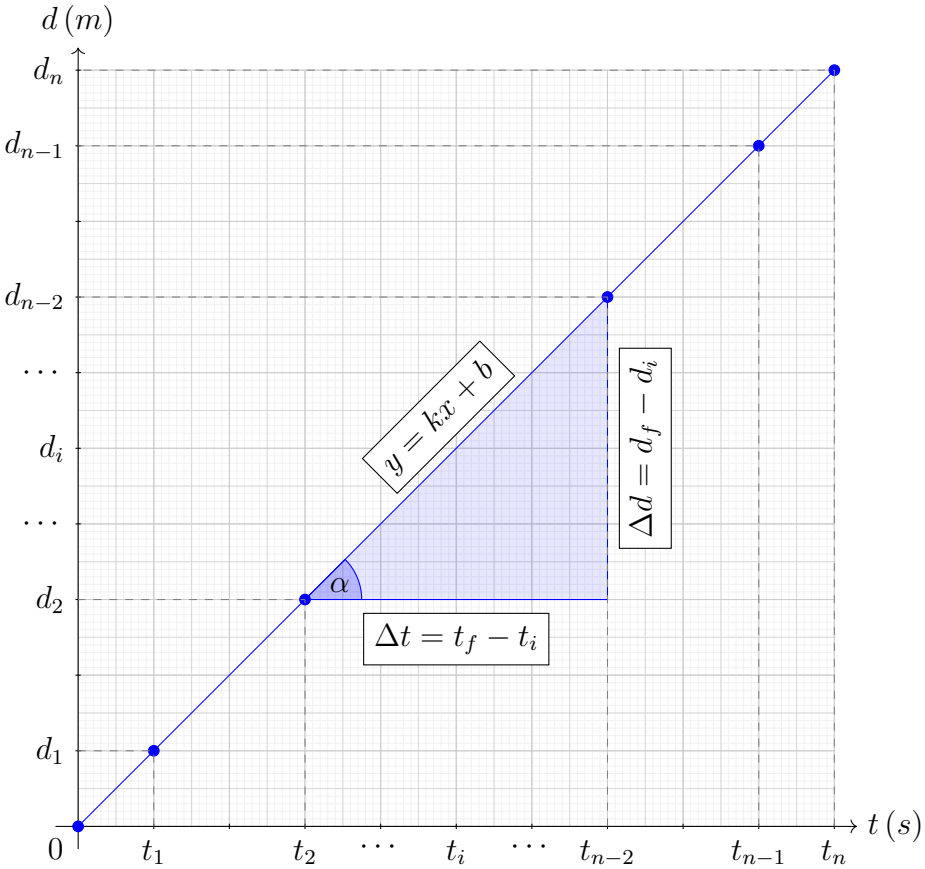
Software de Graficación

Manual de la versión 1.0.0




```tikz
\begin{tikzpicture}
\draw[step=0.1cm,black!5!white,very thin] (0,0) grid (10,10);
\draw[step=0.5cm,black!15!white,very thin] (0,0) grid (10,10);
\draw[step=1cm,black!20!white,very thin] (0,0) grid (10,10);
\path [->,draw] (-0.3,0) -- (10.3,0) node [right] {$t\left(s\right)$};
\path [->,draw] (0,-0.3) -- (0,10.3) node [above] {$d\left(m\right)$};
\draw [blue,fill] (0,0) circle [radius=2pt] node [] {};
\draw [blue,fill] (1,1) circle [radius=2pt] node [] {};
\draw [blue,fill] (3,3) circle [radius=2pt] node [] {};
\draw [blue,fill] (7,7) circle [radius=2pt] node [] {};
\draw [blue,fill] (9,9) circle [radius=2pt] node [] {};
\draw [blue,fill] (10,10) circle [radius=2pt] node [] {};
\foreach \x/\w in {1/$t_1$,2/$ $,3/$t_2$,4/$\cdots$,5/$t_i$,6/$ \cdots$,7/$t_{n-2}$,8/$ $,9/$t_{n-1}$,10/$t_{n}$}
\draw (\x,1pt) -- (\x,-1pt) node[anchor=north] {\w};
\foreach \y/\z in {1/$d_1$,2/$ $,3/$d_2$,4/$\cdots$,5/$d_i$,6/$ \cdots$,7/$d_{n-2}$,8/$ $,9/$d_{n-1}$,10/$d_{n}$}
\draw (1pt,\y) -- (-1pt,\y) node[anchor=east] {\z};
\node at (-.3,-.3) {0};
\draw [blue] (0,0) -- (10,10);
\draw [blue] (3,3)--(7,3)--(7,7);
\path [fill=blue, fill opacity=0.1] (3,3)--(7,3)--(7,7)--cycle;
\draw[dashed,gray] (1,0) -- (1,1)--(0,1);
\draw[dashed,gray] (3,0) -- (3,3)--(0,3);
\draw[dashed,gray] (7,0) -- (7,7)--(0,7);
\draw[dashed,gray] (9,0) -- (9,9)--(0,9);
\draw[dashed,gray] (10,0) -- (10,10)--(0,10);
\node[draw=black, fill=white,rotate=45,above=5pt] at (5,5) {$y=kx+b$};
\node[draw=black, fill=white,rotate=0,below=5pt] at (5,3) {$\Delta t=t_f-t_i$};
\node [draw=black,fill=white,right=0.5cm, rotate=90, anchor=center] at (7,5) {$\Delta d=d_f-d_i$};
\draw [draw=blue,fill=blue, fill opacity=0.2] (3,3) ++(0.75cm,0cm) arc (0:45:0.75cm) -- (3,3);
\path (3,3) -- ++(23:0.5cm) node {$\alpha$};
\end{tikzpicture}
```

Chari • Fernando

SOFTWARE DE GRAFICACIÓN

Versión 1.0.0

Tutor
Zavala • Diego, PhD.
Facultad de Filosofía, Letras y Ciencias de la Educación
Carrera de Pedagogía en las Ciencias Experimentales Matemática y Física
Universidad Central del Ecuador

No está permitida la reproducción total o parcial de este libro por ningún medio: electrónico, mecánico u otros métodos; sin la autorización previa y por escrito de los autores.

Software de Graficación

Manual de la versión 1.0.0

https://graficarferchari.shinyapps.io/graficador_app2/

Chari Fernando
Carrera de Pedagogía en las Ciencias Experimentales Matemática y Física
Universidad Central del Ecuador

12 de noviembre de 2023

Contenido

1 Introducción **5**
1.1 Ejemplo: Teorema de Pitágoras . . . 5
1.2 Importante: Prerrequisitos . . . 8

2 Instalación **9**
2.1 Instalación de MiKTEX . . . 10
2.2 Instalación de TEXStudio . . . 10
2.3 Instalación de R . . . 11
2.4 Instalación de RStudio . . . 11
2.5 Instalación de Sweave . . . 12

3 Configuración **13**
3.1 Archivo LATEX . . . 14
3.2 Archivos R y R Sweave . . . 14
3.3 Archivos HTML y CSS . . . 14

4 LATEX **15**
4.1 Plantilla . . . 16
4.1.1 Ejes coordenados . . . 16
4.1.2 Cuadrícula . . . 17
4.1.3 Tarjas . . . 19
4.1.4 Datos experimentales . . . 20
4.1.5 Rastros . . . 21
4.1.6 Recta de regresión lineal . . . 22
4.1.7 Triángulo de inspección . . . 24
4.1.8 Leyenda . . . 27
4.1.9 Título del diagrama . . . 28

5 RSweave **31**
5.1 Plantilla RSweave: 4 datos . . . 33
5.1.1 Magnitudes: *Chunk 1* . . . 34
5.1.2 Datos variable independiente: *Chunk 2* . . . 36
5.1.3 Datos variable dependiente: *Continuación del Chunk 2* . . . 39
5.1.4 Función cero: Continuación del *Chunk 2* . . . 40

5.1.5 Regresión Lineal: *Chunk 3* 41
5.1.6 Parámetros del Triángulo de Inspección: Continuación del *Chunk 3* 43
5.1.7 Análisis Matemático: *Chunk 4* 44
5.2 Plantilla RSweave: 5 datos 45
5.3 Plantilla RSweave: 6 datos 47
5.4 Plantilla RSweave: 7, 8, 9 y 10 datos 48

6 Flujograma **49**
6.1 Algoritmo de búsqueda 50
6.2 Algoritmo de asignación 51
6.3 Algoritmo para compilar 51
6.4 Algoritmo guardar magnitudes, unidades y dimensiones 52
6.5 Algoritmo guardar datos experimentales 53
6.6 Algoritmo ejecutar 53
6.7 Algoritmo entradas vacías 54

7 R **55**
7.1 Librería Shiny 56
7.1.1 Interfaz de Usuario UI 56
7.1.2 Servidor 58

8 HTML y CSS **61**
8.1 HTML y CSS 62
8.1.1 Diseño HTML 62
8.1.2 Diseño CSS 64

9 Publicación **67**
9.1 Shiny Apps 68

10 Referencias Bibliográficas **69**

Introducción

Bienvenidos al *Manual de de desarrollo de un Software de Graficación para la representación, verificación y/o rectificación de datos experimentales.* Como estudiante de la Carrera de Pedagogía de las Ciencias Experimentales Matemática y Física, quien, al igual que muchos de mis lectores, enfrente varios desafíos al elaborar mis diagramas de modelos líneales de diferentes prácticas de laboratorio. Inicialmente, utilice una variedad de herramientas como Word, Excel; sin embargo, la transición a la enseñanza y el aprendizaje en línea durante la pandemia de Covid-19 complicó aún más este proceso, por lo cual, decidi transcicionar a GeoGebra, Scientific Workplace, Adobe Illustrator, pero obtuve los mismos resultados que con los primero programas.

En paralelo, por curiosidad y en recomendación del Lic. Espinosa Juan incorporé el lenguaje de marcado LaTeX en mi guía de autoaprendizaje. LaTeX, un sistema de composición de textos científicos, se reveló como una elección poderosa que no se limita a la creación de ecuaciones matemáticas. Por ejemplo, el código `$\Delta d=v\Delta t$`, produce $\Delta d = v\Delta t$, pero LaTeXva mucho más allá que una ecuación de MRU. Los estudiantes de LaTeX tienen a disposición una multitud de herramientas que permiten elaborar párrafos, libros, gráficos, entre otros. Por ejemplo, el código `\tikz \fill[blue] (0,0) circle (1mm);` produce ●.

En contraste, el lenguaje de programación **R** brinda la capacidad de realizar numerosas operaciones matemáticas (modelización lineal y no lineal, entre otras), estadísticas (pruebas estadísticas, entre otras), y gráficas. Por ejemplo, al escribir el código `> x <- 2`, se define una variable llamada x y se le asigna el valor 2. A partir de este punto, es posible realizar diversas operaciones matemáticas. Por ejemplo, sumar el mismo número consigo mismo, el código `> x + x` produce el valor de 4, que es visible en la consola como `[1] 4`.

A pesar de lo atractivo que puede llegar a ser **R** para los estadíticos y de las previas especificaciones, puede que aún se pregunte ¿Por qué **R**? "Un archivo fuente simple contiene el texto de documentación y el código **R**, los cuales son entrelazados dentro de un documento final que contiene el texto dedocumentación junto con el código **R** y/o la salida del código"(Morales, 2006, p. 2). Asimismo, Leisch (2005) menciona que: "todo el poder de LaTeX (para composición de texto de alta calidad) y de **R** (para análisis de datos) puede usarse simultáneamente"(p. 1). Estas definiciones resumen de manera *sucinta* la razón detrás de esta elección.

Se enfatiza en la palabra *suscinta*, debido a que, el aprendizaje autodidacta de LaTeX, **R** y **Sweave** involucra un sinfín de frustraciones y supone una inversión considerable de tiempo, para empezar a tener resultados aceptables. Bajo esta óptica, nace el deseo de desarrollar un "*Software* de Graficación para la representación, verificación y/o rectificación de datos experimentales"de modo que, estudiantes ajenos a este campo eviten lidiar con el código cada vez que deseen representar datos experimentales y, al mismo tiempo, brindar una solución en términos de tiempo de aprendizaje para quienes desean aventurarse con estos lenguajes.

1.1 Ejemplo: Teorema de Pitágoras

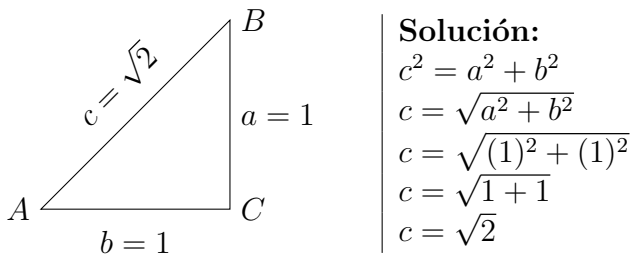
El profesor A, del área de matemáticas planea redactar un deber de un ejercicio diferente para cada uno de sus 50 estudiantes sobre el teorema de pitágoras. Por tanto,

construye en la parte izquierda del documento un triángulo rectángulo de base 1 cm y de altura 1 cm. Y, en la parte derecha el proceso de solución.

Para ello el profesor A, podría utilizar:

```
\begin{tikzpicture}[scale=2.5]
 \draw [] (0,0) node [left]{$A$} --(1,0) node [right]{$C$} --(1,1)node[right] {$B$}--
    cycle;
 \node [below=4pt] at (.5,0) {$b=1$};
 \node [right] at (1,.5) {$a=1$};
 \node [above=4pt,rotate=45] at (.5,.5) {$c=\sqrt{2}$};
 \begin{scope}[xshift=2.5cm,yshift=.4cm]
 \node at (0,0) [rectangle,minimum size=-0.5cm]{
  \begin{tabular}{|l}
  \textbf{Solucion:}\\
  $c^2=a^2+b^2$ \\
  $c=\sqrt{a^2+b^2}$ \\
  $c=\sqrt{(1)^2+(1)^2}$ \\
  $c=\sqrt{1+1}$ \\
  $c=\sqrt{2}$
  \end{tabular}};
 \end{scope}
\end{tikzpicture}
```

Obteniendo, el resultado esperado:



Sin embargo, este proceso es de carácter estático por lo que el profesor A tendría que cambiar los valores de los catetos a y b, además, realizar las operaciones matemáticas para determinar el valor de la hipotenusa c de forma manual por 50 veces seguidas, para la sección del gráfico y la solución, respectivamente. Lo cuál resulta ser nada eficiente en términos de tiempo de ejecución. En concecuencia, el profesor A, decide incluir algunas líneas de código de **R** en su plantilla LaTeX para facilitar este proceso repetitivo.

Para ello el profesor A, podría hacer uso de:

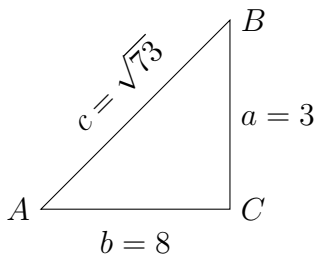
```
\begin{tikzpicture}[scale=2.5]
# INICIO DEL CODIGO R
<<echo=false>>=
a <-sample(1:10,1)
b <-sample(1:10,1)
ac <- a^2
bc <- b^2
c <- ac+bc
@
# FIN DEL CODIGO R
\draw [] (0,0) node [left]{$A$} --(1,0) node [right]{$C$} --(1,1)node[right] {$B$}--
    cycle;
\node [below=4pt] at (.5,0) {$b=\Sexpr{b}$};
\node [right] at (1,.5) {$a=\Sexpr{a}$};
\node [above=4pt,rotate=45] at (.5,.5) {$c=\sqrt{\Sexpr{c}}$};
\begin{scope}[xshift=2.5cm,yshift=.4cm]
 \node at (0,0) [rectangle,minimum size=-0.5cm]{
  \begin{tabular}{|l}
   \textbf{Solucion:}\\
   $c^2=a^2+b^2$ \\
```

```
   $c=\sqrt{a^2+b^2}$ \\
   $c=\sqrt{(\Sexpr{a})^2+(\Sexpr{b})^2}$ \\
   $c=\sqrt{\Sexpr{ac}+\Sexpr{bc}}$ \\
   $c=\sqrt{\Sexpr{c}}$
 \end{tabular}};
\end{scope}
\end{tikzpicture}
```

El profesor A encuentra que por cada nueva compilación[1] del código se obtiene un triángulo rectángulo totalmente diferente. *Simplemente*, el profesor A tendrá que compilar 50 veces o dar click 50 veces en el botón compilar y guardar los archivos con los nombres de sus estudiantes.

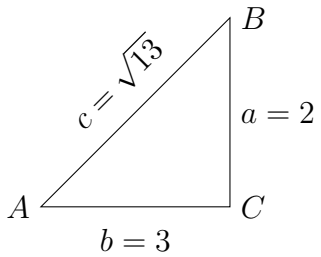
Suponiendo, que el profesor A compiló tres veces el archivo, obtendrá:

Compilación 1: Estudiante 1



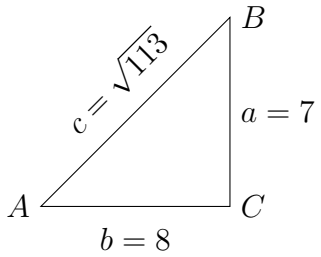
Solución:
$c^2 = a^2 + b^2$
$c = \sqrt{a^2 + b^2}$
$c = \sqrt{(3)^2 + (8)^2}$
$c = \sqrt{9 + 64}$
$c = \sqrt{73}$

Compilación 2: Estudiante 2



Solución:
$c^2 = a^2 + b^2$
$c = \sqrt{a^2 + b^2}$
$c = \sqrt{(2)^2 + (3)^2}$
$c = \sqrt{4 + 9}$
$c = \sqrt{13}$

Compilación 3: Estudiante 3



Solución:
$c^2 = a^2 + b^2$
$c = \sqrt{a^2 + b^2}$
$c = \sqrt{(7)^2 + (8)^2}$
$c = \sqrt{49 + 64}$
$c = \sqrt{113}$

Sin embargo, el profesor A esta inconforme con estos resultados porque no tiene el tiempo suficiente para dar 50 clicks al botón compilar. Por lo tanto, quiere ir más alla y decide crear un archivo en **R**, de extensión[2] `.R`, que compile el archivo **Sweave**, de

[1] Ejectuar las líneas de código
[2] sufijo compuesto por caracteres que señala el tipo de archivo

extensión `.Rnw` tantas veces como el deberes necesite y guarde los documentos generados, de extensión `.pdf` con un nombre específico.

Para ello, si el profesor quiere 5 deberes puede hacer uso de:

```
library(knitr)
setwd("C:/RStudio/Deberes_aleatorios")

compilePDF <- function(archivo, num_copias) {
 for (i in 1:num_copias) {
  knit(paste0("deber", archivo, ".Rnw"))
  tools::texi2pdf(paste0("deber", archivo, ".tex"))
  file.rename(paste0("deber", archivo, ".pdf"), paste0("deber", archivo, "_", i, ".pdf")
    )
 }
}

compilePDF(1, 5)
```

En concencuencia, se crean en la dirección `C:/RStudio/Deberes_aleatorios` los archivos correspondientes:

1. `deber_1.pdf`
2. `deber_2.pdf`
3. `deber_3.pdf`
4. `deber_4.pdf`
5. `deber_5.pdf`

Finalmente, el profesor A está satisfecho con los resultados obtenidos. No obstante, el profesor A considera seguir estudiando **R**, **Sweave** y LaTeX de manera que pueda ayudar a otros profesores ajenos a este campo.

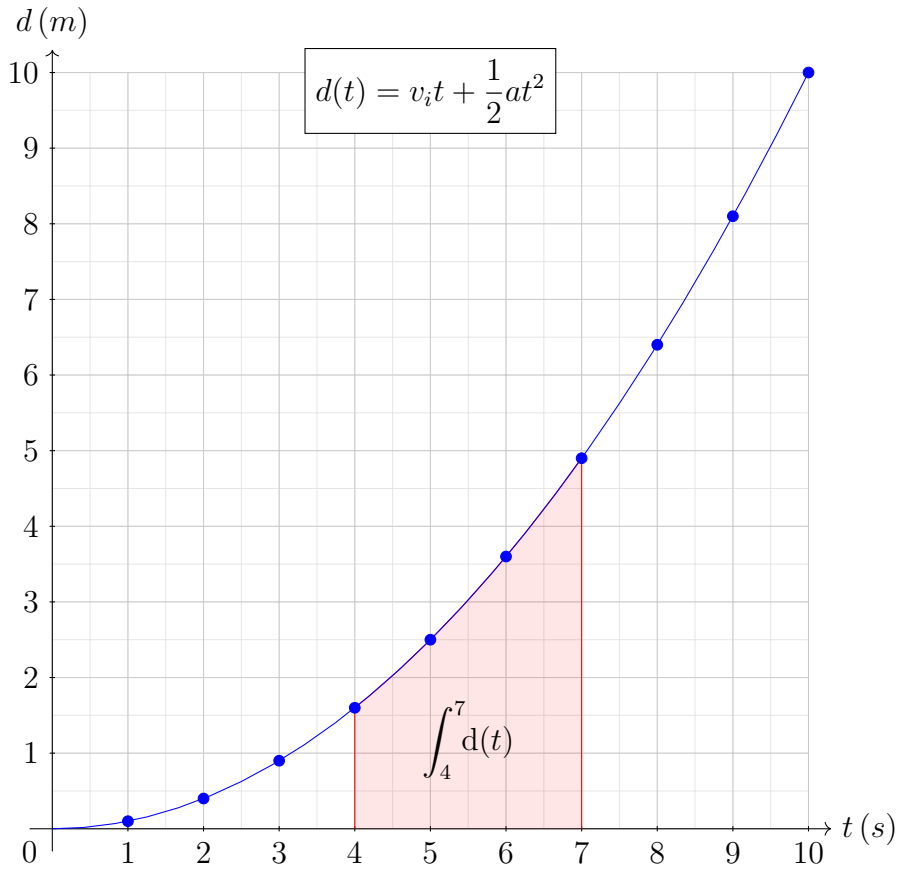
1.2 Importante: Prerrequisitos

Como se puede apreciar en el ejemplo 1.1, nuestros lectores deben poseer conocimientos previos en el uso de LaTeX, **R** y **Sweave**. Esto se debe a que el presente manual no tiene la intención de servir como una introducción en el aprendizaje de estos lenguajes. Más bien, se enfoca en proporcionar una explicación detallada sobre cómo desarrollar un "*Software* de Graficación para la representación, verificación y/o rectificación de datos experimentales", y todos los aspectos relacionados con ello. Esto incluye la instalación y configuración del entorno de desarrollo integrado (IDE), estructuras, explicación de los comandos de LaTeX, de objetos, variables y funciones de **R**, la creación de nuevas funciones mediante flujogramas, el desarrollo de la interfaz de usuario UI y la publicación del *software*.

Chari Castillo Fernando Javier
Quito, 12 de noviembre de 2023

Instalación

La instalación de un paquete, lenguaje y/o IDE son procesos que marcan el punto de partida del presente *Manual de desarrollo de un Software de Graficación para la representación, verificación y/o rectificación de datos experimentales.*




```
\begin{tikzpicture}[domain=0:10]
 \normalsize
 \draw[step=0.5cm,black!15!white,very thin] (0,0) grid (10,10);
 \draw[step=1cm,black!20!white,very thin] (0,0) grid (10,10);
 \path [->,draw] (-0.3,0) -- (10.3,0) node [right] {$t\left(s\right)$};
 \path [->,draw] (0,-0.3) -- (0,10.3) node [above] {$d\left(m\right)$};
 \foreach \x in {1,2,3,4,5,6,7,8,9,10}
 \draw (\x,1pt) -- (\x,-1pt) node[anchor=north] {\x};
 \foreach \y in {1,2,3,4,5,6,7,8,9,10}
 \draw (1pt,\y) -- (-1pt,\y) node[anchor=east] {\y};
 \node at (-.3,-.3) {0};
 \path [draw=red, fill=red, fill opacity=0.1, domain=4:7, variable=\x] (4,0) --  plot (\
    x,0.1*\x*\x) -- (7,0);
 \draw[color=blue] plot (\x,0.1*\x*\x);
 \node[draw=black, fill=white,below=5pt] at (5,10.5) {$d(t) = v_it+\displaystyle\frac
    {1}{2}at^2$};
 \node[below=5pt] at (5.5,2) {$\displaystyle\int_{4}^{7} \!\!\mathrm{d}(t)$};
 \foreach \x/\w in {1/0.1,2/0.4,3/0.9,4/1.6,5/2.5,6/3.6,7/4.9,8/6.4,9/8.1,10/10}
 \draw [blue,fill] (\x,\w) circle [radius=2pt] node[] {};
\end{tikzpicture}
```

Es de importancia mencionar que existen otras multiplataformas que permiten trabajar con estos lenguajes. Entre los más reconocidos se incluyen Overleaf, Visual Studio Code, Posit Cloud. No obstante, en el presente manual nos enfocaremos exclusivamente en el uso de las respectivas IDE de LaTeX y de **R**. Por lo tanto, puedo dar instrucciones detalladas sobre la instalación de estos programas.

Observación: Los siguientes procedimientos de instalación están diseñados para sistemas operativos Windows de 64 bits.

2.1 Instalación de MiKTeX

MiKTeX es un gestor de paquetes mayoritariamente desarrollados en TeXpara ser utilizados en LaTeX, de acuerdo a las necesidades del usuario. Para el correcto procedimiento de instalación de MiKTeX, se debe seguir la siguiente secuencia de procesos:

1. Ingresar a la sección de descargas de la página de MiKTeX.
2. Descargar la última versión de MiKTeX.
3. Abrir la ubicación del archivo `basic-miktex-23.10-x64.exe`.
4. Ejecujar como administrador el archivo `basic-miktex-23.10-x64`.
5. Leer y Aceptar las condiciones de la ventana emergente `Copyng Conditions`.
6. Dejar la configuración por defecto de las ventanas `Installation Scope`, `Directory` y `Settings`.
7. Seleccionar el botón `Start` de la ventana `Review`.
8. Esperar y cerrar el proceso de instalación.

Además, esta disponible las guía de instalación de MikTeX para Mac y Linux en la página de documentos de MikTeX.

2.2 Instalación de TeXStudio

TeXStudio es un entorno de escritura integrado para crear documentos LaTeX. Para el correcto procedimiento de instalación de TeXStudio, se debe seguir la siguiente secuencia de procesos:

1. Ingresar a la sección `Donwload` de la página de TeXStudio.
2. Descargar la última versión de TeXStudio.
3. Abrir la ubicación del archivo `texstudio-4.6.3-win-qt6`.
4. Ejecutar como administrados el archivo `texstudio-4.6.3-win-qt6`.
5. Leer la ventana emergente de `Bienvenida` y presionar `Siguiente` para continuar.
6. Leer y dejar la configuración por defecto de la ventana `lugar de instalación` y presionar `Instalar` para iniciar con la instalación de TeXStudio.

7. Esperar y cerrar el proceso de instalación.

Asimismo, TEXStudio esta disponible para MacOS y Linux en la sección `Donwload` de la página de TEXStudio.

2.3 Instalación de R

R es un lenguaje de programación para el análisis estadístico. Para el correcto procedimiento de instalación de **R**, se debe seguir la siguiente secuencia de procesos:

1. Ingresar a la sección `The Comprehensive R Archive Network` de **R**.
2. Seleccionar la opción de `R for Windows`.
3. Presionar sobre `install R for the first time`.
4. Presionar sobre `Download R-4.3.1 for Windows`.
5. Abrir la ubicación del archivo `R-4.3.1-win`.
6. Ejecutar como administrador el archivo `R-4.3.1-win`.
7. Seleccionar el idioma de preferencia para el proceso de instalación.
8. Leer el contenido de la ventana emergente `Información` y presionar en `Siguiente`.
9. Leer y dejar la configuración por defecto de las ventanas emergentes `Carpeta de Destino`, `Componentes`, `Configuración`, `Carpeta de Menú de Inicio` y `Tareas Adicionales` y presionar en `Siguiente`.
10. Esperar y cerrar el proceso de instalación.

Es importante mencionar que existen versiones de **R** para MacOS y Linux, disponible en la página de `The Comprehensive R Archive Network` de **R**.

2.4 Instalación de RStudio

RStudio es un entorno de desarrollo integrado IDE para **R** y **Python** desarrollado por Posit. Para el correcto procedimiento de instalación de **RStudio**, se debe seguir la siguiente secuencia de procesos:

1. Ingresar a la sección `Download RStudio Desktop` de la página de **Posit**.
2. Presionar sobre el botón `Download RStudio Desktop for Windows` y experamos que culmine el proceso de descarga .
3. Abrir la ubicación del archivo `RStudio-2023.09.1-494`.
4. Ejecutar como administrador el archivo `RStudio-2023.09.1-494`.
5. Leer la ventana emergente de `Bienvenida` y presionar en `Siguiente` para continuar.
6. Leer y dejar la configuración por defecto de las ventanas emergentes `Lugar de instalación` y presionar en `Siguiente`.

7. Leer y dejar la configuración por defecto de las ventanas emergentes `Carpeta del Menú Inicio` y presionar en `Instalar`.

8. Esperar y cerrar el proceso de instalación.

Además, existen versiones de **RStudio** para MacOS y Linux, disponible en la sección final de la página de `Download RStudio Desktop` de **Posit**.

2.5 Instalación de Sweave

Sweave es una librería de **R** desarrollada por Friedrich Leisch[1] que permite incrustar el código **R** para realizar análisis completos de datos en documentos de LaTeX. Para el correcto procedimiento de instalación de **Sweave**, se debe seguir la siguiente secuencia de procesos:

1. Crear una carpeta con el nombre **Sweave** en la dirección `...\texmf\tex\latex`.

2. Copiar en la carpeta **Sweave** el archivo `Sweave.sty` que se encuentra en la carpeta `...\R\R-4.3.1\share\texmf`.

3. Ingresar a la aplicación de **MiKTeX Console**.

4. Seleccionar la pesataña `Updates`.

5. Presionar sobre el botón `Check for updates`

6. Presionar sobre el botón `Update Now`

7. Esperar y cerrar la aplicación **MiKTeX Console**.

Para verificar si el proceso de instalación de **Sweave** se ha completado con éxito, el usuario debe proceder de la siguiente manera en la aplicación **RStudio**, la cual está siendo utilizada para el desarrollo del Software de Graficación:

1. Abra la aplicación **RStudio**.

2. Navegue hacia la opción `New File` ubicada en la pestaña `File` en la barra de menú.

3. Dentro del menú desplegado, busque la nueva opción denominada `R Sweave`.

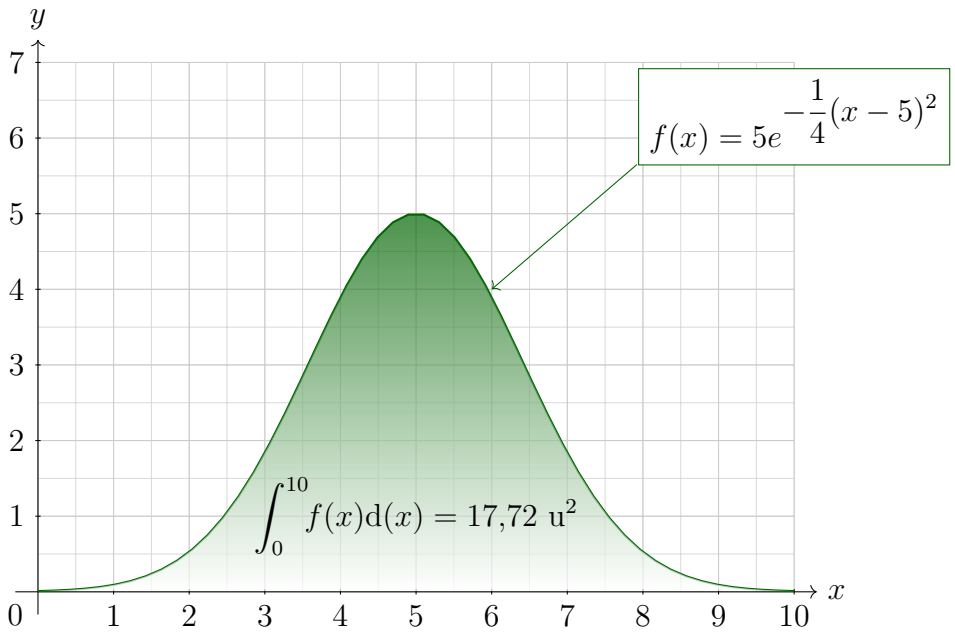
Si la opción `R Sweave` está disponible, esto indica que la instalación se ha realizado correctamente. No obstante, si esta opción no se encuentra disponible, se sugiere una revisión minuciosa del procedimiento de instalación, centrándose en la verificación de las rutas de archivo y la precisión de los nombres de los archivos.

En caso de que persistan los problemas, como última medida, se recomienda desinstalar y volver a instalar adecuadamente las siguientes aplicaciones: **R**, **RStudio**, y MiKTeX.

Este proceso de verificación asegurará que el entorno de desarrollo esté configurado de manera adecuada para el desarrollo del software y garantizará un flujo de trabajo sin problemas.

[1] Documentación disponible en `https://n9.cl/s3qkt`

Configuración




```
\begin{tikzpicture}[domain=0:10]
 \draw[step=0.5cm,black!15!white,very thin] (0,0) grid (10,7);
 \draw[step=1cm,black!20!white,very thin] (0,0) grid (10,7);
 \path [->,draw] (-0.3,0) -- (10.3,0) node [right] {$x$};
 \path [->,draw] (0,-0.3) -- (0,7.3) node [above] {$y$};
 \foreach \x in {1,2,3,4,5,6,7,8,9,10}
 \draw (\x,1pt) -- (\x,-1pt) node[anchor=north] {\x};
 \foreach \y in {1,2,3,4,5,6,7}
 \draw (1pt,\y) -- (-1pt,\y) node[anchor=east] {\y};
 \node at (-.3,-.3) {0};
 \draw [verdeoscuro,samples=50,thick] plot (\x,{5*exp(-0.25*\x*\x+2.5*\x-6.25)});
 \node[fill=white,below=5pt,draw=verdeoscuro] at (10,7.1) {$f(x)=5e^{\displaystyle-\frac
    {1}{4}(x-5)^2}$};
 \draw [verdeoscuro,<-] (6,4) -- (7.92,5.65);
 \shade [top color=verdeoscuro,bottom color=white,opacity=.7] (0,0) [samples=50] plot (\
    x,{5*exp(-0.25*\x*\x+2.5*\x-6.25)}) (10,0);
 \node at (5,1) {$\displaystyle\int_{0}^{10} \!\!f(x)\mathrm{d}(x)=17.72 \ \mathrm{u}^2$
    };
\end{tikzpicture}
```

Durante el proceso de desarrollo del *Software de Graficación*, se generarán múltiples archivos con extensiones específicas. Por lo tanto, es esencial establecer una estructura organizativa que permita el almacenamiento adecuado de estos archivos y facilite la gestión del trabajo en curso.

3.1 Archivo LaTeX

En este contexto, se recomienda la creación de una carpeta destinada a albergar el archivo que contendrá el código LaTeX, comúnmente referido como "plantilla". Para llevar a cabo esta configuración de manera efectiva, se sugiere seguir el siguiente procedimiento:

1. Inicie creando la carpeta `Software_de_graficacion` en el disco local C.
2. Acceda a la ubicación a través de la dirección `C:\Software_de_graficacion` y cree una subcarpeta con el nombre "Plantilla".
3. Navegue a la dirección `C:\Software_de_graficacion\Plantilla` y proceda a crear un archivo llamado `Plot.tex`

Con esta configuración establecida, se podrá trabajar en el código de la plantilla LaTeX, específicamente en el archivo `Plot.tex` durante el desarrollo del Capítulo 3. Esta estructura organizativa promoverá la claridad y la eficiencia en la gestión de los archivos del proyecto.

3.2 Archivos R y R Sweave

En esta sección, se aborda la configuración de los archivos **R** y **R Sweave**. En este sentido, se sugiere seguir el siguiente procedimiento:

1. Acceda a la ubicación `C:\Software_de_graficacion` y cree una subcarpeta con el nombre `Graficador`.
2. En la ruta `C:\Software_de_graficacion\Graficador` proceda a crear un archivo llamado `Graficador.R` y diez archivos adicionales con los siguientes nombres: `Plot4.Rnw`, `Plot5.Rnw`, `Plot6.Rnw`, `Plot7.Rnw`, `Plot8.Rnw`, `Plot9.Rnw` y `Plot10.Rnw`.

Estos archivos serán fundamentales para el funcionamiento del servidor y la generación de los diagramas de dispersión de acuerdo a cada plantilla de datos experimentales.

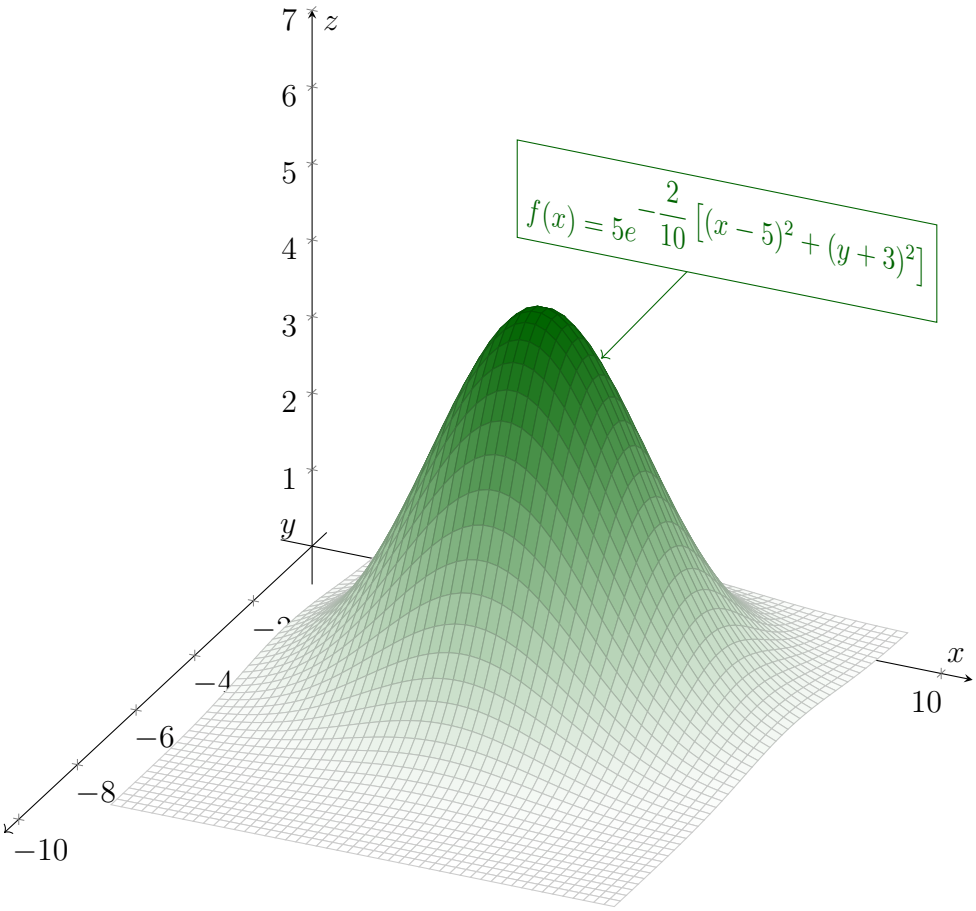
3.3 Archivos HTML y CSS

Finalmente, proceda configurar las direcciones de los archivos que contedrán el HTML y el CSS. Para lo cuál, se recomienda el siguiente procedimiento:

1. Acceda a la ubicación `C:\Software_de_graficacion\Graficador` y cree una subcarpeta con el nombre `www`.
2. En la ruta `C:\Software_de_graficacion\Graficador\www` proceda a crear dos archivos con los nombres `index.html` y `index.css`.

Al establecer esta estructura de directorios, se facilita la organización y el acceso a estos componentes críticos del desarrollo del *Software de Graficación*.

LaTeX




```
\begin{tikzpicture}
\begin{axis}[width=15cm,height=15cm,
 xlabel=$x$, ylabel=$y$,zlabel=$z$,
 axis lines=center,
 xmin=-0.5,xmax=10.5,ymax=0.5,ymin=-10.5,zmin=-0.5,zmax=7,
 every inner y axis line/.append style={<-}]
 \addplot3 [
 surf,
 domain y=-9:1,
 domain  =1:9,
 samples=50,
 ] {5*exp(-0.2*(x-5)^2-0.2*(y+3)^2)};
 \begin{scope}[canvas is xz plane at y=12]
  \draw[verdeoscuro,<-] (-1,-2,5.5) -- (1,0,5.5) node [fill=white,draw=verdeoscuro,
    transform shape] {$f(x)=5e^{\displaystyle-\frac{2}{10}\left[(x-5)^2+(y+3)^2\right]}$
    };
 \end{scope}
\end{axis}
\end{tikzpicture}
```

4.1 Plantilla

Una vez instalado y configurado los recursos necesarios a utilizarse. Podemos iniciar a escribir código LaTeX. Para ello, abrimos el archivo `Plot.tex` que se encuentra en la ubicación `C:\Software_de_graficacion\Plantilla`, con la aplicación TeXStudio.

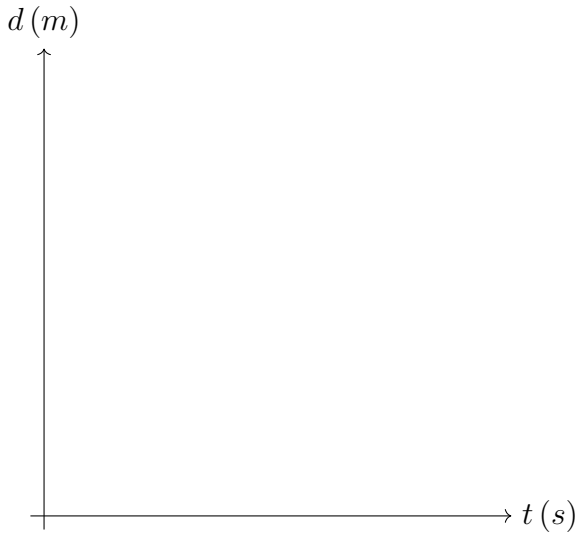
Para crear un gráfico utilizando el paquete Ti*k*Z, es necesario iniciar un entorno de dibujo en LaTeX. Esto se logra mediante el uso de los comandos `\begin{tikzpicture}` para abrir el entorno de dibujo y `\end{tikzpicture}` para cerrarlo. La estructura se asemeja a la siguiente:

```
% PREAMBULO DEL DOCUMENTO
\documentclass[a4paper,12pt]{book}
\usepackage{tikz}
% DOCUMENTO
\begin{document}
Iniciamos el entorno de dibujo:
\begin{tikzpicture}
Contenido del dibujo.
\end{tikzpicture}
\end{document}
```

De esta manera, se establece un entorno adecuado para la construcción de gráficos con Ti*k*Z.

4.1.1 Ejes coordenados

El comando `\path [opciones gráficas] ⟨operación de ruta⟩;` representa la forma más elemental de un dibujo en Ti*k*Z. En esta construcción, la operación de ruta se inicia con una coordenada `(x,y)` que establece el punto de partida, seguido de una extensión de ruta `--`, y finaliza con otra coordenada `(x,y)`. Por ejemplo, el código `\tikz \path [draw] (0,0) -- (2,0);` produce _____. Es importante notar que el comando `\path` puede incluir opciones gráficas, como `[draw]` que tiene un efecto inmediato en las operaciones de ruta subsiguientes. En esta situación, `[draw]` dibuja el camino especificado en la operación de ruta. Este concepto es la base para construir gráficos más complejos, como ejes cartesianos.




```
\usetikzlibrary{intersections}
\begin{tikzpicture}
 \path [draw,->,name path=eje x] (-.3,0) --
    (10.3,0) node [right] {$t\left(s\right)$
    };
 \path [draw,->,name path=eje y] (0,-.3) --
    (0,10.3) node [above] {$d\left(m\right)$
    };
\end{tikzpicture}
```

Se utiliza la librería `intersections` para acceder a la opción de ruta `->` y dibujar la saetas de los ejes, las opciones `name path=eje x` y `name path=eje y` sirven para asignar

un nombre a la ruta trazada con `\path`, respectivamente. Y, referenciarla en alguna operación posterior.

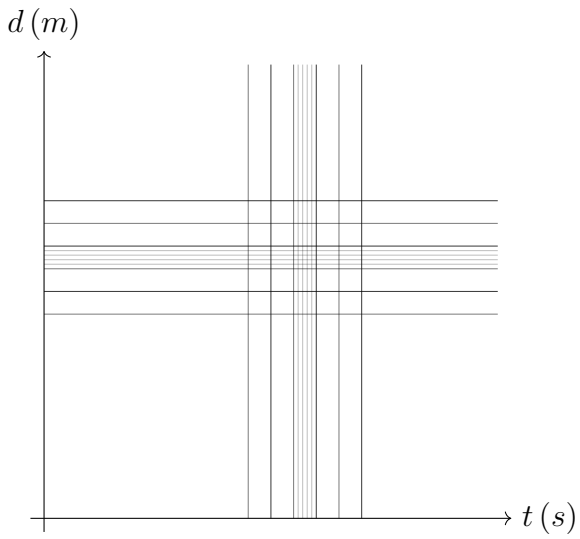
El comando `\path ... node ⟨foreach⟩ [opciones] (nombre) {contenido}...;`, se emplea para crear nodos en Ti*k*Z. Normalmente, un nodo es alguna figura simple con algo de texto en él. Todos lo elementos de la sintaxis del nodo son opcionales, y se describirán en detalle a medida que aparezcan en el manual.

Las opciones de nodo, como `right` y `above`, se utilizan para controlar la posición del contenido del nodo. La opción `right` coloca el contenido del nodo a la derecha de la coordenada `(10.3,0)`, mientras que `above` lo coloca por encima de la coordenada `(0,10.3)`.

Finalmente, dentro del contenido del nodo se etiqueta el eje x y el eje y con el nombre `{$t\left(s\right)$}` y `{$d\left(m\right)$}`, respectivamente.

4.1.2 Cuadrícula

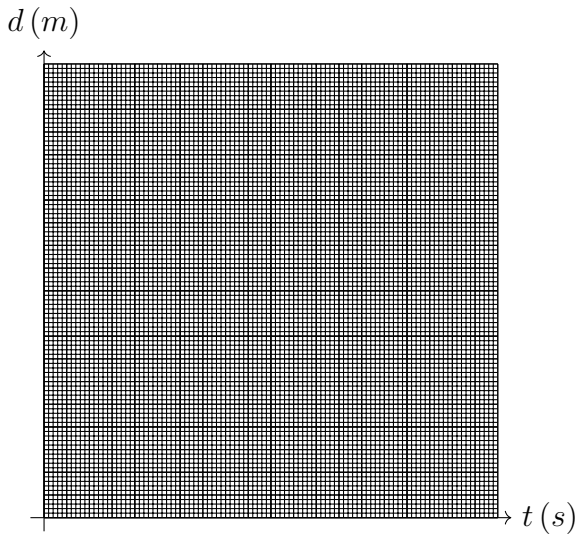
El siguiente proceso es emular una cuadrícula similar a la cuadrícula de un papel milimétrico. Existen varios métodos para lograr este cometido, uno podría pensar en dibujar varios caminos, uno por cada centímetro con cierta opacidad, luego por cada medio centímetro con una opacidad menor y por cada milímetro con una opacidad menor a las dos anteriores.




```
\path [draw,->,name path=eje x] (-.3,0) --
   (10.3,0) node [right] {$t\left(s\right)$
   };
\path [draw,->,name path=eje y] (0,-.3) --
   (0,10.3) node [above] {$d\left(m\right)$
   };
\path [draw,opacity=.75] (0,5)--(10,5);
\path [draw,opacity=.75] (0,6)--(10,6);
\path [draw,opacity=.75] (0,7)--(10,7);
\path [draw,opacity=.75] (5,0)--(5,10);
\path [draw,opacity=.75] (6,0)--(6,10);
\path [draw,opacity=.75] (7,0)--(7,10);
\path [draw,opacity=.5] (0,4.5)--(10,4.5);
\path [draw,opacity=.5] (0,5.5)--(10,5.5);
\path [draw,opacity=.5] (0,6.5)--(10,6.5);
\path [draw,opacity=.5] (4.5,0)--(4.5,10);
\path [draw,opacity=.5] (5.5,0)--(5.5,10);
\path [draw,opacity=.5] (6.5,0)--(6.5,10);
\path [draw,opacity=.25] (0,5.6)--(10,5.6);
\path [draw,opacity=.25] (0,5.7)--(10,5.7);
\path [draw,opacity=.25] (0,5.8)--(10,5.8);
\path [draw,opacity=.25] (0,5.9)--(10,5.9);
\path [draw,opacity=.25] (5.6,0)--(5.6,10);
\path [draw,opacity=.25] (5.7,0)--(5.7,10);
\path [draw,opacity=.25] (5.8,0)--(5.8,10);
\path [draw,opacity=.25] (5.9,0)--(5.9,10);
\end{tikzpicture}
```

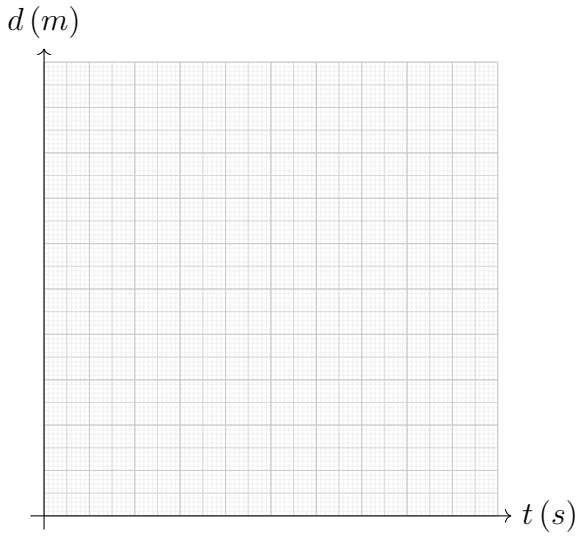
```
\begin{tikzpicture}
```

Si bien este método puede ser bueno en otras situaciones, para el desarrollo del presente manual no lo es. Por lo cual, el comando `\draw [opciones] ⟨operación de ruta⟩;` simplifica esta acción, si se agrega `grid` a la operación de ruta, se puede construir una cuadrícula. Por ejemplo, el código `tikz \draw [step=2pt] (0,0) grid (10pt,10pt);` produce ▦. Nótese, que la opción `step=2pt` define los pasos (separación entre cada recta de la cuadrícula) en el `eje x` y el `eje y`. Bajo este contexto, se puede utilizar el siguiente código:




```
\begin{tikzpicture}
 \path [draw,->,name path=eje x] (-.3,0) --
    (10.3,0) node [right] {$t\left(s\right)$
    };
 \path [draw,->,name path=eje y] (0,-.3) --
    (0,10.3) node [above] {$d\left(m\right)$
    };
 \draw [step=0.1cm] (0,0) grid (10,10);
 \draw [step=0.5cm] (0,0) grid (10,10);
 \draw [step=1cm]   (0,0) grid (10,10);
\end{tikzpicture}
```

Al observar nuevamente el dibujo, se puede deducir que no coincide con el resultado esperado. En términos generales, se requiere que todas las líneas de la cuadrícula tengan un grosor menor que el de los ejes. Además, la cuadrícula definida con `step=1cm` debe presentar un color más tenue en comparación al color de los ejes. De igual forma, la cuadrícula de `step=0.5cm` debe tener un color más tenue que la cuadrícula anterior, y así sucesivamente. Es importante, que todas las cuadrículas esten dibujadas por debajo de los ejes cartesianos. Para lograr estos ajustes, se deben implementar los siguientes cambios en las opciones de ruta de `\draw` para las diferentes cuadrículas.




```
\begin{tikzpicture}
 \draw [step=0.1cm,black!5!white,very thin]
     (0,0) grid (10,10);
 \draw [step=0.5cm,black!15!white,very thin]
     (0,0) grid (10,10);
 \draw [step=1cm,black!20!white,very thin]
     (0,0) grid (10,10);
 \path [draw,->,name path=eje x] (-.3,0) --
    (10.3,0) node [right] {$t\left(s\right)$
    };
 \path [draw,->,name path=eje y] (0,-.3) --
    (0,10.3) node [above] {$d\left(m\right)$
    };
\end{tikzpicture}
```

Se añaden dos opciones más a la instrucción `\draw` para dibujar la cuadrícula. La primera, el color `black!5!white` para la líneas de la cuadrícula de `step=0.1cm` ó cuadrícula pequeña, el color `black!15!white` para la líneas de la cuadrícula de `step=0.5cm` ó cuadrícula mediana y el color `black!20!white` para la líneas de la cuadrícula de `step=1cm` ó cuadrícula grande.

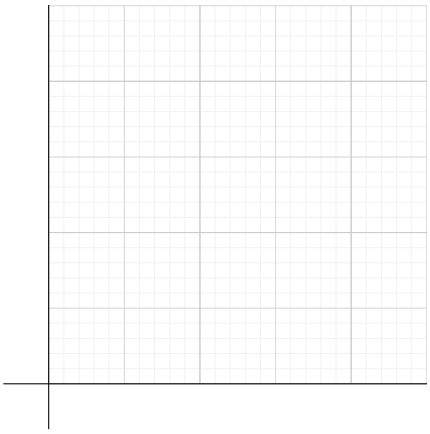
Es de importancia mencionar, que el color `black!5!white` significa la mezcla de 5 % de negro y 95 % de blanco, y esta interpretación se aplica de manera análoga a las cuadrículas de tamaño mediano y grande. En segundo lugar, se ha reducido el grosor de las líneas

de todas las cuadrículas, desde la más pequeña hasta la más grande, estableciéndolas en `very thin`.

Por último, se ha organizado la secuencia de comandos de tal manera que primero se dibuje la cuadrícula de menor tamaño, seguida por la cuadrícula mediana, y finalmente la cuadrícula de mayor tamaño. Esto asegura que todas las demás representaciones gráficas se superpongan a estas cuadrículas.

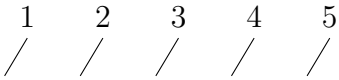
4.1.3 Tarjas

Con fin de ahorrar espacio en este manual, se recortará el dibujo final para visualizar las partes "*más interesantes*"del mismo. Para realizar un recorte en Ti*k*Z, puede utilizar el comando `\clip [opciones] ⟨operación de ruta⟩;` si se agrega en la operación de ruta la instrucción `(coordenada 1) rectangle (coordenada 2)`, de modo que Ti*k*Z recorta todo lo el dibujo que este fuera de la operación de ruta. Continuando con el ejemplo, se obtiene el siguiente dibujo.



```
\begin{tikzpicture}
 \draw [step=0.1cm,black!5!white,very thin]  (0,0)
     grid (10,10);
 \draw [step=0.5cm,black!15!white,very thin] (0,0)
     grid (10,10);
 \draw [step=1cm,black!20!white,very thin]   (0,0)
     grid (10,10);
 \path [draw,->,name path=eje x] (-.3,0) --
    (10.3,0) node [right] {$t\left(s\right)$};
 \path [draw,->,name path=eje y] (0,-.3) --
    (0,10.3) node [above] {$d\left(m\right)$};
\end{tikzpicture}
```

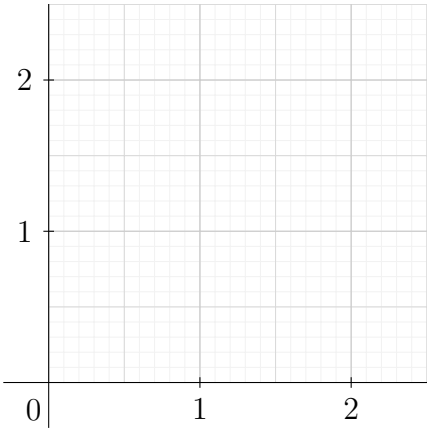
Lo siguiente a dibujar, son las tarjas `-1,0,1,2,...,10` en los ejes cartesianos. Es importante observar, que las tarjas se repiten a lo largo del `eje x` y el `eje y`. Por lo cual, sería bueno utilizar un buble para evitar construir una a una estas tarjas. En su forma básica, Ti*k*Z incluye el comando `\foreach \macro in {lista} ⟨codigo⟩`. Por ejemplo.




```
\foreach \x in {1,2,3,4,5}
\draw [xshift=\x cm] (0,0) -- (.3,.5) node [above] {\x};
```

Un aspecto crucial en el proceso anterior radica en la definición de la macro `\x` y los valores `1,2,3,4,5` que la macro `\x` tomará como punto de partida para el bucle. En consecuencia, se utilizará la instrucción `\draw` para trazar una ruta desde `(0,0)` hasta `(.3,.5)`. Sobre la última coordenada de esta ruta, se incorporará un nodo que contendrá los elementos de la macro `\x`, desde el primero hasta el último elemento, marcando así el final del bucle. Además, se ha incluido la opción `xshift= \x cm` para desplazar el dibujo `\x` unidades de distancia en la dirección horizontal.

En este contexto, puede implementar en este comando a la plantilla para dibujar las tarjas en el `eje x` y el `eje y`.




```
\begin{tikzpicture}[scale=2]
 \clip [] (-.5,-.5) rectangle (2.5,2.5);
 \draw [step=0.1cm,black!5!white,very thin]  (0,0)
     grid (10,10);
 \draw [step=0.5cm,black!15!white,very thin] (0,0)
     grid (10,10);
 \draw [step=1cm,black!20!white,very thin]   (0,0)
     grid (10,10);
 \path [draw,->,name path=eje x] (-.3,0) --
    (10.3,0) node [right] {$t\left(s\right)$};
 \path [draw,->,name path=eje y] (0,-.3) --
    (0,10.3) node [above] {$d\left(m\right)$};
 \foreach \x in {1,2,3,...,10} \draw (\x,1pt) --
    (\x,-1pt) node[below] {\x};
 \foreach \y in {1,2,3,...,10} \draw (1pt,\y) --
    (-1pt,\y) node[left] {\y};
 \node [below] at (-.1,-.04) {$0$};
\end{tikzpicture}
```

Observación: Nótese, que no se incluyó el `0` dentro de los elementos de la macro `\x` ó `\y`, para evitar superposiciones de sus correspondientes nodos. La solución se resume en colocar un `\node` individual en la posición `(-.1,-.04)` para representar la tarja del `0`.

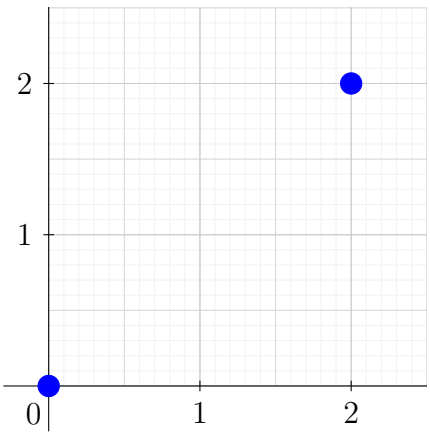
4.1.4 Datos experimentales

El siguiente desafío consiste en representar los datos experimentales en un plano cartesiano. Para lograr esto, es posible utilizar marcadores en forma de círculos, rectángulos, cruces, u otras formas pequeñas que representen la posición de los datos experimentales en el plano. Por ejemplo, si en una primera toma de datos se encuentra que la variable independiente tiene un valor de `1` y la variable dependiente un valor de `4`, la coordenada del marcador que representa este dato experimental será `(1,4)`. Con esta breve introducción, usted podrá construir pares ordenados en función de los valores que toman las variables. Para lograrlo, se utiliza el concepto de iteración en múltiples variables.

$(1,1)\ (2,2)\ (3,3)\ (4,4)\ (5,5)$

```
\foreach \x/\y in {1/1,2/2,3/3,4/4,5/5} \node at (\x,0) {$
    (\x,\y)$};
```

`\foreach \macro1/\macro2/... in {list1/list2,...}` ⟨`operación de ruta`⟩ es un comando que permite la iteración sobre múltiples variables. El funcionamiento de esta instrucción es similar al del bucle `foreach` para una sola variable.




```
\clip [] (-.5,-.5) rectangle (2.5,2.5);
 \draw [step=0.1cm,black!5!white,very thin]  (0,0)
     grid (10,10);
 \draw [step=0.5cm,black!15!white,very thin] (0,0)
     grid (10,10);
 \draw [step=1cm,black!20!white,very thin]   (0,0)
     grid (10,10);
 \path [draw,->,name path=eje x] (-.3,0) --
    (10.3,0) node [right] {$t\left(s\right)$};
 \path [draw,->,name path=eje y] (0,-.3) --
    (0,10.3) node [above] {$d\left(m\right)$};
 \foreach \x in {1,2,3,...,10} \draw (\x,1pt) --
    (\x,-1pt) node[below] {\x};
 \foreach \y in {1,2,3,...,10} \draw (1pt,\y) --
    (-1pt,\y) node[left] {\y};
 \node [below] at (-.1,-.04) {$0$};
 \foreach \listx/\listy in {0/0,2/2,8/8,10/10}
 \draw [blue, fill] (\listx,\listy) circle [radius
    =2pt];
```

En el primer conjunto, la lista de valores de la variable independiente se asocia con la

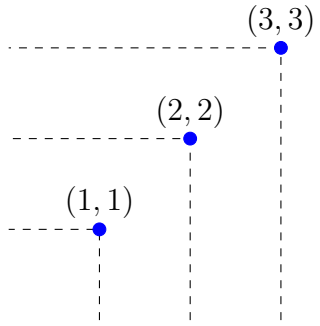
macro `\listx`, mientras que la segunda lista, con los valores de la variable dependiente, se relaciona con la macro `\listy`, separadas por una barra. Finalmente, la operación de ruta se encarga de dibujar un marcador circular con un radio de `2 pt` utilizando la instrucción `circle` y la opción `radius=2pt`, en la coordenada `(\listx,\listy)`, que itera sobre cada valor de la lista, permitiendo obtener las coordenadas de los datos experimentales.

Este enfoque resulta muy útil para representar datos experimentales de manera eficiente y precisa en un gráfico.

4.1.5 Rastros

Se entiende como rastro.ª la unión de la coordenada del dato experimental con la marca en el eje x y en el eje y mediante líneas discontinuas. Para llevar a cabo esta tarea, se pueden elegir entre dos métodos: rutas individuales mediante el comando `\draw` o rutas repetidas utilizando el comando `\foreach`.

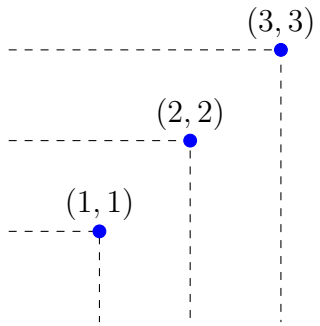
Rutas individuales: Consisten en trazar una ruta desde el eje x hasta el dato experimental (x, y) y luego continuar hacia el eje y. En otras palabras, se generan dos segmentos de línea, el primero paralelo al eje y y el segundo paralelo al eje x.




```
\begin{tikzpicture}
 \draw [dashed] (1,0) -- (1,1) -- (0,1);
 \draw [dashed] (2,0) -- (2,2) -- (0,2);
 \draw [dashed] (3,0) -- (3,3) -- (0,3);
 \draw [blue, fill] (1,1) node [above,black] {$(1,1)$}
    circle [radius=2pt];
 \draw [blue, fill] (2,2) node [above,black] {$(2,2)$}
    circle [radius=2pt];
 \draw [blue, fill] (3,3) node [above,black] {$(3,3)$}
    circle [radius=2pt];
\end{tikzpicture}
```

A pesar de su eficacia en otras situaciones en las que es necesario manipular individualmente cada ruta, este método resulta poco conveniente para el desarrollo de una plantilla. Implicaría generar una línea de código por cada ruta, lo cual podría resultar en una extensión innecesaria del código, especialmente cuando se trabaja con un gran número de datos experimentales.

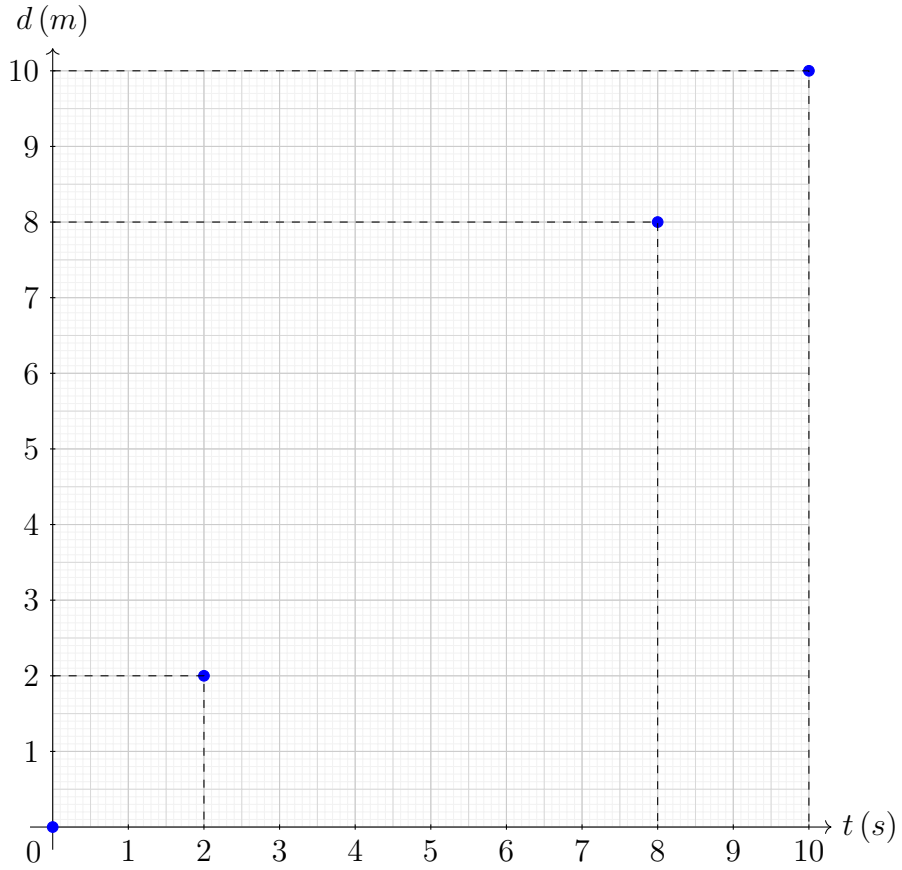
Rutas repetidas: Este método permite reducir todas las líneas de código del método anterior a tan solo cuatro líneas, independientemente del número de datos experimentales con los que se esté trabajando.




```
\begin{tikzpicture}
 \foreach \x/\y in {1/1,2/2,3/3} {
  \draw [dashed] (0,\y) -- (\x,\y) -- (\x,0);
  \draw [blue, fill] (\x,\y) circle [radius=2pt];
  \node [above] at (\x,\y) {$(\x,\y)$};
 }
\end{tikzpicture}
```

Ahora, la operación de ruta del comando `foreach` consta de tres instrucciones. En primer lugar, la instrucción `\draw` se utiliza para trazar los rastros de las coordenadas de los datos experimentales. Para hacer que la línea sea discontinua, se agrega la opción `dashed`. A continuación, el comando `\draw` con las opciones `blue,fill` se encargan de

dibujar el marcador. Por último, el comando `\node` con la opción `above` coloca un texto `$(\x,\y)$` por encima del valor de la coordenada del dato experimental.



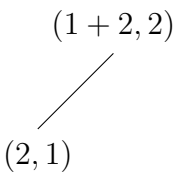

```
\begin{tikzpicture}
 \draw [step=0.1cm,black!5!white,very thin]  (0,0) grid (10,10);
 \draw [step=0.5cm,black!15!white,very thin] (0,0) grid (10,10);
 \draw [step=1cm,black!20!white,very thin]   (0,0) grid (10,10);
 \path [draw,->,name path=eje x] (-.3,0) -- (10.3,0) node [right] {$t\left(s\right)$};
 \path [draw,->,name path=eje y] (0,-.3) -- (0,10.3) node [above] {$d\left(m\right)$};
 \foreach \x in {1,2,3,...,10} \draw (\x,1pt) -- (\x,-1pt) node[below] {\x};
 \foreach \y in {1,2,3,...,10} \draw (1pt,\y) -- (-1pt,\y) node[left] {\y};
 \node [below] at (-.25,-.04) {$0$};
 \foreach \listx/\listy in {0/0,2/2,8/8,10/10} {
  \draw [blue, fill] (\listx,\listy) circle [radius=2pt];
  \draw [dashed] (0,\listy) -- (\listx,\listy) -- (\listx,0);
 }
\end{tikzpicture}
```

4.1.6 Recta de regresión lineal

El siguiente desafío es determinar la recta de regresión lineal utilizando el concepto de mínimos cuadrados. Inicialmente, esto se puede lograr mediante comandos de TEX o LATEX. Por ejemplo, supongamos que definimos tres variables: $a = 1$, $b = 2$, y luego calculamos $c = a + b$. A continuación, trazamos una línea con el comando `\draw` desde `(2,1)` hasta `(3,2)`. Sin embargo, dada la definición de a, b, y c, también podríamos trazar la línea desde `(b,a)` hasta `(c,b)`.

```
\def\a{1}
\def\b{2}
\def\c{\a+\b}
\begin{tikzpicture}
 \draw (\b,\a) -- (\c,\b);
\end{tikzpicture}
```

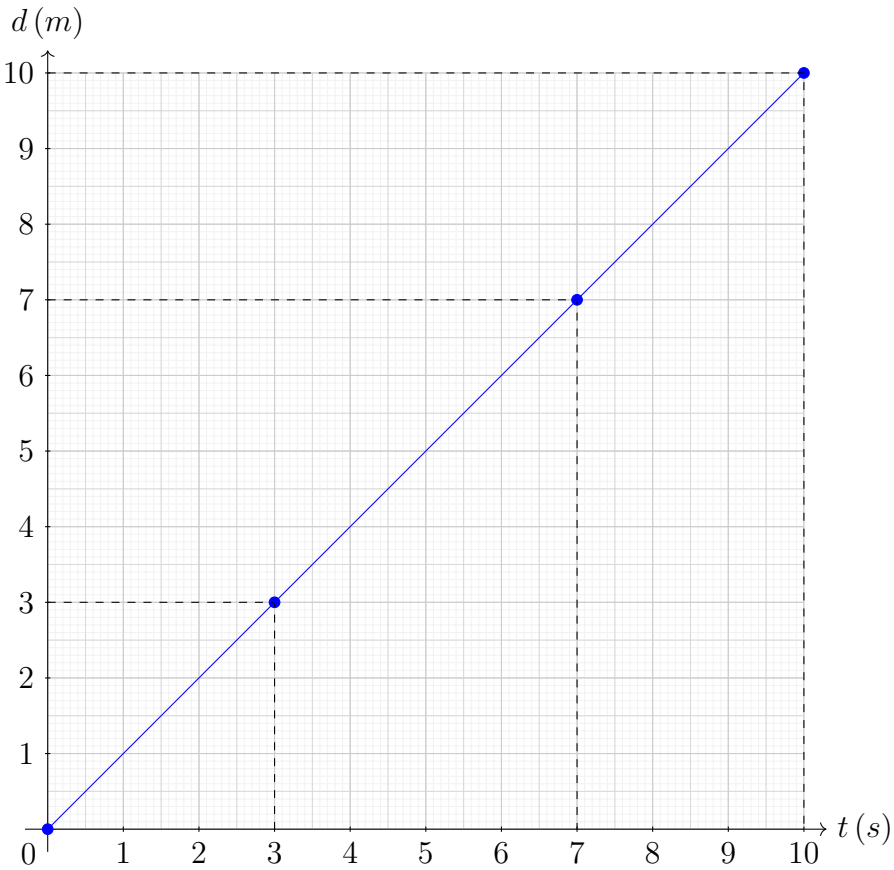
En este contexto, podemos utilizar la operación `{\a+\b}` encerrada en llaves para calcular c, y el resultado 3 se almacena en la macro `\c`.




```
\def\a{1}
\def\b{2}
\def\c{\a+\b}
\begin{tikzpicture}
 \draw (\b,\a) node [below] {$(\b,\a)$}--(\c,\b)node [
    above] {$(\c,\b)$};
\end{tikzpicture}
```

Sin embargo, al tratar de imprimir el valor de las macros `\a`, `\b` o `\c` para fines ilustrativos u otros, se presentan limitaciones en las estructuras de LaTeX y TeXÂunque es posible abordar estos errores mediante comandos internos de TeX, no resulta eficiente ni productivo.

Por este motivo, optaremos por trazar la recta $y = 1x + 0$ a modo ilustrativo y abordaremos la determinación de la recta de regresión lineal en el siguiente capítulo, incorporando el lenguaje de programación **R** en la plantilla.




```
\begin{tikzpicture}
 \draw [step=0.1cm,black!5!white,very thin]  (0,0) grid (10,10);
 \draw [step=0.5cm,black!15!white,very thin] (0,0) grid (10,10);
 \draw [step=1cm,black!20!white,very thin]   (0,0) grid (10,10);
```

```
\path [draw,->,name path=eje x] (-.3,0) -- (10.3,0) node [right] {$t\left(s\right)$};
\path [draw,->,name path=eje y] (0,-.3) -- (0,10.3) node [above] {$d\left(m\right)$};
\foreach \x in {1,2,3,...,10} \draw (\x,1pt) -- (\x,-1pt) node[below] {\x};
\foreach \y in {1,2,3,...,10} \draw (1pt,\y) -- (-1pt,\y) node[left] {\y};
\node [below] at (-.25,-.04) {$0$};
\foreach \listx/\listy in {0/0,3/3,7/7,10/10} {
 \draw [blue, fill] (\listx,\listy) circle [radius=2pt];
 \draw [dashed] (0,\listy) -- (\listx,\listy) -- (\listx,0);
}
\path [draw,blue,name path=regre,domain=0:10] plot (\x,1*\x+0);
\end{tikzpicture}
```

La instrucción `-- plot[opciones locales] ⟨expresión de coordenadas⟩` se utiliza en conjunto con el comando `\path` para dibujar la recta $f(x) = x$. En la parte de `⟨expresión de coordenadas⟩`, se debe definir una macro, por ejemplo, la macro `\x`, que se establece en los diferentes resultados de la expresión `\x` a medida que se evalúa. En otras palabras, si la `⟨expresión de coordenadas⟩` fuese `(\x,\x+1)`, primero se evalúa la expresión `\x+1`, y los resultados se almacenan en la macro `\x`. Luego, `\path` utiliza estos valores para trazar la ruta con las siguientes opciones:

1. `[draw]`: Dibuja la ruta
2. `[blue]`: Asigna el color azul a la ruta trazada.
3. `[name path=⟨name⟩]`: Asigna un nombre a la ruta trazada.
4. `domain=⟨start⟩:⟨end⟩`: Establece el dominio del cual se toman los valores para evaluar la expresión. Si no se asigna ningún dominio, Ti*k*Z toma los valores de -5 hasta 5.

Finalmente, se modifican los elementos de las listas `\listx` y `\listy` con el propósito de centrar el triángulo de inspección que se construirá en la siguiente sección.

4.1.7 Triángulo de inspección

El tríangulo de inspección permite visualizar la tasa de cambio Δ de las variables del `eje x` y del `eje y`. En otras palabras, $\Delta t = t_f - t_i$ para el `eje x`, y $\Delta d = d_f - d_i$ para el `eje y`.

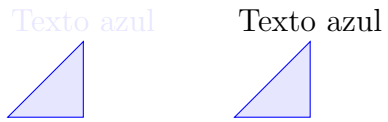
Para ello, se debe tomar en cuenta los siguientes comandos y opciones:

El comando `-- cycle` cierra la operación de ruta, uniedo suavemente el último punto con el primer punto. Para apreciar, la diferencia considere el siguiente ejemplo:

Sin `-- cycle` Con `-- cycle`

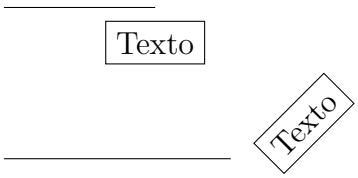
```
\begin{tikzpicture}[line width=5pt]
 \draw (0,0) -- (1,0) -- (1,1) -- (0,0);
 \node at (.5,-.5) {Sin \verb|-- cycle|};
 \draw (3,0) -- (4,0) -- (4,1) -- cycle;
 \node at (3.5,-.5) {Con \verb|-- cycle|};
\end{tikzpicture}
```

La opción `[...fill opcity = ⟨valor⟩]` establece la opacidad de los rellenos, incluyendo el texto. Si desea cambiar el color del texto, se debe agregar la opción `...,color` a la operación `... node`. Por ejemplo:




```
\begin{tikzpicture}
 \draw [blue, fill=blue, fill opacity=0.1]
    (0,0) -- (1,0) -- (1,1) node [above] {Texto
     azul} -- cycle;
 \draw [blue, fill=blue, fill opacity=0.1]
    (3,0) -- (4,0) -- (4,1) node [above,black,
    fill opacity=1] {Texto negro} -- cycle;
\end{tikzpicture}
```

La opción `[...rotate=⟨grado⟩]` gira el sistema de coordenadas por grados. Ahora, si quiero que el nodo se desplace una distancia específica, hacia arriba `above=⟨valor⟩`, abajo `below=⟨valor⟩`, izquierda `left=⟨valor⟩`, derecha `right=⟨valor⟩`. Por ejemplo:



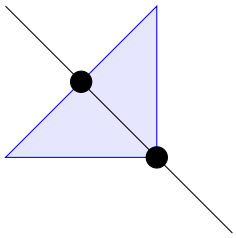

```
\begin{tikzpicture}
 \draw (0,0) -- (2,0) node [fill=white,draw=black,
    below=5pt] {Texto};
 \draw (0,-2) -- (3,-2) node [fill=white,draw=black,
    right=.5cm,rotate=45] {Texto};
\end{tikzpicture}
```

La opción `... name intersections={opciones}`, las `{opciones}` determinan entre otras cosas que caminos utilizar para la intersección. Una vez procesadas las `{opciones}`, se encuentran las posibles intersecciones. Se crea una coordenada en cada intersección.

Para las opciones se pueden utilizar las siguientes claves:

1. `{of=⟨name path 1⟩ and ⟨name path 2⟩}`: Esta clave se utiliza para especificar los nombres de las rutas que se utilizarán para la intersección.
2. `{...,name=⟨prefijo⟩}`: Especifica el nombre del prefijo para los nodos de coordenadas colocados en cada intersección.
3. `{...,by=⟨lista separada por comas⟩}`: Especificar una lista de nombres para cada coordenada de intersección encontrada.

El siguiente ejemplo, ilustra los beneficios de esta opción:

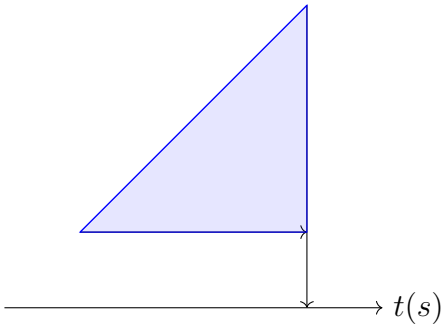


```
\begin{tikzpicture}[scale=2]
 \path [draw=blue, fill=blue, fill opacity=0.1, name
     path = triangle] (3,0) -- (4,0) -- (4,1) --
    cycle;
 \path [draw=black, name path=line] (4.5,-.5) --
    (3,1);
 \draw [name intersections={of=triangle and line, by
    ={a,b}},fill=black]
  (a) circle [radius=2pt]
  (b) circle [radius=2pt];
\end{tikzpicture}
```

Un caso especial frecuente de intersecciones de caminos es la intersección de una línea vertical que pasa por un punto p y una recta horizontal que pasa por algún otro punto q. Para esta situación existe un sistema de coordenadas útil.

1. `(⟨q⟩-| ⟨p⟩)`: Significa que la ruta ⟨q⟩ se extiende horizontalmente hasta intersecciona con la ruta ⟨p⟩ que se extiende verticalmente.
2. `(⟨q⟩|- ⟨p⟩)`: Significa que la ruta ⟨q⟩ se extiende verticalmente hasta interseccionar con la ruta ⟨p⟩ que se extiende horizontalmente.

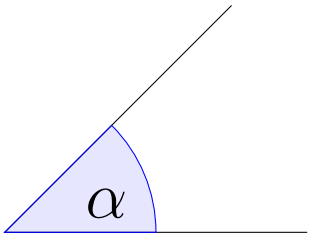
El siguiente ejemplo ilustra de mejor manera la sintaxis de esta opción:




```
\begin{tikzpicture}
 \draw [->] (0,0) -- (5,0) node (ejex) [right] {$t
    (s)$};
 \draw [blue] (1,1) coordinate (a) -- (4,4)
    coordinate (b);
 \path [draw,black,->] (b) -- (b|-ejex) coordinate
     (c);
 \path [draw,black,->] (a) -- (a-|c) coordinate (d
    );
 \draw [blue,fill=blue,fill opacity=0.1] (a) -- (b
    ) -- (d) -- cycle;
\end{tikzpicture}
```

Para agregar un arco de circunferencia a cualquier ruta se debe utilizar la operación `\path...arc[⟨opciones⟩]...;`. Se puede describir en `⟨opciones⟩` las siguientes claves:

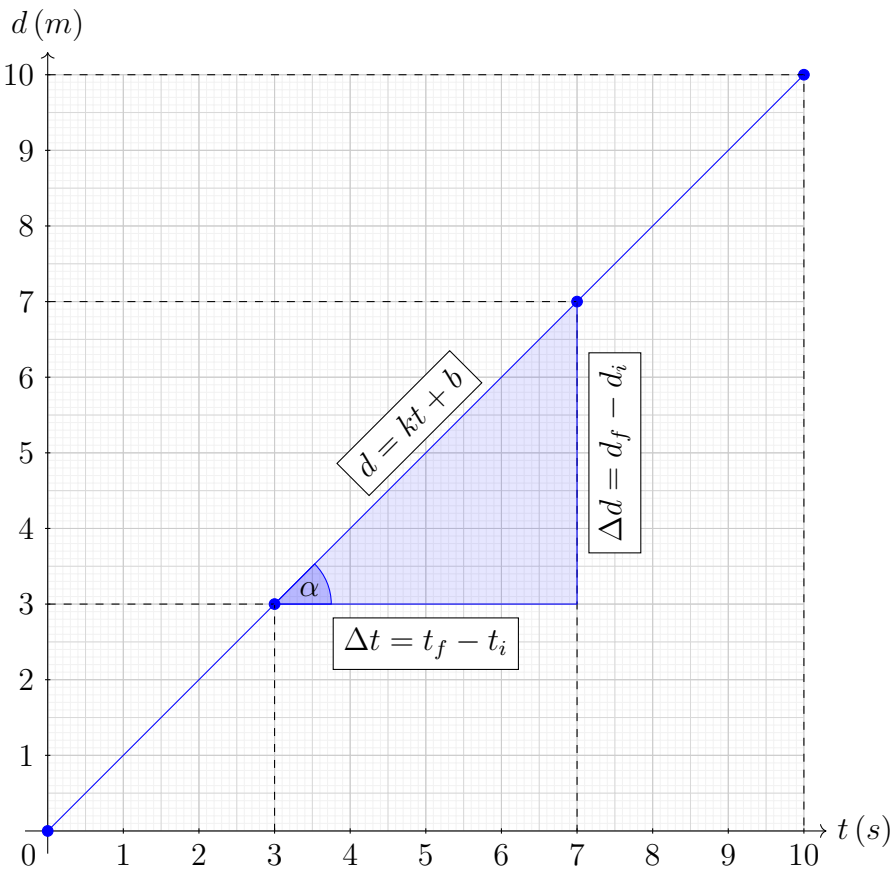
1. `[start angle=⟨grados⟩]`: Establece el valor del ángulo inicial.
2. `[end angle=⟨grados⟩]`: Establece el valor del ángulo final.
3. `[radius=⟨valor⟩]`: Establece el valor del radio del arco de circunferencia.




```
\begin{tikzpicture}
 \draw (3,3) -- (0,0) -- (4,0) ;
 \path [draw=blue,fill=blue,fill opacity=0.1]
    (0,0) -- (2cm,0cm) arc [start angle=0, end
    angle=45, radius=2cm] -- cycle;
 \path (0,0) -- (45/2:1cm) node [right] {\Huge $\
    alpha$};
\end{tikzpicture}
```

En concecuencia, se debe dibujar primero el tríangulo rectángulo, donde sus dos primeros vértices están determinados por las coordenadas de los datos experimentales, es decir, el primer vértice $(3, 3)$ y el segundo vértice $(7, 7)$, y el tercer vértice se construye a partir de la componente 7 del segundo vértice y de la componente 3 del primer vértice, siendo el resultado la coordenada $(7, 3)$.

Sin embargo, este procedimiento no es necesariamente correcto, porque, se conoce las coordenas $(3, 3)$ y $(7, 7)$ pero no se conoce las coordenadas de la recta de regresión lineal. En otras palabras, se deconoce $f(3)$, $f(7)$, $(3, f(3))$ y $(7, f(7))$. Con estas bases, se puede seguir con la construcción de la plantilla.



```
\begin{tikzpicture}
 \draw [step=0.1cm,black!5!white,very thin]  (0,0) grid (10,10);
 \draw [step=0.5cm,black!15!white,very thin] (0,0) grid (10,10);
 \draw [step=1cm,black!20!white,very thin]   (0,0) grid (10,10);
 \path [draw,->,name path=ejex] (-.3,0) -- (10.3,0) node [right] {$t\left(s\right)$};
 \path [draw,->,name path=ejey] (0,-.3) -- (0,10.3) node [above] {$d\left(m\right)$};
 \foreach \x in {1,2,3,...,10} \draw (\x,1pt) -- (\x,-1pt) node[below] {\x};
 \foreach \y in {1,2,3,...,10} \draw (1pt,\y) -- (-1pt,\y) node[left] {\y};
 \node [below] at (-.25,-.04) {$0$};
 \foreach \listx/\listy in {0/0,3/3,7/7,10/10} {
  \draw [blue, fill] (\listx,\listy) circle [radius=2pt];
  \draw [dashed] (0,\listy) -- (\listx,\listy) -- (\listx,0);
 }
 \path [draw,blue,name path=regresion,domain=0:10] plot (\x,1*\x+0);
 % Inicio de comandos de construccion
 \path (7,7) coordinate (a) -- (a|-ejex) coordinate (b);
 \path (3,3) coordinate (c) -- (c-|a) coordinate (d);
 \path (0,10) -- (10,10) coordinate (e);
 \path (c) -- (c|-e) coordinate (f);
 \path (a) -- (a-|f) coordinate (g);
 \path (a) -- (a-|f) coordinate (g);
 \path [name path=diagonal] (g) -- (d);
 \path [name intersections={of=diagonal and regresion,by=interseccion1}] (interseccion1)
    ;
 \path (interseccion1) -- (interseccion1-|b) coordinate (h);
 \path (interseccion1) -- (interseccion1|-d) coordinate (i);
 % Fin de comandos de construccion
 \draw [blue] (a) -- (d);
 \draw [blue] (c) -- (d);
 \path [fill=blue,fill opacity=0.1] (a) -- (d) -- (c) -- cycle;
 \node [draw=black,fill=white,rotate=45,above=5pt] at (interseccion1) {$d=kt+b$};
 \node [draw=black,fill=white,below=5pt] at (i) {$\Delta t=t_f-t_i$};
 \node [draw=black,fill=white,right=0.5cm,rotate=90,anchor=center] at (h) {$\Delta d=d_f
    -d_i$};
 \draw [draw=blue,fill=blue,fill opacity=0.2] (c) -- ++(.75cm,0cm) arc [start angle=0,
    end angle=45,radius=.75cm] -- (c);
 \path (c) -- ++(45/2:0.5cm) node {$\alpha$};
\end{tikzpicture}
```

4.1.8 Leyenda

Dentro de un entorno `{tikzpicture}` puede crear ámbitos utilizando el entorno `{scope}`. Este entorno sólo está disponible dentro del entorno `{tikzpicture}`. En decir, un mini entorno de dibujo secundario, que no es independiente del dibujo primario.

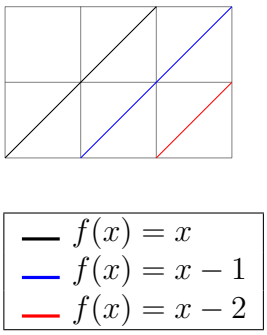
```
% Iniciamos el entorno de dibujo:
\begin{tikzpicture}[opciones]
 % Contenido del dibujo primario (Diagrama).
 \begin{scope}[opciones]
  % Contenido del dibujo secundario (Leyenda).
 \end{scope}
\end{tikzpicture}
```

El siguiente paso es desplazar el entorno `{scope}` al lado derecho del dibujo primario o diagrama. Para ello, se debe hacer uso de las siguientes opciones:

1. `[x shift=⟨dimension⟩]`: Agrega `⟨dimension⟩` al valor x de todas las cordenadas.
2. `[y shift=⟨dimension⟩]`: Agrega `⟨dimension⟩` al valor y de todas las cordenadas.

Asimismo, se puede configurar dentro de `{contenido}` de la sintaxis del comando o ruta `\path ... node ⟨foreach⟩ [opciones] (nombre) {contenido}...;` un entorno `{tabular}` para que la tabla sea parte del dibujo primario.

Suponga dibujar tres ecuaciones: $f(x) = x$, $f(x) = x - 1$ y $f(x) = x - 2$, y desea que el nombre de las ecuaciones aparezca en una tabla, entonces usted podría utilizar un código similar al siguiente:



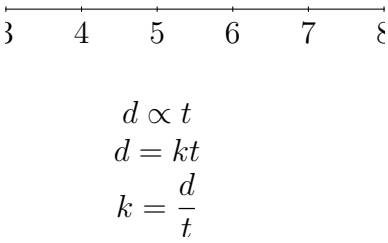

```
\begin{tikzpicture}
\draw[help lines] (0,0) grid (3,2);
\draw[black,domain=0:2] plot (\x,\x);
\draw[blue,domain=1:3] plot (\x,\x-1);
\draw[red,domain=2:3] plot (\x,\x-2);
\begin{scope}[xshift=1.7cm,yshift=-.5cm]
\node at (0,-1) {
 \begin{tabular}{|l|}\hline
 \tikz \draw [black,very thick] (0,0) -- (.5,0); $f(x)=x$\\
 \tikz \draw [blue,very thick] (0,0) -- (.5,0); $f(x)=x-1$\\
 \tikz \draw [red,very thick] (0,0) -- (.5,0); $f(x)=x-2$\\
 \hline
 \end{tabular}};
\end{scope}
\end{tikzpicture}
```

Transpolando estos conceptos al desarrollo de la plantilla, se obtiene una estructura similar a la siguiente:

```
\begin{tikzpicture}
 ...
 \begin{scope}[x shift=valor cm, y shift=valor cm]
 \node at (x,y) {
  \begin{tabular}[posicion]{formato}
   xxx & xxx & ... & xxx \\
   xxx & xxx & ... & xxx \\
   ... & ... & ... & ... \\
   xxx & xxx & ... & xxx \\
  \end{tabular}}
 \end{scope}
\end{tikzpicture}
```

Observación Se puede trabajar con el entorno `{tabular}` de forma independiente al dibujo del entorno `{tikzpicture}`, haciendo uso de entornos como: `{minipage}`, `{figure}`, entre otros.

Además, deben incluir un nodo debajo del `ejex` y centrado que incluya la proporcionalidad entre las variables analizadas; lo llamaremos *nodo de proporcionalidad*. Para ello, deben aplicar los conceptos aprendidos en esta subsección.




```
\begin{tikzpicture}
 \clip[] (3,-3) rectangle (8,.5);
 \draw (0,0) -- (10,0);
 \foreach \x in {0,1,...,10} \draw (\x,1pt)
    -- (\x,-1pt) node [below] {\x};
 \node [below=1cm] at (5,0) {
  \begin{tabular}{c}
   $d \propto t $ \\
   $d=kt$ \\
   $k=\displaystyle \frac{d}{t}$
  \end{tabular}
 };
\end{tikzpicture}
```

Puedes preguntarte por qué la coordenada del nodo es `(5,0)`. Utilizando un poco de matemática elemental, podemos deducir que todo el *eje* x tiene una longitud de 10 cm. Por lo tanto, para centrar el nodo de proporcionalidad, dividimos la longitud del *eje* x por 2, lo que nos da 5. Este resultado es la componente x de la coordenada del nodo de proporcionalidad.

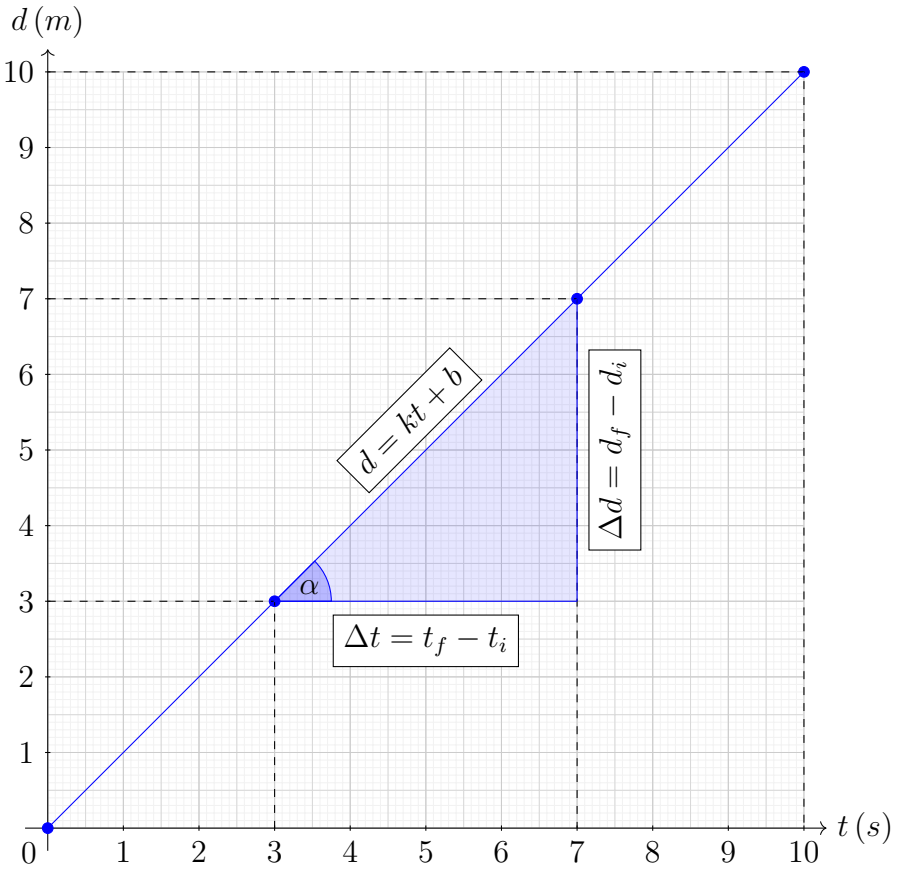
4.1.9 Título del diagrama

Finalmente, queda por incluir un título $d = f(t)$ ó d vs t para el diagrama de dispersión. Afortunadamente, se puede revisar la documentación del paquete {`titlesec`} y modificar los formatos de los títulos según sus preferencias.

Entonces, con todos los procesos completados hasta este punto, deben tener un resultado similar al siguiente:

ANEXO 1

Gráfico 1: $d = f(t)$



Escala
$t : 1\,(s) \cong 1\ cm$ $d : 1\,(m) \cong 1\ cm$
Análisis Matemático
$m = \tan\alpha$ $k = \dfrac{\Delta d}{\Delta t} = \dfrac{7-3}{7-3}$ $k = 1$
Análisis Unidades
$k = \dfrac{\Delta d}{\Delta t} = \dfrac{m}{s}$ $k = v$
Análisis Dimensional
$k = \dfrac{\Delta d}{\Delta t} = \dfrac{[m]}{[s]}$ $k = [LT^{-1}]$

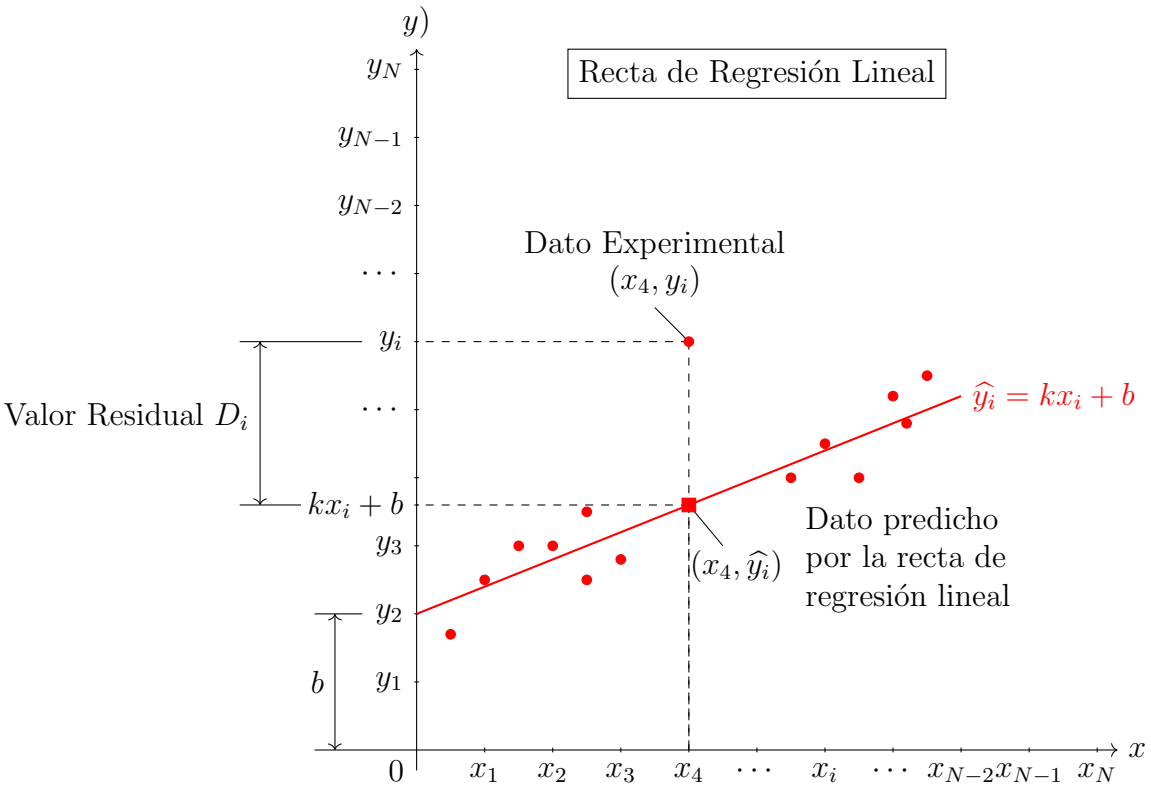
$$d \propto t$$
$$d = kt$$
$$k = \frac{d}{t}$$

Asimismo, sus líneas de código deben ser idénticas al código proporcionado a continuación, a menos que hayan optado por otro método de graficación explicado en líneas anteriores.

```
% PREAMBULO DEL DOCUMENTO
\documentclass[a4paper,12pt]{book}
% PAQUETES
\usepackage{amssymb,amsmath,geometry}
\usepackage[spanish]{babel}
\usepackage{tikz,titlesec,booktabs}
% LIBRERIAS DE TIKZ
\usetikzlibrary{babel,intersections}
% GEOMETRIA DEL DOCUMENTO
\geometry{bottom=2.54cm,top=2.54cm,left=1.3cm,right=1.3cm,headsep=1cm,head=1cm,foot=1cm,
marginparwidth=0cm,includeall=false}
% MODIFICACION DEL ESTILO DE LOS TITULOS
\titleformat{\section}[frame]
{\normalfont}{\filcenter\small
```

```
 \ ANEXO \thesection \ }
{7pt}{\Large\bfseries\filcenter}
% INICIO DEL DOCUMENTO
 \pagestyle{empty}
\begin{document}
 \section{Grafico 1: \ $d=f\left(t\right)$}
\begin{tikzpicture}
 \draw [step=0.1cm,black!5!white,very thin]  (0,0) grid (10,10);
 \draw [step=0.5cm,black!15!white,very thin] (0,0) grid (10,10);
 \draw [step=1cm,black!20!white,very thin]   (0,0) grid (10,10);
 \path [draw,->,name path=ejex] (-.3,0) -- (10.3,0) node (ejex) [right] {$t\left(s\right
    )$};
 \path [draw,->,name path=ejey] (0,-.3) -- (0,10.3) node (ejey) [above] {$d\left(m\right
    )$};
 \foreach \x in {1,2,3,...,10} \draw (\x,1pt) -- (\x,-1pt) node[below] {\x};
 \foreach \y in {1,2,3,...,10} \draw (1pt,\y) -- (-1pt,\y) node[left] {\y};
 \node [below] at (-.25,-.04) {$0$};
 \foreach \listx/\listy in {0/0,3/3,7/7,10/10} {
  \draw [blue, fill] (\listx,\listy) circle [radius=2pt];
  \draw [dashed] (0,\listy) -- (\listx,\listy) -- (\listx,0);}
 \path [draw,blue,name path=regresion,domain=0:10] plot (\x,1*\x+0);
 % Inicio de comandos de construccion
 \path (7,7) coordinate (a) -- (a|-ejex) coordinate (b);
 \path (3,3) coordinate (c) -- (c-|a) coordinate (d);
 \path (0,10) -- (10,10) coordinate (e);
 \path (c) -- (c|-e) coordinate (f);
 \path (a) -- (a-|f) coordinate (g);
 \path (a) -- (a-|f) coordinate (g);
 \path [name path=diagonal] (g) -- (d);
 \path [name intersections={of=diagonal and regresion,by=interseccion1}] (interseccion1)
    ;
 \path (interseccion1) -- (interseccion1-|b) coordinate (h);
 \path (interseccion1) -- (interseccion1|-d) coordinate (i);
 % Fin de comandos de construccion
 \draw [blue] (a) -- (d);
 \draw [blue] (c) -- (d);
 \path [fill=blue,fill opacity=0.1] (a) -- (d) -- (c) -- cycle;
 \node [draw=black,fill=white,rotate=45,above=5pt] at (interseccion1) {$d=kt+b$};
 \node [draw=black,fill=white,below=5pt] at (i) {$\Delta t=t_f-t_i$};
 \node [draw=black,fill=white,right=0.5cm,rotate=90,anchor=center] at (h) {$\Delta d=d_f
    -d_i$};
 \draw [draw=blue,fill=blue,fill opacity=0.2] (c) -- ++(.75cm,0cm) arc [start angle=0,
    end angle=45,radius=.75cm] -- (c);
 \path (c) -- ++(45/2:0.5cm) node {$\alpha$};
 \begin{scope}[xshift=14.25cm,yshift=5cm]
  \node at (0,0) {
   \begin{tabular}{|l|}\toprule
    Escala \\ \midrule
    $\displaystyle t: 1\left(s\right)\cong1 \ cm$ \\
    $\displaystyle d: 1\left(m\right)\cong1 \ cm$ \\ \midrule
    Analisis Matematico \\ \midrule
    $ m=\tan\alpha$  \\
    $\displaystyle k=\frac{\Delta d}{\displaystyle \Delta t}=\frac{7-3}{7-3}$ \\
    $k=1$ \\ \midrule
    Analisis Unidades \\ \midrule
    $\displaystyle k=\frac{\Delta d}{\displaystyle \Delta t}=\frac{\displaystyle m}{\
    displaystyle s}$ \\
    $k=v$ \\ \midrule
    Analisis Dimensional \\ \midrule
    $\displaystyle k=\frac{\Delta d}{\displaystyle \Delta t}=\frac{\displaystyle \left[m
    \right]}{\displaystyle\left[s\right]}$ \\ $k=\left[LT^{-1}\right]$ \\
    \bottomrule
  \end{tabular}};
 \end{scope}
 \node [below=1cm] at (5,0) {
  \begin{tabular}{c}
   $d \propto t $ \\
   $d=kt$ \\
   $k=\displaystyle \frac{d}{t}$
 \end{tabular}};
\end{tikzpicture}
\end{document}
% FIN DEL DOCUMENTO
```

RSweave




```
\begin{tikzpicture}
 \path [->,draw] (-1.5,0) -- (10.3,0) node (xaxis) [right] {$x$};
 \path [->,draw] (0,-0.3) -- (0,10.3) node (yaxis) [above] {$y)$};
 \foreach \x/\z in {1/$x_1$,2/$x_2$,3/$x_3$,4/$x_4$,5/$\cdots$,6/$x_i$,7/$\cdots$,8/$x_{
    N-2}$,9/$x_{N-1}$,10/$x_N$}
 \draw (\x,1pt) -- (\x,-1pt) node[anchor=north] {\z};
 \foreach \y/\v in {1/$y_1$,2/$y_2$,3/$y_3$,3.6/$kx_i+b$,4/,5/$\cdots$,6/$y_i$,7/$\cdots
    $,8/$y_{N-2}$,9/$y_{N-1}$,10/$y_N$}
 \draw (1pt,\y) -- (-1pt,\y) node[anchor=east] {\v};
 \node at (-.3,-.3) {0};
 \node[draw=black, fill=white,below=5pt] at (5,10.5) {Recta de Regresion Lineal};
 \coordinate (a) at (4,3.6);
 \coordinate (b) at (4,6);
 \draw [dashed] (a) -- (a|-xaxis);
 \draw [dashed] (a) -- (a-|yaxis);
 \draw [dashed] (b) -- (b|-xaxis);
 \draw [dashed] (b) -- (b-|yaxis);
 \draw (b) -- (3.5,6.5) coordinate (c) node [above] {$(x_4,y_i)$};
 \node at (c) [above=.5cm] {Dato Experimental};
 \draw (a) -- (4.5,3);
 \node at (4.7,2.7) {$(x_4,\widehat{y_i})$};
 \node at (5,2.8) [right=.5cm,text width=3cm] {Dato predicho por la recta de regresion
    lineal};
 \draw (-.8,2) -- (-1.5,2);
 \draw (-1.7,3.6) -- (-2.6,3.6);
 \draw (-.8,6) -- (-2.6,6);
 \draw [<->](-1.2,0) -- (-1.2,2);
 \draw [<->](-2.3,3.6) -- (-2.3,6);
 \node at (-2.3,4.9) [left] {Valor Residual $D_i$};
 \node at (-1.2,1) [left] {$b$};
 \draw [red, thick,domain=0:8] plot (\x,0.4*\x+2) node [right] {$\widehat{y_i}=kx_i+b$};
 \foreach \l/\p in {1/2.5,.5/1.7,1.5/3,2/3,2.5/3.5,2.5/2.5,3/2.8,5.5/4,6/4.5,6.5/4,7/
    5.2,7.5/5.5,7.2/4.8,4/6}{\draw [red,fill] (\l,\p) circle [radius=2pt];}
 \draw [red,fill] (3.9,3.5) rectangle (4.1,3.7);
\end{tikzpicture}
```

En el presente capítulo van a incorporar y definir funciones, definir varialbes, llamada de variables, llamada de funciones y realizar operaciones matemáticas en la plantilla desarrollada previamente. En primera instancia, deben seguir el siguiente procedimiento:

1. En la ruta `C:\Software_de_graficacion\Graficador` procede a crear dos archivos de texto con el nombre `base_de_datos_1` y `base_de_datos_2`, respectivamente.

2. En el archivo `base_de_datos_1.txt`, ingrese la magnitud, las unidades y la dimensión de la variable dependiente, independiente y de la constante de proporcionalidad separados por un espacio, de la siguiente forma.

   base_de_datos_1.txt: Bloc de notas

   ```
   magnitud t d v
   unidad s m m/s
   dimension T L LT{-1}
   ```

3. En el archivo `base_de_datos_2.txt`, ingrese los datos experimentales de la variable dependiente e independiente separados por un espacio, de la siguiente forma.

   base_de_datos_2.txt: Bloc de notas

   ```
   Independiente 0 1 2 3
   Dependiente 0 2 4 6
   ```

Una vez configurados estos archivos en la carpeta correspondiente, proceda a abrir el archivo `Plot4.Rnw`, que esta en la ubicación `C:\Software_de_graficacion\Graficador`, utilizando la aplicación **RStudio**. Luego, copien el código desarrollado en el archivo `Plot.tex` y compilen el archivo con el atajo de teclado `Ctrl+Shift+K`. Si el archivo PDF se genera sin problemas, pueden continuar con el manual. En caso contrario, les recomiendo revisar detenidamente las secciones anteriores para identificar y corregir cualquier error.

Incorporar R en la plantilla Sweave: El código de **R** se mezcla con el texto usando *chunks* de código, los cuales tiene una sintaxis similar a la siguiente:

```
% PREAMBULO DEL DOCUMENTO
\documentclass[a4paper,12pt]{book}
% PAQUETES
\usepackage{name}
% DOCUMENTO
\begin{document}
 <<echo=FALSE>>=
 Chunk de codigo
 @
\end{document}
```

Cualquier texto que quede después del signo = (en la misma línea) es ignorado por **R** al momento de ejecutar el código, es decir, se asume como un comentario.

Debido a que muchas de las veces al usuario no le interesa saber el código de programación se incluye el argumento `echo=FALSE` dentro de `<< >>` para indicarle a **Sweave** que no queremos el código en el documento o PDF final.

Extraer datos de los *chunks*: En este punto, usted talvez se preguntará ¿Cómo extraer los datos programados en **R**?. Para ello, se utiliza el comando `\Sexpr{variable}`. Por ejemplo:

$(4,2)$ $(6,2)$

```
...
\begin{document}
 <<echo=FALSE>>=
 a <- 2^2
 b <- 3-1
 c <- a+b
 @
 \begin{tikzpicture}
 \draw (\Sexpr{a},\Sexpr{b}) node {$(\Sexpr{a},\Sexpr{b})$} --
    (6,2) node {$(6,2)$};
 \end{tikzpicture}
\end{document}
```

Observaciones: Para evitar errores en la compilación del archivo, se debe considerar dos aspectos fundamentales:

1. Los *chunks* se declaran antes de llamar cualquier tipo de variable perteneciente al *chunk*. ¿Qué no debo hacer? Llamar la variable mediante el comando `\Sexpr{...}` y después declarar el *chunk*.

```
\begin{docuemt}
...
$x=\Sexpr{function}+2$
...
<<echo=FALSE>>=
funcion <- 22.3435^2
@
...
\end{docuemt}
```

2. Llamar con el comando `\Sexpr{...}` una variable que tenga más de un dato. Por ejemplo: vectores, matrices, tablas, arreglos, entre otros.

5.1 Plantilla RSweave: 4 datos

Todo el procedimiento de conexión[1], lo vamos a dividir en los siguientes procesos:

1. **Magnitudes:** Lectura del archivo `base_de_datos_1.txt` para extraer la magnitud, unidad y dimension de la variable independiente, dependiente y la constante de proporcionalidad.
2. **Datos variable independiente:** Lectura del archivo `base_de_datos_2.txt` para extraer los datos experimentales de la variable independiente, y determinar la escala adecuada.
3. **Datos variable dependiente:** Lectura del archivo `base_de_datos_2.txt` para extraer los datos experimentales de la variable dependiente, y determinar la escala adecuada.
4. **Función cero:** Construir una función que permita detectar un cero en las tarjas del eje x y del eje y.
5. **Regresión lineal:** Regresión lineal de los datos experimentales sin escalar mediante mínimos cuadrados.
6. **Tríangulo de inspección:** Determinar la variación en el $ejex$ y $ejey$, en función de los datos que conforman el triángulo de inspección.
7. **Análisis Matemático:** Determinar el valor de las variaciones de la variable dependiente e independiente, y el valor de la constante de proporcionalidad (pendiente).

[1] Conjuntos de procesos para enlazar código **R** con un documento LaTeX

5.1.1 Magnitudes: *Chunk 1*

R puede leer datos guardados en archivos de texto en formato tabla mediante la siguiente función `read.table` y crear un marco de datos a partir de el, a los cuales se pueden acceder.

```
datos1 <- read.table("base_de_datos_1.txt", header=FALSE)
```

Una vez **R** lee los datos de `base_de_datos_1.txt`, debe almacenarlos en algún "lugar" llamado `datos1`[2]. El argumento `header=FALSE` indica que el archivo no tiene una fila de encabezado con nombres de columnas.

El siguiente paso es extraer uno a uno los datos almacenados en la variable `datos1`.

```
datos1 <- read.table("base_de_datos_1.txt", header=FALSE)
vim <- datos1[1,2]
viu <- datos1[2,2]
vid <- datos1[3,2]
vdm <- datos1[1,3]
vdu <- datos1[2,3]
vdd <- datos1[3,3]
kpm <- datos1[1,4]
kpu <- datos1[2,4]
kpd <- datos1[3,4]
```

Estas líneas de código asignan valores específicos de `datos1` a las variables `vim`, `viu`, `vid`, `vdi`, `vdu`, `vdd`, `kpm`, `kpu` y `kpd`. Cada línea selecciona un valor de la matriz de datos `datos1` utilizando notación de índice `[fila, columna]`.

Para incorporar estás líneas de código, se hará uso de la sintaxis *chunk*. En este sentido se tiene la siguiente estructura:

```
...
% INICIO DEL DOCUMENTO
<<echo=FALSE>>=
datos1 <- read.table("base_de_datos_1.txt", header=FALSE)
vim <- datos1[1,2]
viu <- datos1[2,2]
vid <- datos1[3,2]
vdm <- datos1[1,3]
vdu <- datos1[2,3]
vdd <- datos1[3,3]
kpm <- datos1[1,4]
kpu <- datos1[2,4]
kpd <- datos1[3,4]
@
\begin{document}
...
\end{document}
```

Se puede extraer la variable `vim` del correspondiente *chunk* mediante el comando `\Sexpr{vim}`. De igual forma, las variables `viu, vid, vdi, ..., kpd` mediantes los comandos `\Sepr{viu},\Sepr{vid},\Sepr{vdm},...,\Sepr{kpd}`, respectivamente. Para reemplazar las magnitudes, unidades y dimensiones del código 4.1.9 pueden tomar en consideración el contenido de la Tabla 5.1.1.

[2]Variable que almacena los datos del archivo `base_de_datos_1.txt`

Variable de **R**	Reemplazo	Comando de LaTeX
vim	→	\Sexpr{vim}
viu	→	\Sexpr{viu}
vid	→	\Sexpr{vid}
...	...	...
kpu	→	\Sexpr{kpu}
kpd	→	\Sexpr{kpd}

Entonces, con los reemplazos correspondiente las líneas de código de la plantilla quedarían así:

```
...
% INICIO DEL DOCUMENTO
\pagestyle{empty}
<<echo=FALSE>>=
datos1 <- read.table("base_de_datos_1.txt", header=FALSE)
vim <- datos1[1,2]
viu <- datos1[2,2]
vid <- datos1[3,2]
vdm <- datos1[1,3]
vdu <- datos1[2,3]
vdd <- datos1[3,3]
kpm <- datos1[1,4]
kpu <- datos1[2,4]
kpd <- datos1[3,4]
@
\begin{document}
\section{Grafico 1: \Sexpr{vdm} \ $=f\left(\Sexpr{vim}\right)$}
...
\path [draw,->,name path=ejex] (-.3,0) -- (10.3,0) node (ejex) [right] {$\Sexpr{vim}\
   left(\Sexpr{viu}\right)$};
\path [draw,->,name path=ejey] (0,-.3) -- (0,10.3) node (ejey) [above] {$\Sexpr{vdm}\
   left(\Sexpr{vdu}\right)$};
...
\node [draw=black,fill=white,rotate=45,above=5pt] at (interseccion1) {$\Sexpr{vdm}=k\
   Sexpr{vim}+b$};
\node [draw=black,fill=white,below=5pt] at (i) {$\Delta \Sexpr{vim}=\Sexpr{vim}_f-\
   Sexpr{vim}_i$};
\node [draw=black,fill=white,right=0.5cm,rotate=90,anchor=center] at (h) {$\Delta \
   Sexpr{vdm}=\Sexpr{vdm}_f-\Sexpr{vdm}_i$};
...
$\displaystyle \Sexpr{vim}: 1\left(\Sexpr{viu}\right)\cong1 \ cm$ \\
$\displaystyle \Sexpr{vdm}: 1\left(\Sexpr{vdu}\right)\cong1 \ cm$ \\ \midrule
...
$\displaystyle k=\frac{\Delta \Sexpr{vdm}}{\displaystyle \Delta \Sexpr{vim}}=\frac
   {7-3}{7-3}$ \\
...
$\displaystyle k=\frac{\Delta \Sexpr{vdm}}{\displaystyle \Delta \Sexpr{vim}}=\frac{\
   displaystyle \Sexpr{vdu}}{\displaystyle \Sexpr{viu}}$ \\
$k=\Sexpr{kpm}$ \\ \midrule
...
$\displaystyle k=\frac{\Delta \Sexpr{vdm}}{\displaystyle \Delta \Sexpr{vim}}=\frac{\
   displaystyle \left[\Sexpr{vdu}\right]}{\displaystyle\left[\Sexpr{viu}\right]}$ \\ $k
   =\left[\Sexpr{kpd}\right]$ \\
...
$\Sexpr{vdm} \propto \Sexpr{vim} $ \\
$\Sexpr{vdm}=k\Sexpr{vim}$ \\
$k=\displaystyle \frac{\Sexpr{vdm}}{\Sexpr{vim}}$
...
\end{document}
```

Verificación: Para comprobar que se este ejecutando el código correctamente puede realizar lo siguiente:

1. Compilar el archivo con el comando `Ctrl+Shift+K`, y generar el archivo `Plot4.pdf` sin mayor complicación.

2. Imprimir algúna variable en la consola de **RStudio**, mediante el comando `print(x,...)`. Por ejemplo:

```
> print(vim)
[1] d
```

3. Abrir el archivo `base_de_datos_1.txt`, luego modificar las magnitudes, unidades, dimensiones y guardar los cambios realizados.

4. Compilar el archivo con el comando `Ctrl+Shift+K`, generar el archivo `Plot4.pdf` sin mayor complicación y verificar que se hayan modificado las magnitudes, unidades y dimensiones en correspondencia a sus cambios realizados en el archivo `base_de_datos_1.txt`.

5. Imprimir los nuevos cambios en la consola de **RStudio**. Por ejemplo:

```
> print(vim)
[1] Nueva magnitud
```

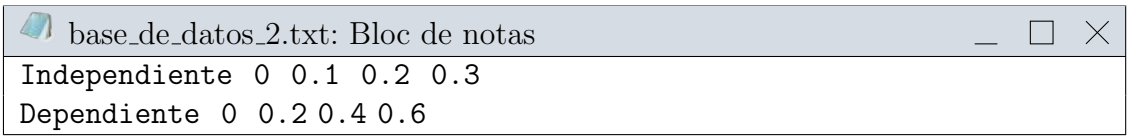
Observaciones: Por cada incorporación de *chunks* y llamada de variables se debe compilar el archivo mediante el comando `Ctrl+Shift+K`, para que el proceso de detección de errores seá optimo. Por ejmemplo, si incorporó el primer *chunk* y compilo el archivo `Plot4.Rnw` y se generá el archivo `Plot4.pdf` significa que no existen errores. Sin embargo, si deseo llamar una variable mediante el comando `\Sexpr{...}` y compilo nuevamente el archivo `Plot4.Rnw` y no se generá el archivo `Plot4.pdf` significa que se produjo un error, cuyos detalles aparencen en la consola de **RStudio**. Por lo tanto, para encontrar el error debo revisar cuidadosamente las modificaciones realizadas desde el proceso de incoporar el *chunk* hasta llamar la variable mediante el comando `\Sexpr{...}`, y no en todo el documento.

5.1.2 Datos variable independiente: *Chunk 2*

De igual forma, se debe leer el archivo `base_de_datos_2.txt` y extraer los datos experimentales de la variable independiente. Entonces, el código es el siguiente:

```
# LECTURA DE DATOS EJE X
datosvarx <- read.table("base_de_datos_2.txt", header=FALSE)
# DATOS SIN ESCALAR EJE X
x1 <- datosvarx[1,2]
x2 <- datosvarx[1,3]
x3 <- datosvarx[1,4]
x4 <- datosvarx[1,5]
```

Ahora bien, sea el caso donde los datos experimentales ingresados en el archivo de texto `base_de_datos_2.txt` sea similares a:



Es necesario determinar un factor de escala para el eje x. Para ello, se define la variable `datosvarxse` como un vector que concatena los datos `x1, x2, x3, x4` utilizando la función `c(...)`. A continuación, se define la función `valormaximo`, la cual toma como argumento un vector o un arreglo numérico y encuentra su valor máximo. Finalmente, se declara la variable `guardvalmax`, la cual invoca la función `valormaximo` para determinar el valor máximo de `datosvarxse`.

```
# VALOR MAXIMO
datosvarxse <- c(x1,x2,x3,x4)
valormaximo <- function(vector){max(vector)}
guardvalmax <- valormaximo(datosvarxse)
```

Para calcular el factor de escala se debe tomar en consideración el valor máximo del eje x y dividirlo para 10.

```
# FACTOR DE ESCALA EJE X
escalax <- round(guardvalmax/10,3)
```

Donde:

1. `guardvalmax` es la variable que se definió previamente y almacena el valor máximo de los datos en el vector `datosvarxse`.
2. `round(guardvalmax/10, 3)` es una expresión que realiza dos operaciones:
   a) Divide `guardvalmax` por 10. Esto significa que estás reduciendo el valor máximo en el eje x, para encontrar cuanto equivale cada centímetro.
   b) `round(..., 3)` redondea el resultado a tres decimales, para obtener un valor más manejable, y evitar desbordamientos de textos del contenido escrito LaTeX.

A continuación, se necesita ajustar los datos `x1,x2,x3,x4` a la escala adecuada, considerando el factor de escala escalax calculado previamente.

```
# DATOS ESCALADOS EJE X
x1esc <- round(x1/escalax,5)
x2esc <- round(x2/escalax,5)
x3esc <- round(x3/escalax,5)
x4esc <- round(x4/escalax,5)
```

Aquí está una explicación breve del código:

1. `x1esc,x2esc,x3esc,x4esc` son nuevas variables que almacenarán los valores escalados de `x1,x2,x3,x4`, respectivamente.
2. Cada línea de código divide el valor original por el factor de escala `escalax` y redondea el resultado a cinco decimales utilizando `round(...,5)`.

Esto da como resultado los valores escalados en el eje x para cada uno de los datos originales. Finalmente el *chunk 2*, queda estructurado de la siguiente manera:

```
 ...
 \path [draw,->,name path=ejey] (0,-.3) -- (0,10.3) node (ejey) [above] {$\Sexpr{vdm}\
    left(\Sexpr{vdu}\right)$};
<<echo=FALSE>>=
# LECTURA DE DATOS EJE X
datosvarx <- read.table("base_de_datos_2.txt", header=FALSE)
# DATOS SIN ESCALAR EJE X
x1 <- datosvarx[1,2]
x2 <- datosvarx[1,3]
x3 <- datosvarx[1,4]
x4 <- datosvarx[1,5]
# VALOR MAXIMO EJE X
datosvarxse <- c(x1,x2,x3,x4)
valormaximo <- function(vector){max(vector)}
guardvalmax <- valormaximo(datosvarxse)
# FACTOR DE ESCALA EJE X
escalax <- round(guardvalmax/10,3)
# DATOS ESCALADOS EJE X
x1esc <- round(x1/escalax,5)
x2esc <- round(x2/escalax,5)
x3esc <- round(x3/escalax,5)
x4esc <- round(x4/escalax,5)
@
 \foreach \x in {1,2,3,...,10} \draw (\x,1pt) -- (\x,-1pt) node[below] {\x};
 ...
```

Para incorporar el valor del factor de escala `escalax` en el código de LaTeX, debe considerar las siguientes expresiones:

Variable de **R**		Comando LaTeX
`escalax`	$\rightarrow$	`\Sexpr{escalax}`

Obteniendo un resultado similar al siguiente:

```
...
$\displaystyle \Sexpr{vim}: \Sexpr{escalax}\left(\Sexpr{viu}\right)\cong 1 \ cm$ \\
...
```

Para incorporar las variables trabajadas en **R**, en el código de LaTeX, debe considerar las siguientes expresiones:

Variables de **R**		Comandos LaTeX
`x1`	$\rightarrow$	`\Sexpr{x1}`
`x2`	$\rightarrow$	`\Sexpr{x2}`
`x3`	$\rightarrow$	`\Sexpr{x3}`
`x4`	$\rightarrow$	`\Sexpr{x4}`
`x1esc`	$\rightarrow$	`\Sexpr{x1esc}`
`x2esc`	$\rightarrow$	`\Sexpr{x2esc}`
`x3esc`	$\rightarrow$	`\Sexpr{x3esc}`
`x4esc`	$\rightarrow$	`\Sexpr{x4esc}`

En primera instancia, es esencial reemplazar los valores almacenados en la macro `\x` de la línea `\foreach \x in {1,2,3,...,10} ...` por los valores escalados, a saber, `x1esc, x2esc, x3esc, x4esc`. Además, mediante la aplicación del método de rutas repetidas (consulte 4.1.5), se garantiza que las tarjas sean escaladas de manera apropiada y etiquetadas con los datos experimentales `x1,x2,x3,x4`.

```
...
\foreach \x/\w in {\Sexpr{x1esc}/\Sexpr{x1},\Sexpr{x2esc}/\Sexpr{x2},\Sexpr{x3esc}/\
    Sexpr{x3},\Sexpr{x4esc}/\Sexpr{x4}} \draw (\x,1pt) -- (\x,-1pt) node[below] {\w};
...
```

Al realizar esta modificación y compilar el archivo `Plot4.Rnw`, es posible que el resultado visual no sea tan agradable y parezca que ha afectado la apariencia de la plantilla. Sin embargo, no es motivo de preocupación, esto es natural cuando se realizan ajustes en los datos o en la plantilla, y es una oportunidad para aprender y perfeccionar el resultado final.

De igual forma, se reemplazan los valores de la lista `listx` con los comandos de la Tabla 5.1.2 de LaTeX para los valores escalados.

```
...
\foreach \listx/\listy in {\Sexpr{x1esc}/0,\Sexpr{x2esc}/3,\Sexpr{x3esc}/7,\Sexpr{x4esc}
    /10} {
 \draw [blue, fill] (\listx,\listy) circle [radius=2pt];
 \draw [dashed] (0,\listy) -- (\listx,\listy) -- (\listx,0);}
...
```

Una vez realizado este ajuste y vuelto a compilar el archivo `Plot4.Rnw`, es probable que los problemas visuales previamente experimentados se resuelvan, y la plantilla recuperará su aspecto visual inicial. Caso contrario, revise minuciosamente el código **R** y LaTeX

5.1.3 Datos variable dependiente: *Continuación del Chunk 2*

En esta nueva subsección, se abordará un proceso esencial pero ligeramente diferente al que se ha explorado anteriormente. Si bien la sección anterior se enfocó en la variable independiente, esta ocasión se centrará en la variable dependiente de nuestros datos experimentales.

Por lo tanto, en el código que sigue, encontrarás las mismas técnicas y procesos aplicados en la subsección anterior, con la diferencia de que ahora se refieren a la variable dependiente y por tanto varían levemente los nombres de las variables declaradas en **R**.

```
 ...
 \path [draw,->,name path=ejey] (0,-.3) -- (0,10.3) node (ejey) [above] {$\Sexpr{vdm}\
    left(\Sexpr{vdu}\right)$};
<<echo=FALSE>>=
 ...
x4esc <- round(x4/escalax,5)
# LECTURA DE DATOS EJE Y
datosvary <- read.table("base_de_datos_2.txt", header=FALSE)
# DATOS SIN ESCALAR EJE Y
y1 <- datosvary[2,2]
y2 <- datosvary[2,3]
y3 <- datosvary[2,4]
y4 <- datosvary[2,5]
# VALOR MAXIMO EJE Y
datosvaryse <- c(y1,y2,y3,y4)
guardvalmay <- valormaximo(datosvaryse)
# FACTOR DE ESCALA EJE Y
escalay <- round(guardvalmay/10,3)
# DATOS ESCALADOS EJE Y
y1esc <- round(y1/escalay,5)
y2esc <- round(y2/escalay,5)
y3esc <- round(y3/escalay,5)
y4esc <- round(y4/escalay,5)
@
 \foreach \x/\w in {\Sexpr{x1esc}/\Sexpr{x1},\Sexpr{x2esc}/\Sexpr{x2},\Sexpr{x3esc}/\
    Sexpr{x3},\Sexpr{x4esc}/\Sexpr{x4}} \draw (\x,1pt) -- (\x,-1pt) node[below] {\w};
 ...
```

Para incorporar el valor del factor de escala `escalay` en el código de LaTeX, debe considerar las siguientes expresiones:

Variable de **R**		Comando LaTeX
`escalay`	→	`\Sexpr{escalay}`

Obteniendo un resultado similar al siguiente:

```
 ...
 $\displaystyle \Sexpr{vdm}: \Sexpr{escalay} \left(\Sexpr{vdu}\right)\cong 1 \ cm$ \\ \
    midrule
 ...
```

Para incorporar las variables trabajadas en **R** en el código de LaTeX, debe considerar las siguientes expresiones:

Variables de **R**		Comandos LaTeX
`y1`	→	`\Sexpr{y1}`
`y2`	→	`\Sexpr{y2}`
`y3`	→	`\Sexpr{y3}`
`y4`	→	`\Sexpr{y4}`
`y1esc`	→	`\Sexpr{y1esc}`
`y2esc`	→	`\Sexpr{y2esc}`
`y3esc`	→	`\Sexpr{y3esc}`
`y4esc`	→	`\Sexpr{y4esc}`

En primera instancia, es esencial reemplazar los valores almacenados en la macro `\y` de la línea `\foreach \y in {1,2,3,...,10} ...` por los valores escalados, es decir, por `y1esc, y2esc, y3esc, y4esc`. Además, mediante la aplicación del método de rutas repetidas (consulte 4.1.5), se garantiza que las tarjas sean escaladas de manera apropiada y etiquetadas con los datos experimentales `y1,y2,y3,y4`.

```
...
\foreach \y/\z in {\Sexpr{y1esc}/\Sexpr{y1},\Sexpr{y2esc}/\Sexpr{y2},\Sexpr{y3esc}/\
   Sexpr{y3},\Sexpr{y4esc}/\Sexpr{y4}} \draw (1pt,\y) -- (-1pt,\y) node[left] {\z};
...
```

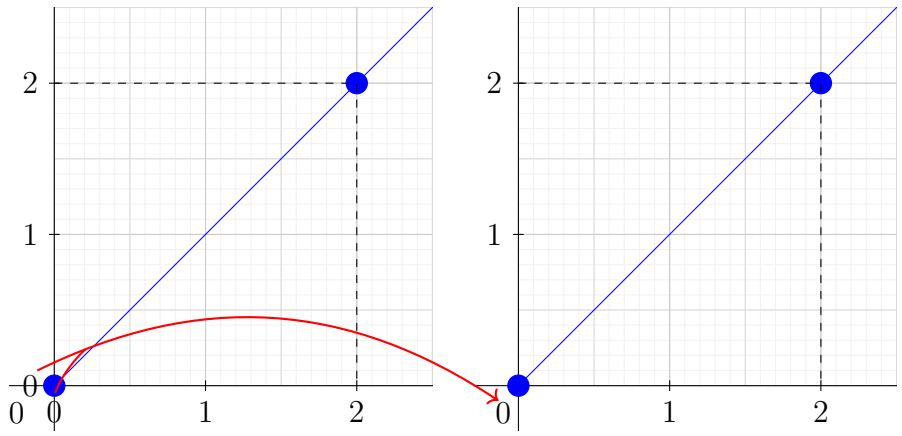
Asimismo, se reemplazan los valores de la lista `listy` con los comandos de la Tabla 5.1.3 de la columna de "Comandos LaTeX" para los valores escalados.

```
...
\foreach \listx/\listy in {\Sexpr{x1esc}/\Sexpr{y1esc},\Sexpr{x2esc}/\Sexpr{y2esc},\
   Sexpr{x3esc}/\Sexpr{y3esc},\Sexpr{x4esc}/\Sexpr{y4esc}} {
\draw [blue, fill] (\listx,\listy) circle [radius=2pt];
\draw [dashed] (0,\listy) -- (\listx,\listy) -- (\listx,0);}
...
```

Verificación: Con el propósito de confirmar que todas las operaciones realizadas con código **R** y su integración en el documento LaTeXhan sido ejecutadas de manera correcta hasta este punto, se invita al usuario a acceder al archivo `base_de_datos_2.txt`. Desde este archivo, puede realizar las modificaciones necesarias en los datos experimentales y observar los resultados generados (archivo `Plot4.pdf`). Es importante destacar que el archivo de resultados reflejará las alteraciones efectuadas en el archivo de datos, siempre y cuando estas hayan sido guardadas de manera adecuada.

5.1.4 Función cero: Continuación del *Chunk 2*

Existe un principal problema, debido a que las datos experimentales de la variable dependiente e independiente contienen ceros. TikZ va a dibujar estos dos ceros en el eje x y en el eje y, respectivamente. Como se muestra a continuación.



La función cero (Ver Flujograma 1) se va a encargar de buscar en los vectores `datosvarxse` y `datosvaryse` la existencia de datos experimentales iguales a cero, de manera individual. Si encuentra algún cero en estos dos vectores, entonces los reemplaza por un cadena vacía. Caso contrario, escribe el valor correspondiente al dato experimental.

```
<<echo=FALSE>>=
...
# FUNCION CERO EJE X
tarjax1 <- if (any(datosvarxse == "0")) {""} else {x1}
# FUNCION CERO EJE Y
tarjay1 <- if (any(datosvaryse == "0")) {""} else {y1}
...
@
```

Donde:

1. `tarjax1` y `tarjay1`, son el nombre de las variables, y se les asigna un valor mediante `<-`.
2. `if () {} else {}` es una estructura de condicional.
3. `any(datosvarxse == "0")` y `any(datosvarxse == "0")` son las condiciones y evalúan si existen almenos un elemento en el vector `datosvarxse` y `datosvaryse` que sea igual a `"0"` mediante la función `any()`.
4. Si la condición es verdadera, entonces se asigna una cadena vacía `""` a la variable `tarjax1` y `tarjay1`, respectivamente.
5. Si la condición es falsa, entonces se asigna una cadena vacía `x1` y `y1` a la variable `tarjax1` y `tarjay1`, respectivamente.

Para incorporar el valor de la función cero para el eje x y el eje y en el código de LaTeX, debe considerar las siguientes expresiones:

Función Cero		Comando LaTeX
`tarjax1`	$\rightarrow$	`\Sexpr{tarjax1}`
`tarjay1`	$\rightarrow$	`\Sexpr{tarjay1}`

Obteniendo como resultado el siguiente código:

```
...
\foreach \x/\w in {\Sexpr{x1esc}/\Sexpr{tarjax1},\Sexpr{x2esc}/\Sexpr{x2},\Sexpr{x3esc}
   /\Sexpr{x3},\Sexpr{x4esc}/\Sexpr{x4}} \draw (\x,1pt) -- (\x,-1pt) node[below] {\w};
\foreach \y/\z in {\Sexpr{y1esc}/\Sexpr{tarjay1},\Sexpr{y2esc}/\Sexpr{y2},\Sexpr{y3esc}
   /\Sexpr{y3},\Sexpr{y4esc}/\Sexpr{y4}} \draw (1pt,\y) -- (-1pt,\y) node[left] {\z};
```

5.1.5 Regresión Lineal: *Chunk 3*

El siguiente objetivo consiste en la construcción de la recta de regresión lineal a partir de los datos experimentales escalados. Es importante destacar que, aunque podríamos optar por trabajar con los datos experimentales originales, esta elección resultaría poco eficiente. La razón radica en que sería necesario escalar nuevamente la recta de regresión lineal, lo cual conllevaría a una mayor cantidad de líneas de código y redundaría en procesos que ya fueron detallados. En consecuencia, se recomienda utilizar los datos experimentales escalados para optimizar la implementación y evitar complicaciones visuales adicionales.

```
<<echo=FALSE>>=
# REGRESION LINEAL DATOS ESCALADOS
datosvarxe <- c(x1esc,x2esc,x3esc,x4esc)
datosvarye <- c(y1esc,y2esc,y3esc,y4esc)
regreesc <- lm(datosvarye ~ datosvarxe)
coeficientesesc <- coef(regreesc)
corteesc <- round(coeficientesesc[1],5)
pendienteesc <- round(coeficientesesc[2],5)
@
```

Donde:

1. `datosvarxe` y `datosvarye` son vectores que contienen datos escalados en el eje x y el eje y, respectivamente.
2. `regreesc` ajusta un modelo de regresión lineal simple usando `lm()`. La fórmula `datosvarye ~datosvarxe` especifica que `datosvarye` es la variable de independiente y `datosvarxe` es la variable dependiente.
3. `coeficientesesc` utiliza la función `coef()` para extraer los coeficientes del modelo de regresión lineal ajustado, que incluyen el punto de corte con el eje de las ordenadas y la pendiente.
4. `corteesc` almacena el punto de corte de la línea de regresión, y `pendienteesc` almacena el valor de la pendiente.

Estos datos se utilizaran para construir la recta de regresión lineal.

Estructura		LaTeX
$y = 1x + 0$	$\rightarrow$	`y=\Sexpr{pendienteesc}x+\Sexpr{corteesc}`
		TikZ
	$\rightarrow$	`... plot (\x,\Sexpr{pendienteesc}*x+\Sexpr{corteesc});`

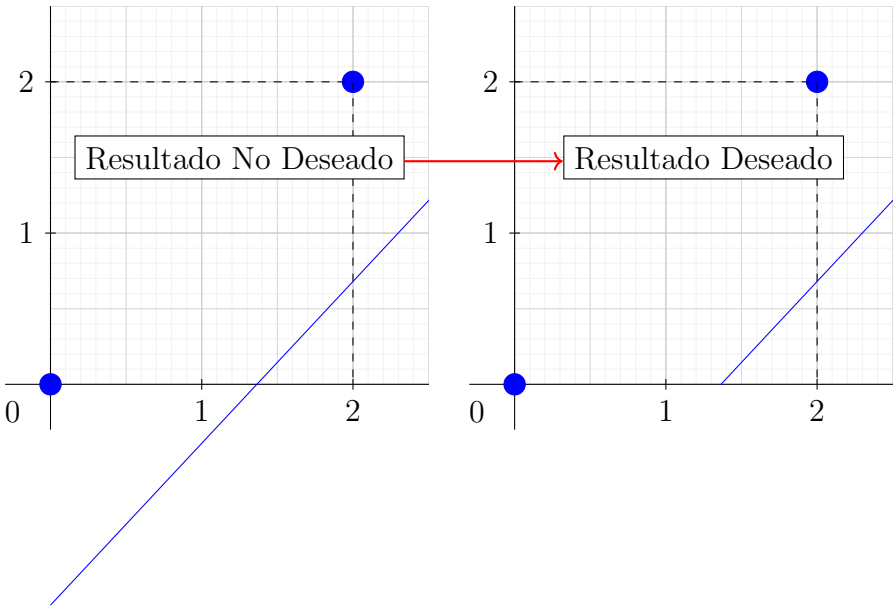
Obteniendo como resultado el siguiente código LaTeX.

```
...
\path [draw,blue,name path=regresion,domain=0:10] plot (\x,\Sexpr{pendienteesc}*\x+\
    Sexpr{corteesc});
...
```

Ahora bien, para efectos prácticos se recomienda probar con los siguientes datos experimentales. Obteniendo resultados no deseado.

base_de_datos_2.txt: Bloc de notas

```
Independiente 0.1 0.2 0.3 0.4
Dependiente 2 4.5 6.5 12
```

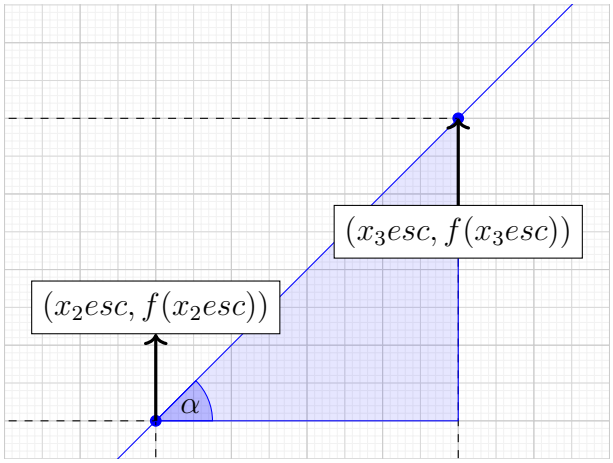


Este problema se origina porque no esta establecido el rango. Para pasar del resultado no deseado al resultado deseado, se puede incluir el rango mediante `domain=⟨start⟩:⟨end⟩` dentro de opciones de `\path [opciones] ... plot ...;`. O por el contrario, se debe usar el compando `\clip (x,y) rectangle (x,y);`. Obteniendo un código similar al siguiente:

```
...
\begin{scope}
\clip (0,0) rectangle (10,10);
\path [draw,blue,name path=regresion,domain=0:10] plot (\x,\Sexpr{pendienteesc}*\x+\
    Sexpr{corteesc});
% Inicio de comandos de construccion
...
\end{scope}
\begin{scope}[xshift=14.25cm,yshift=5cm]
...
```

5.1.6 Parámetros del Triángulo de Inspección: Continuación del *Chunk 3*

Como se mencionó previamente (ver párrafo 4.1.7), en realidad se deconoce las coordenadas del trángulo de inspección, puesto que este se contruye con $(x_2esc, f(x_2esc))$, $(x_3esc, f(x_3esc))$ y no con (x_2, y_2), (x_3, y_3).



Se puede observar que los puntos $(x_2esc, f(x_2esc))$ y $(x_3esc, f(x_3esc))$ pertenecen a la recta de regresión lineal `y=\Sexpr{pendienteesc}x+\Sexpr{corteesc}`. Por lo tanto, para obtener $f(x_2esc)$ y $f(x_3esc)$ basta con evaluar la recta de regresión lineal en los puntos x_2esc y x_3esc.

`f(`x_2esc`)=\Sexpr{pendienteesc}\Sexpr{x2esc}+\Sexpr{corteesc}`

`f(`x_3esc`)=\Sexpr{pendienteesc}\Sexpr{x3esc}+\Sexpr{corteesc}`

Asimismo, puedo calcular el valor del ángulo α a partir del valor calculado de la pendiente `\Sexpr{pendienteesc}`. Es decir:

$$\texttt{pendienteesc} = tan(\alpha)$$
$$\alpha = tan^{-1}(\texttt{pendienteesc})$$

Para ello, podemos utilizar el siguiente código:

```
<<echo=FALSE>>=
# REGRESION LINEAL DATOS ESCALADOS
...
fx2esc <- round(pendienteesc*x2esc+corteesc,5)
fx3esc <- round(pendienteesc*x3esc+corteesc,5)
alpharad <- round(atan(pendienteesc),2)
@
```

Además, se debe procurar trabajar con grados sexagesimales para el valor de α. Para ello, podemos utilizar el siguiente código:

```
<<echo=FALSE>>=
...
alpharad <- round(atan(pendienteesc),2)
alphagrad <-  round((alpharad*180)/pi,2)
@
```

Para incorporar las coordenadas en el código de LaTeX, debe considerar las siguientes expresiones:

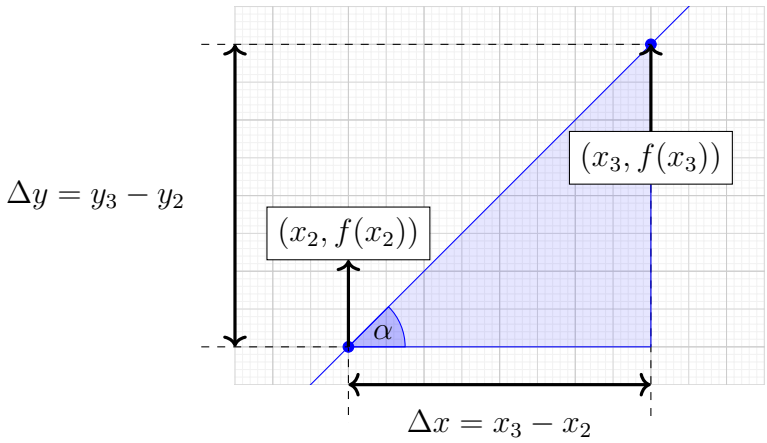
R		LaTeX
`fx2esc`	$\rightarrow$	`\Sexpr{fx2esc}`
`fx3esc`	$\rightarrow$	`\Sexpr{fx3esc}`
`alphagrad`	$\rightarrow$	`\Sexpr{alphagrad}`

Obteniendo como resultado el siguiente código:

```
...
% Inicio de comandos de construccion
\path (\Sexpr{x3esc},\Sexpr{fx3esc}) coordinate (a) -- (a|-ejex) coordinate (b);
\path (\Sexpr{x2esc},\Sexpr{fx2esc}) coordinate (c) -- (c-|a) coordinate (d);
...
\draw [draw=blue,fill=blue,fill opacity=0.2] (c) -- ++(.75cm,0cm) arc [start angle=0,
   end angle=\Sexpr{alphagrad},radius=.75cm] -- (c);
\path (c) -- ++(\Sexpr{alphagrad}/2:0.5cm) node {$\alpha$};
...
```

5.1.7 Análisis Matemático: *Chunk 4*

Finalmente, se debe calcular los parámetros del análisis matemático, tales como: variación de la variable dependiente y variación de la variable independiente. Ahora bien, esto puede causar un poco de confusión para los lectores. Puesto que, para este punto se necesita el valor de los datos experimentales originales (datos sin escalar).



Para ello, puede utilizar el siguiente código:

```
<<echo=FALSE>>=
# REGRESION LINEAL DATOS ORIGINALES
regre <- lm(datosvaryse ~ datosvarxse)
coeficientes <- coef(regre)
corte <- round(coeficientes[1],5)
pendienteregre <- round(coeficientes[2],5)
fx2 <- round(pendienteregre*x2+corte,2)
fx3 <- round(pendienteregre*x3+corte,2)
pendienteecuac <- round({(fx3-fx2)/(x3-x2)},2)
@
```

Para incorporar las coordenadas en el código de LaTeXpara el cálculo de variaciones, debe considerar las siguientes expresiones:

R		LaTeX
`fx2`	$\rightarrow$	`\Sexpr{fx2}`
`x2`	$\rightarrow$	`\Sexpr{x2}`
`fx3`	$\rightarrow$	`\Sexpr{fx3}`
`x3`	$\rightarrow$	`\Sexpr{x3}`

Obteniendo como resultado el siguiente código:

```
...
$\displaystyle k=\frac{\Delta d}{\displaystyle \Delta t}=\frac{\Sexpr{fx3}-\Sexpr{fx2
   }}{\Sexpr{x3}-\Sexpr{x2}}$ \\
$k=\Sexpr{pendienteecuac}$ \\ \midrule
...
```

Asimismo, para incorporar el valor de la pendiente y el punto de corte en la etiqueta del triángulo de inspección, debe considerar las siguientes expresiones:

R		LaTeX
`pendienteregre`	$\rightarrow$	`\Sexpr{pendienteregre}`
`corte`	$\rightarrow$	`\Sexpr{corte}`

Obteniendo como resultado el siguiente código:

```
...
\node [draw=black,fill=white,rotate=45,above=5pt] at (interseccion1) {$\Sexpr{vdm}=\
   Sexpr{pendienteregre}\Sexpr{vim}+\Sexpr{corte}$};
...
```

5.2 Plantilla RSweave: 5 datos

Una vez desarrollada la plantilla RSweave para 4 datos datos experimentales. El siguiente punto es extender esta idea a 5 datos experimentales. En resumen, incluir 1 dato experimental adicional.

Para ello, usted puede guiarse en el siguiente procedimiento:

1. Ingresar a la ruta `C:\Software_de_graficacion\Graficador` y abrir el archivo `Plot5.Rnw` con **RStudio**.
2. Selecionar todo el código desarrollado de `Plot4.Rnw` y copiarlo en la plantilla de 5 datos experimentales `Plot5.Rnw`.
3. **Modificar el *chunk 2***: El proceso de modificación se enfoca en aumentar un dato experimental `x5`, `y5` y escalarlo de forma adecuada por `x5esc`,`y5esc`, respectivamente.

```
<<echo=FALSE>>=
...
x4 <- datosvarx[1,5]
x5 <- datosvarx[1,6]
# VALOR MAXIMO EJE X
datosvarxse <- c(x1,x2,x3,x4,x5)
...
x4esc <- round(x4/escalax,5)
x5esc <- round(x5/escalax,5)
...
y4 <- datosvary[2,5]
y5 <- datosvary[2,6]
# VALOR MAXIMO EJE Y
datosvaryse <- c(y1,y2,y3,y4,y5)
...
y4esc <- round(y4/escalay,5)
y5esc <- round(y5/escalay,5)
...
@
```

4. `x5` se puede representar en LaTeXcomo `\Sexpr{x5}` y `x5esc` como `\Sexpr{x5esc}`. De manera similar, `y5` como `\Sexpr{y5}` y `y5esc` como `\Sexpr{y5esc}`.

```
...
\foreach \x/\w in {\Sexpr{x1esc}/\Sexpr{tarjax1},\Sexpr{x2esc}/\Sexpr{x2},\Sexpr{
    x3esc}/\Sexpr{x3},\Sexpr{x4esc}/\Sexpr{x4},\Sexpr{x5esc}/\Sexpr{x5}} \draw (\x
    ,1pt) -- (\x,-1pt) node[below] {\w};
\foreach \y/\z in {\Sexpr{y1esc}/\Sexpr{tarjay1},\Sexpr{y2esc}/\Sexpr{y2},\Sexpr{
    y3esc}/\Sexpr{y3},\Sexpr{y4esc}/\Sexpr{y4},\Sexpr{y5esc}/\Sexpr{y5}} \draw (1pt
    ,\y) -- (-1pt,\y) node[left] {\z};
\node [below] at (-.25,-.04) {$0$};
\foreach \listx/\listy in {\Sexpr{x1esc}/\Sexpr{y1esc},\Sexpr{x2esc}/\Sexpr{y2esc
    },\Sexpr{x3esc}/\Sexpr{y3esc},\Sexpr{x4esc}/\Sexpr{y4esc},\Sexpr{x5esc}/\Sexpr{
    y5esc}} {
\draw [blue, fill] (\listx,\listy) circle [radius=2pt];
\draw [dashed] (0,\listy) -- (\listx,\listy) -- (\listx,0);}
...
```

Es esencial asegurarse de que el triángulo de inspección esté centrado en todas las plantillas. En otras palabras, si se cuentan con 4 datos experimentales, el triángulo de inspección debe estar posicionado de manera que sus coordenadas sean $(x_2esc, f(x_2esc))$ y $(x_3esc, f(x_3esc))$. Si se dispone de 5 datos experimentales, el triángulo deberá estar centrado en $(x_2esc, f(x_2esc))$ y $(x_4esc, f(x_4esc))$, y si hay 6 datos experimentales, las coordenadas adecuadas serán $(x_3esc, f(x_3esc))$ y $(x_4esc, f(x_4esc))$, y así sucesivamente.

5. **Modificar el *chunk 3***: Incorporar `x5esc` y `y5esc` en `datosvarxe` y `datosvarye`, respectivamente. Además, determinar el valor de $f(x_2)$ y $f(x_4)$.

```
<<echo=FALSE>>=
# REGRESION LINEAL DATOS ESCALADOS
datosvarxe <- c(x1esc,x2esc,x3esc,x4esc,x5esc)
datosvarye <- c(y1esc,y2esc,y3esc,y4esc,y5esc)
...
fx2esc <- round(pendienteesc*x2esc+corteesc,5)
fx4esc <- round(pendienteesc*x4esc+corteesc,5)
...
@
```

6. En LaTeX reemplazar `\Sexpr{fx3esc}` por `\Sexpr{fx4esc}`, y `\Sexpr{x3esc}` por `\Sexpr{x4esc}`.

```
...
\path (\Sexpr{x4esc},\Sexpr{fx4esc}) coordinate (a) -- (a|-ejex) coordinate (b);
\path (\Sexpr{x2esc},\Sexpr{fx2esc}) coordinate (c) -- (c-|a) coordinate (d);
...
```

7. **Modificar el *chunk 4:*** Calcular las variaciones de las variables dependiente e independiente, representadas por `fx2` y `fx4`, respectivamente. Estas variaciones se encuentran directamente relacionadas con las coordenadas del triángulo de inspección.

```
<<echo=FALSE>>=
...
fx2 <- round(pendienteregre*x2+corte,2)
fx4 <- round(pendienteregre*x4+corte,2)
pendienteecuac <- round({(fx4-fx2)/(x4-x2)},2)
@
```

8. Incorpoar `fx2` y `fx4` en el análisis matemático de la Leyenda, mediante `\Sexpr{fx2}` y `\Sexpr{fx4}`, respectivamente. De igual manera, `x2` mediante `\Sexpr{x2}` y `x4` mediante `\Sexpr{x4}`.

```
...
$\displaystyle k=\frac{\Delta d}{\displaystyle \Delta t}=\frac{\Sexpr{fx4}-\Sexpr{
    fx2}}{\Sexpr{x4}-\Sexpr{x2}}$ \\
...
```

Una vez que haya seguido los procedimientos adecuados, tendrá a su disposición la plantilla `Plot5.Rnw` especialmente diseñada para trabajar con un conjunto de 5 datos experimentales.

5.3 Plantilla RSweave: 6 datos

Una vez desarrollada la plantilla RSweave para 5 datos datos experimentales. El siguiente punto es extender esta idea a 6 datos experimentales. En resumen, incluir 1 dato experimental adicional.

Para ello, usted puede guiarse en el siguiente procedimiento:

1. Ingresar a la ruta `C:\Software_de_graficacion\Graficador` y abrir el archivo `Plot6.Rnw` con **RStudio**.
2. Selecionar todo el código desarrollado de `Plot5.Rnw` y copiarlo en la plantilla de 5 datos experimentales `Plot6.Rnw`.
3. **Modificar el *chunk 2***: El proceso se enfoca en aumentar un dato experimental `x6`, `y6` y escalarlo de forma adecuada por `x6esc`,`y6esc`, respectivamente.

```
<<echo=FALSE>>=
...
x5 <- datosvarx[1,6]
x6 <- datosvarx[1,7]
# VALOR MAXIMO EJE X
datosvarxse <- c(x1,x2,x3,x4,x5,x6)
...
x5esc <- round(x5/escalax,5)
x6esc <- round(x6/escalax,5)
...
y5 <- datosvary[2,6]
y6 <- datosvary[2,7]
# VALOR MAXIMO EJE Y
datosvaryse <- c(y1,y2,y3,y4,y5,y6)
...
y5esc <- round(y5/escalay,5)
y6esc <- round(y6/escalay,5)
...
@
```

4. `x6` se puede representar en LaTeXcomo `\Sexpr{x6}` y `x6esc` como `\Sexpr{x6esc}`. De manera similar, `y6` como `\Sexpr{y6}` y `y6esc` como `\Sexpr{y6esc}`.

```
...
\foreach \x/\w in {\Sexpr{x1esc}/\Sexpr{tarjax1},\Sexpr{x2esc}/\Sexpr{x2},\Sexpr{
    x3esc}/\Sexpr{x3},\Sexpr{x4esc}/\Sexpr{x4},\Sexpr{x5esc}/\Sexpr{x5},\Sexpr{
    x6esc}/\Sexpr{x6}} \draw (\x,1pt) -- (\x,-1pt) node[below] {\w};
\foreach \y/\z in {\Sexpr{y1esc}/\Sexpr{tarjay1},\Sexpr{y2esc}/\Sexpr{y2},\Sexpr{
    y3esc}/\Sexpr{y3},\Sexpr{y4esc}/\Sexpr{y4},\Sexpr{y5esc}/\Sexpr{y5},\Sexpr{
    y6esc}/\Sexpr{y6}} \draw (1pt,\y) -- (-1pt,\y) node[left] {\z};
\node [below] at (-.25,-.04) {$0$};
\foreach \listx/\listy in {\Sexpr{x1esc}/\Sexpr{y1esc},\Sexpr{x2esc}/\Sexpr{y2esc
    },\Sexpr{x3esc}/\Sexpr{y3esc},\Sexpr{x4esc}/\Sexpr{y4esc},\Sexpr{x5esc}/\Sexpr{
    y5esc},\Sexpr{x6esc}/\Sexpr{y6esc}} {
\draw [blue, fill] (\listx,\listy) circle [radius=2pt];
\draw [dashed] (0,\listy) -- (\listx,\listy) -- (\listx,0);}
...
```

5. **Modificar el *chunk 3***: Incorporar `x6esc` y `y6esc` en `datosvarxe` y `datosvarye`, respectivamente. Además, determinar el valor de $f(x_3)$ y $f(x_5)$.

```
<<echo=FALSE>>=
# REGRESION LINEAL DATOS ESCALADOS
datosvarxe <- c(x1esc,x2esc,x3esc,x4esc,x5esc,x6esc)
datosvarye <- c(y1esc,y2esc,y3esc,y4esc,y5esc,y6esc)
...
fx3esc <- round(pendienteesc*x3esc+corteesc,5)
fx4esc <- round(pendienteesc*x4esc+corteesc,5)
...
@
```

6. En LaTeX reemplazar `\Sexpr{fx2esc}` por `\Sexpr{fx3esc}`, y `\Sexpr{x2esc}` por `\Sexpr{x3esc}`.

```
...
\path (\Sexpr{x4esc},\Sexpr{fx4esc}) coordinate (a) -- (a|-ejex) coordinate (b);
\path (\Sexpr{x3esc},\Sexpr{fx3esc}) coordinate (c) -- (c-|a) coordinate (d);
...
```

7. **Modificar el *chunk 4:*** Calcular las variaciones de las variables dependiente e independiente, representadas por `fx3` y `fx4`, respectivamente. Estas variaciones se encuentran relacionadas con las coordenadas del triángulo de inspección.

```
<<echo=FALSE>>=
...
fx4 <- round(pendienteregre*x4+corte,2)
fx3 <- round(pendienteregre*x3+corte,2)
pendienteecuac <- round({(fx4-fx3)/(x4-x3)},2)
@
```

8. Incorpoar `fx4` y `fx3` en el análisis matemático de la Leyenda, mediante `\Sexpr{fx4}` y `\Sexpr{fx3}`, respectivamente. De igual manera, `x3` mediante `\Sexpr{x3}` y `x4` mediante `\Sexpr{x4}`.

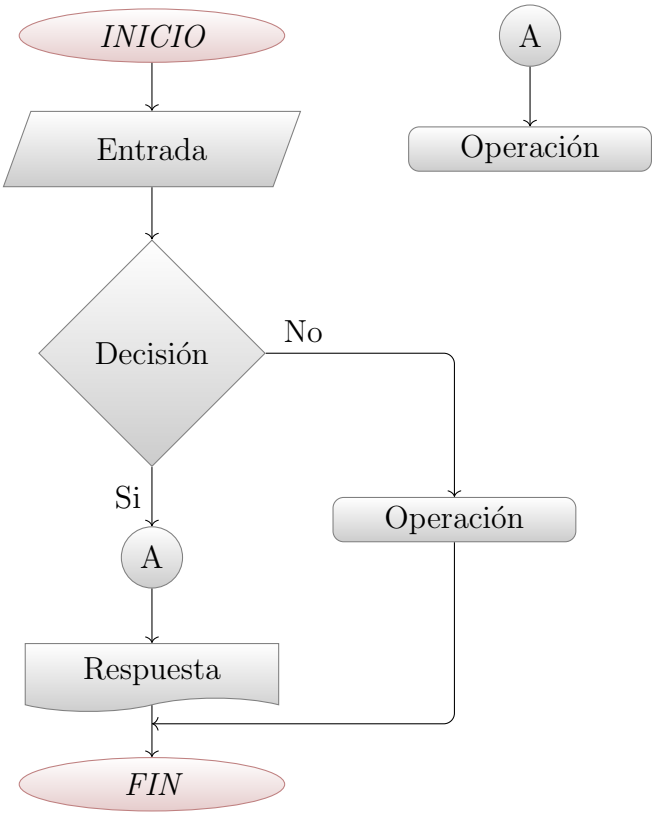
```
...
$\displaystyle k=\frac{\Delta d}{\displaystyle \Delta t}=\frac{\Sexpr{fx4}-\Sexpr{
    fx3}}{\Sexpr{x4}-\Sexpr{x3}}$ \\
...
```

Una vez que haya seguido los procedimientos adecuados, tendrá a su disposición la plantilla `Plot6.Rnw` especialmente diseñada para trabajar con un conjunto de 6 datos experimentales.

5.4 Plantilla RSweave: 7, 8, 9 y 10 datos

Para cada nueva plantilla, se recomienda repetir los procesos del 1 al 8 que se han desarrollado previamente en la adapatación de la plantilla de 5 o 6 datos experimentales.

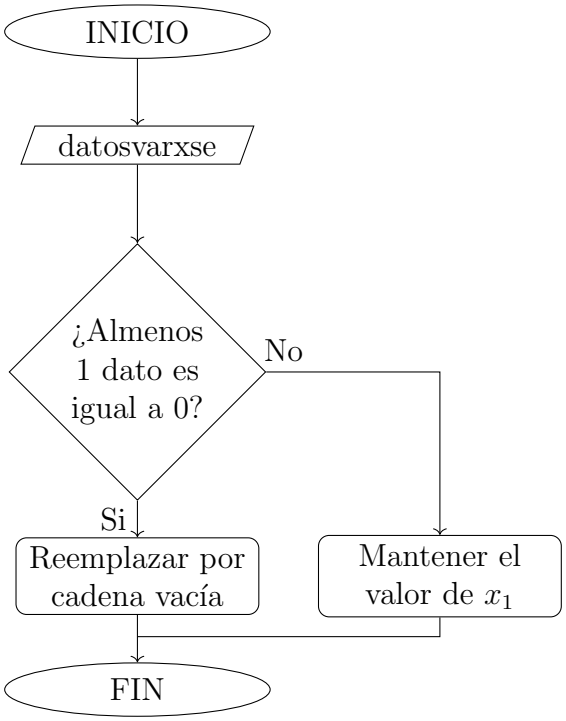
Flujograma



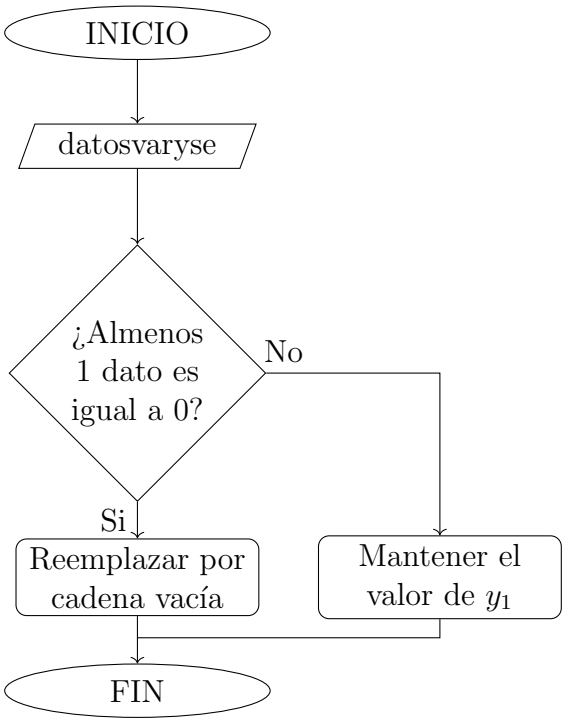

```
\usetikzlibrary {arrows.meta,graphs,shapes.misc,positioning}
\tikzstyle{elipse} = [ellipse, text width=5.5em, align=center, inner sep=3pt, draw=red!
    50!black!50, minimum size=6mm,top color=white,bottom color=red!50!black!20,font=\
    itshape]
\tikzstyle{trapecio} = [trapezium,trapezium left angle=70, trapezium right angle=110,
    text width=2.5cm,align=center,inner sep=10pt,draw=black!50,top color=white,bottom
    color=black!20]
\tikzstyle{rombo} = [diamond,text width=2cm,align=center,inner sep=5pt,draw=black!50,top
     color=white,bottom color=black!20]
\tikzstyle{circulo} = [circle, radius=5pt,draw=black!50,top color=white,bottom color=
    black!20]
\tikzstyle{imprimir} = [tape,tape bend top=none,text width=3cm,align=center,inner sep=5
    pt,draw=black!50,top color=white,bottom color=black!20]
\tikzstyle{rectangulo} = [rectangle,text width=3cm,align=center,inner sep=3pt,rounded
    corners,draw=black!50,top color=white,bottom color=black!20]
\tikzstyle{linea} = [->, rounded corners]
\begin{tikzpicture}[node distance=1.5cm]
 \node [elipse]     (a) {INICIO};
 \node [trapecio]   (b) [below of=a] {Entrada} edge [<-] (a);
 \node [rombo]      (c) [below of=b, yshift=-1.2cm] {Decision} edge [<-] (b);
 \node [circulo]    (d) [below of=c, yshift=-1.2cm] {A} edge [<-,"Si"] (c);
 \node              (e) [right of=d, xshift=2.5cm] {};
 \node [imprimir]   (f) [below of=d] {Respuesta} edge [<-] (d);
 \node [rectangulo] (h) [above of=e, yshift=-1cm] {Operacion};
 \node [elipse]     (g) [below of=f] {FIN} edge [<-] (f);
 \draw [linea]      (h) |- ($(g)+(0,.8)$);
 \draw [linea]      (c) node [xshift=2cm, above] {No} -| (h);
\end{tikzpicture}
\begin{scope}[xshift=5cm]
 \node [circulo]    (i) {A};
 \node [rectangulo] (j) [below of=i] {Operacion} edge [<-] (i);
\end{scope}
```

6.1 Algoritmo de búsqueda

El siguiente algoritmo, busca en el arreglo `datosvarxse` la existencia de almenos un elemento igual a cero.

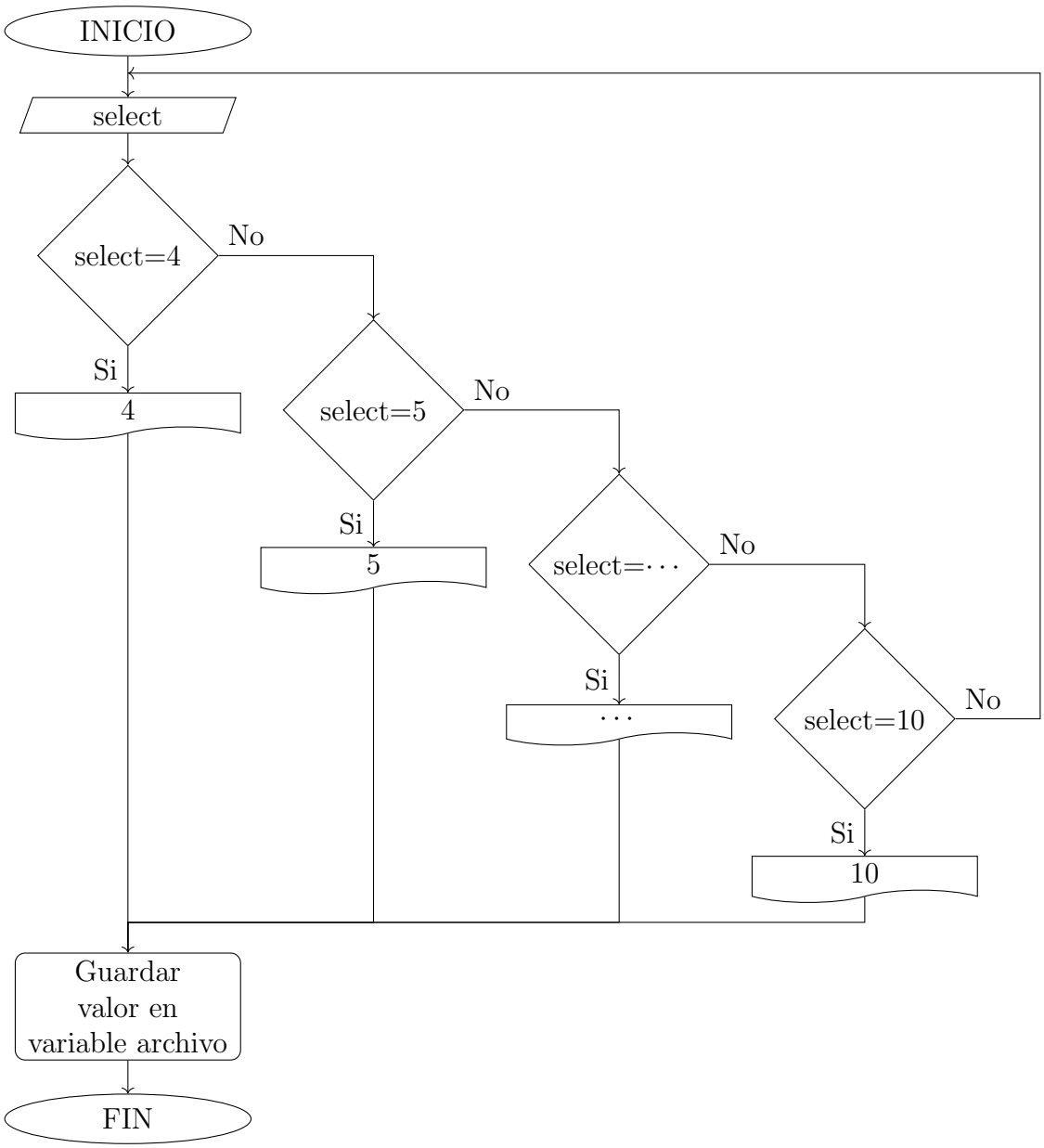


El siguiente algoritmo, busca en el arreglo `datosvaryse` la existencia de almenos un elemento igual a cero.



6.2 Algoritmo de asignación

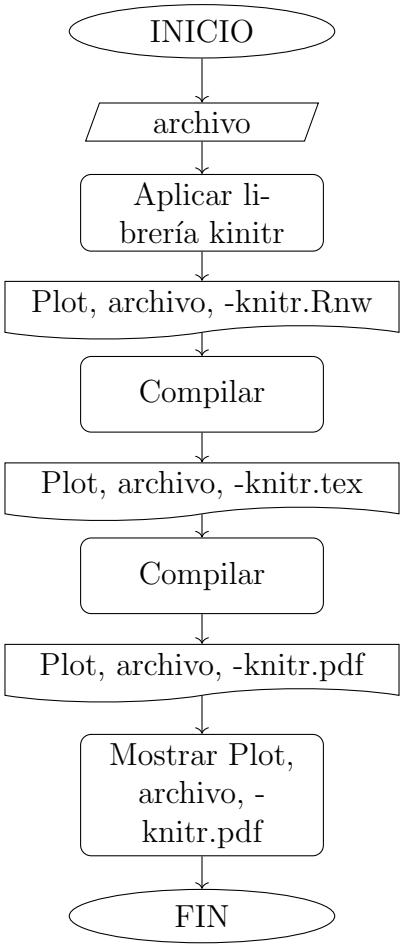
El siguiente algoritmo asigna un valor (output) a la variable archivo a partir de una entrada "select"(input).



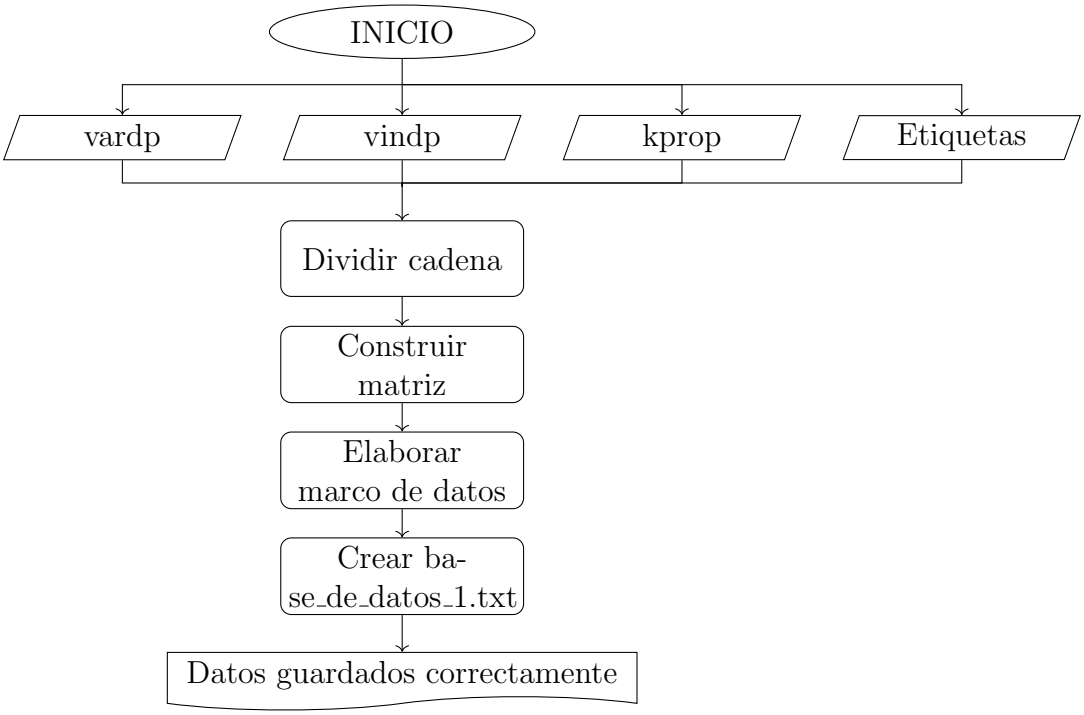
Observación: Se podría utilizar otro algoritmo que sea más eficiente y que tenga menos condicionales `if (...) {...} else {...}`.

6.3 Algoritmo para compilar

El algoritmo para compilar un archivo Sweave, toma como entrada la variable archivo.



6.4 Algoritmo guardar magnitudes, unidades y dimensiones

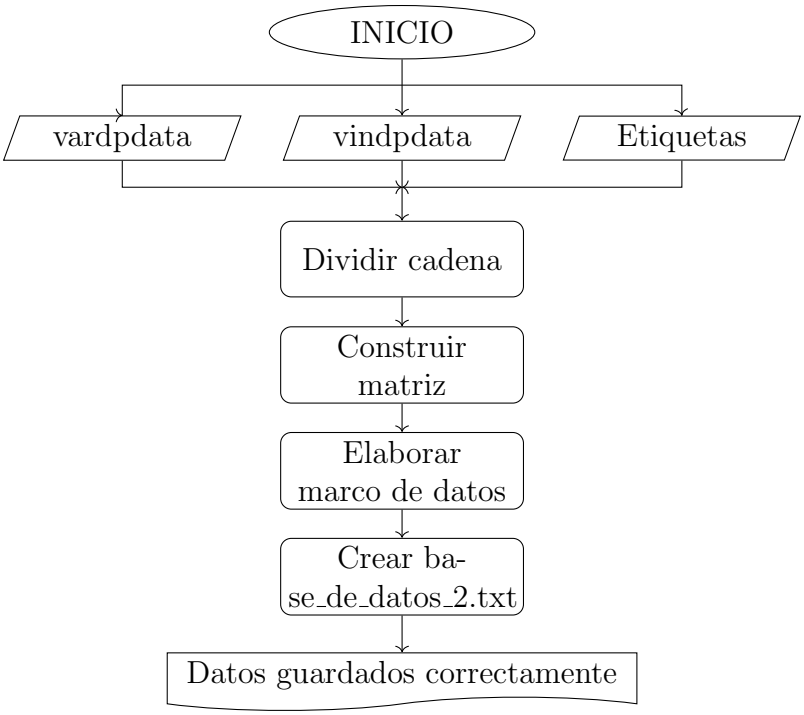


Este algoritmo toma cuatro entradas: los datos de la variable independiente `vindp`, de-

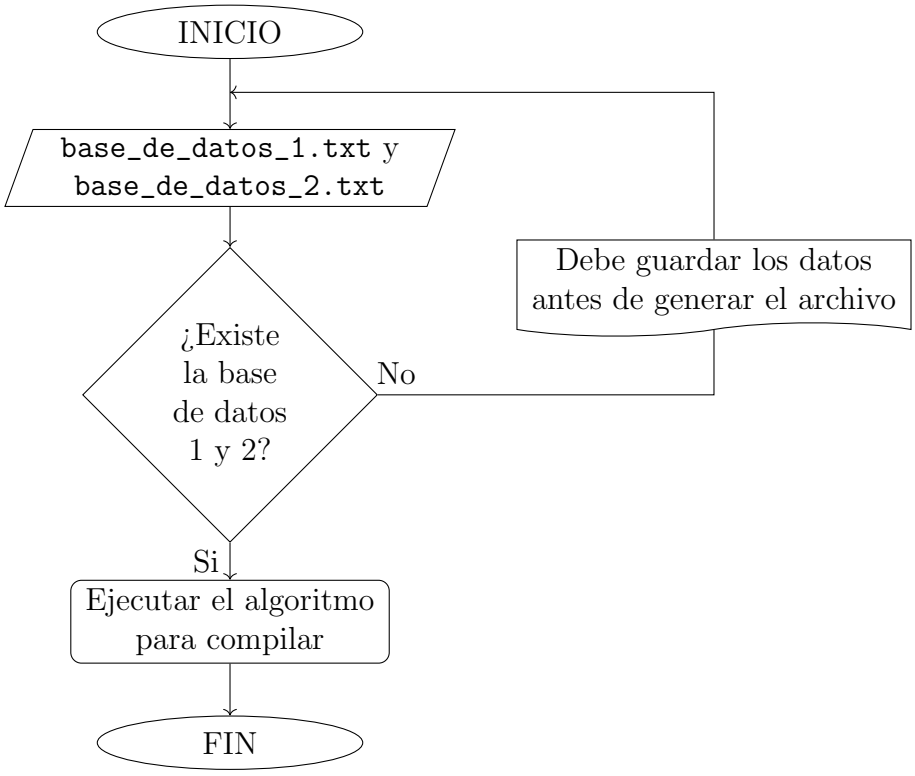
pendiente `vardp` y las etiquetas. Su propósito es generar el archivo `base_de_datos_1.txt`.

6.5 Algoritmo guardar datos experimentales

Este algoritmo toma tres entradas: datos de la variable independiente `vindpdata`, dependiente `vardpdata` y las etiquetas. Su propósito es generar `base_de_datos_2.txt`.



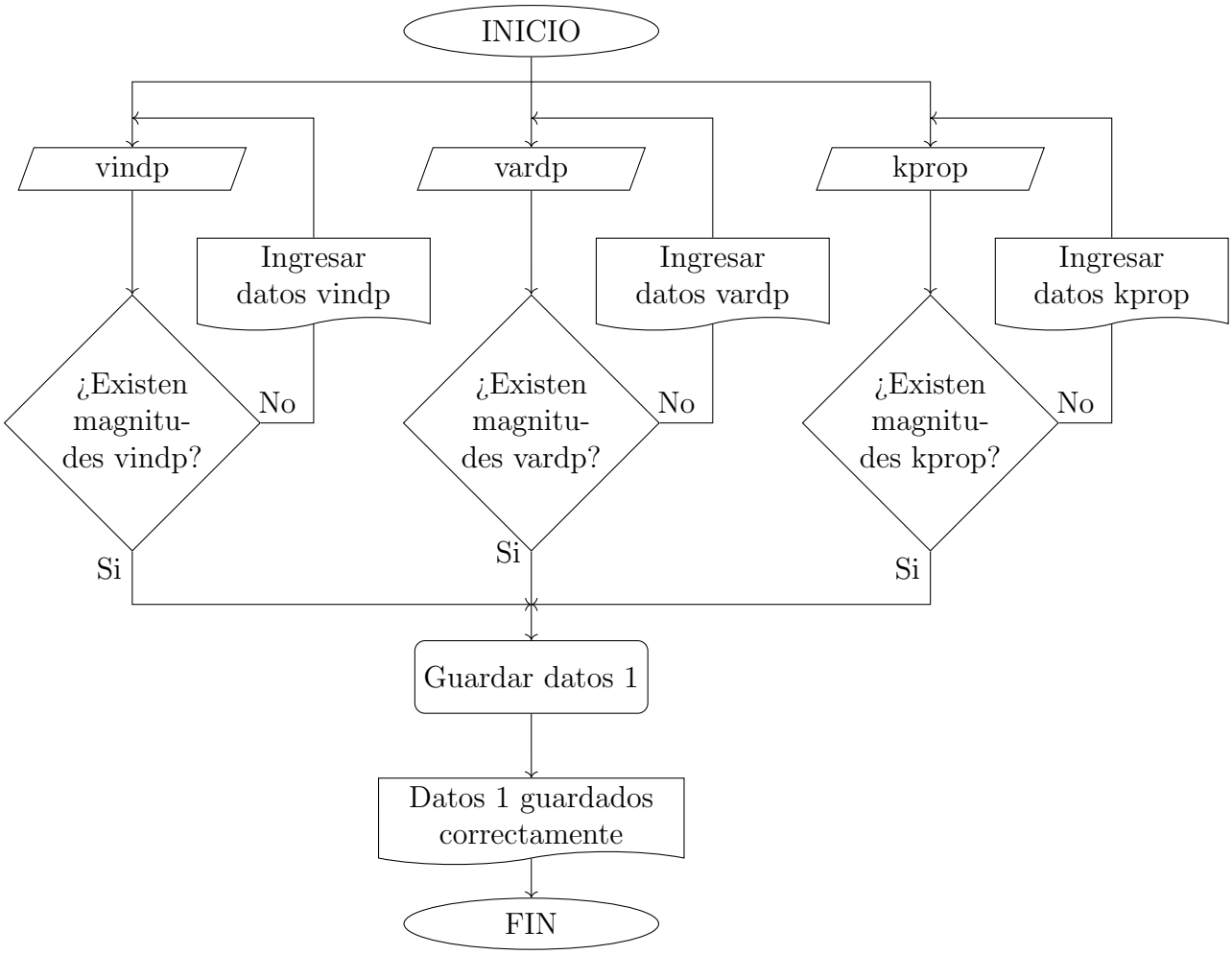
6.6 Algoritmo ejecutar



Este algoritmo acciona el algoritmo para compilar, pero antes verifica la existencia de los archivos `base_de_datos_1.txt` y `base_de_datos_2.txt`.

6.7 Algoritmo entradas vacías

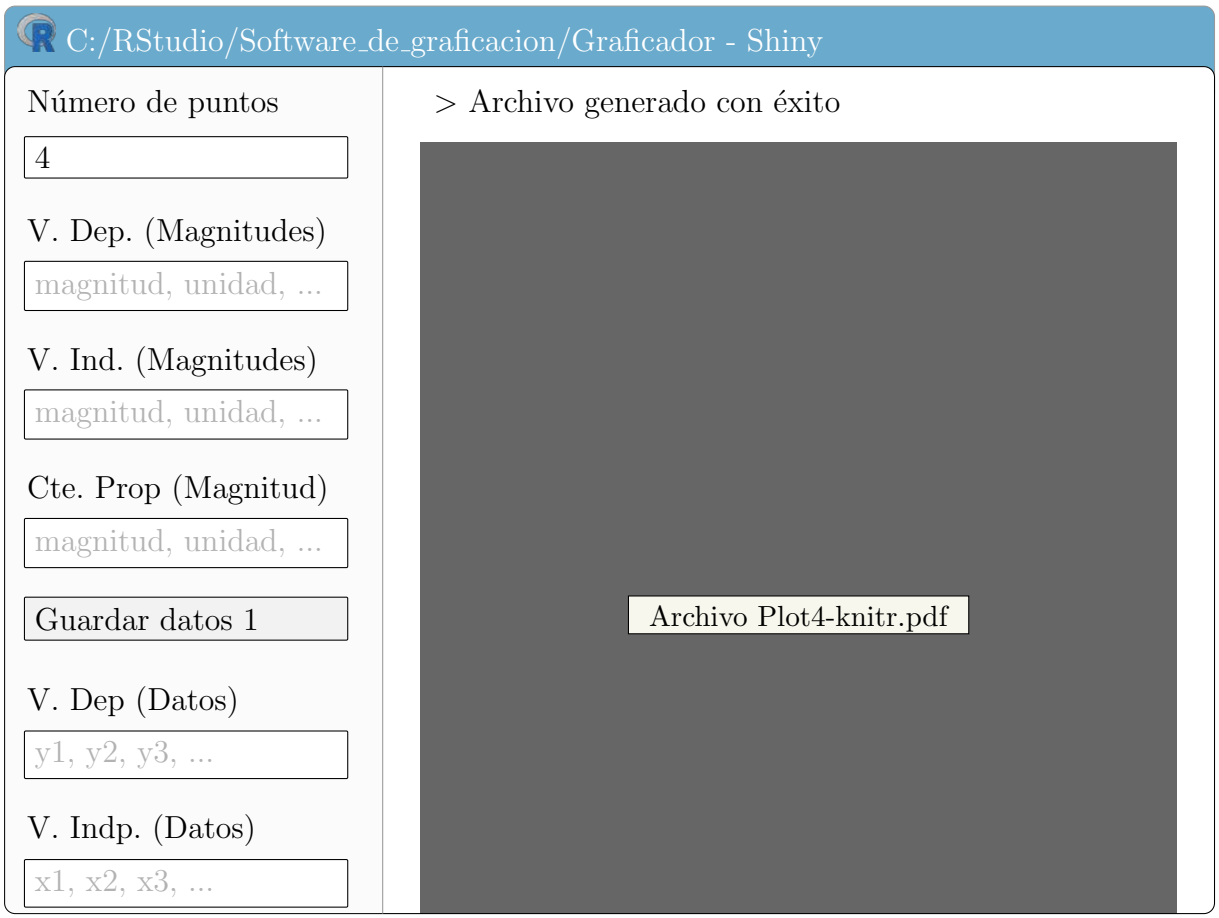
Es necesario un algoritmo, que busque si se ha ingresado datos (magnitudes, unidades y dimensiones) en las entradas variable dependiente `vindp`, variable independiente `vardp` y constante de proporcionalidad `kprop`.



Observación: Se puede aplicar el algoritmo de detección de entradas vacías para verificar si se han ingresado los datos experimentales de la variable dependiente (`vindpdata`) y la variable independiente (`vardpdata`). Esto es necesario para generar el archivo `base_de_datos2.txt`.

Por último, el desarrollo de estos diagramas de flujo permite comprender cómo los distintos procesos se ejecutarán en el servidor del *Software de Graficación*.

R




```
\begin{tikzpicture}
\path [fill=bluewhite,draw=black!40!,rounded corners] (-5,3.5) rectangle (11,4.8);
\node at (0.5,4.4) {\includegraphics[width=.5cm,height=.5cm]{R} \textcolor{white}{C:/
    RStudio/Software\_de\_graficacion/Graficador - Shiny}};
\draw [fill=white,rounded corners] (-5,-7.2) rectangle (11,4);
\begin{scope}
 \clip [rounded corners] (-5,-7.2) rectangle (.5,4);
 \draw [black!40!,fill=black!15!,fill opacity=.1] (-5.5,-7.3) rectangle (0,5);
\end{scope}
\node [box] at (-2.7,3.5) {Numero de puntos};
\node [box,draw,rounded corners=.5,fill=white] at (-2.6,2.8) {4};
\node [box] at (-2.7,1.8) {V. Dep. (Magnitudes)};
\node [box,draw,rounded corners=.5,fill=white] at (-2.6,1.1)  {\textcolor{black!30!}{
    magnitud, unidad, ...}};
\node [box] at (-2.7,.1) {V. Ind. (Magnitudes)};
\node [box,draw,rounded corners=.5,fill=white] at (-2.6,-.6)  {\textcolor{black!30!}{
    magnitud, unidad, ...}};
\node [box] at (-2.7,-1.6) {Cte. Prop (Magnitud)};
\node [box,draw,rounded corners=.5,fill=white] at (-2.6,-2.3)  {\textcolor{black!30!}{
    magnitud, unidad, ...}};
\node [box,draw,rounded corners=.5,fill=black!5!] at (-2.6,-3.3)  {Guardar datos 1};
\node [box] at (-2.7,-4.4) {V. Dep (Datos)};
\node [box,draw,rounded corners=.5,fill=white] at (-2.6,-5.1)  {\textcolor{black!30!}{y
    1, y2, y3, ...}};
\node [box] at (-2.7,-6.1) {V. Indp. (Datos)};
\node [box,draw,rounded corners=.5,fill=white] at (-2.6,-6.8)  {\textcolor{black!30!}{x
    1, x2, x3, ...}};
\path [fill=black,fill opacity=.6] (.5,-7.2) rectangle (10.5,3);
\node [rectangle,text width=9.65cm,align=left] at (5.5,3.5) {$>$ Archivo generado con
    exito};
\end{tikzpicture}
```

7.1 Librería Shiny

En esta sección, se deben centrar exclusivamente en el desarrollo de la aplicación *web* que albergará y ejecutará la mayoría de las actividades realizadas por **RStudio**. Para lograrlo, siga el siguiente procedimiento:

1. Revise detenidamente el capítulo de flujogramas, ya que muchos de los algoritmos desarrollados en dicho capítulo se aplicarán en este contexto.

2. Acceda a la ruta `C:\Software_de_graficacion\Graficador` y abra el archivo `Graficado.R` mediante la aplicación **RStudio**.

3. Instale la librería Shiny utilizando el siguiente comando:

   ```
   > install.packages("shiny")
   ```

   Nota: Recuerde que, para instalar e importar cualquier librería en **R**, debe ejecutar el código línea por línea utilizando el comando `Ctrl+Enter`.

4. Importar la librería Shiny y construya la interfaz de usuario (UI) y el servidor (Server). Puede utilizar el siguiente código como ejemplo:

   ```
   library(shiny)

   ui <- fluidPage(
   ...
   # AQUI SE DESARROLLA LA INTERFAZ DE USUARIO
   ...
   )

   server <- function(input, output, session) {
   ...
   # AQUI SE DESARROLLA EL SERVIDOR
   ...
   }

   shinyApp(ui, server)
   ```

7.1.1 Interfaz de Usuario UI

Ahora, se define en una interfaz de usuario (UI) mediante `sidebarLayout()` que constará de dos paneles: panel lateral o menú `sidebarPanel()` y un panel principal `mainPanel()`.

```
ui <- fluidPage(
 sidebarLayout(
  sidebarPanel(

  ),
  mainPanel(

  )
 )
)
```

Dentro del panel lateral, se requieren diversas entradas necesarias para ejecutar los algoritmos correspondientes (consulte el Capítulo 6). Esto incluye lo siguiente:

1. Una opción para seleccionar el número de datos que se desean graficar.

2. Tres campos de entrada para especificar la magnitud, unidad y dimensión de la variable dependiente, independiente y la constante de proporcionalidad.

3. Un botón para guardar los datos ingresados en estos tres campos (denominados "datos 1").

4. Dos campos de entrada para introducir los valores de los datos experimentales de la variable dependiente e independiente.

5. Otro botón para guardar los datos ingresados en estos dos campos (denominados "datos 2").

6. Además, se requiere un último botón en el panel lateral que permita compilar el archivo.

7. En cuanto al panel principal, se necesita un área de salida de texto que muestre mensajes relacionados con el éxito, fracaso y posibles errores del proceso.

Estos elementos en el panel lateral y principal son esenciales para interactuar y gestionar los datos experimentales y la ejecución de los algoritmos de forma efectiva en la aplicación.

```
ui <- fluidPage(
 sidebarLayout(
  sidebarPanel(
   selectInput(),
   textInput(),
   textInput(),
   textInput(),
   actionButton(),
   textInput(),
   textInput(),
   actionButton()
  ),
  mainPanel(
   textOutput()
  )
 )
)
```

Es importante proporcionar estructura a la entrada selectInput() y asignar etiquetas y nombres apropiados a las entradas de texto y botones.

```
ui <- fluidPage(
 sidebarLayout(
  sidebarPanel(
   selectInput("select", "Numero de puntos:", choices = list("4" = 4, "5" = 5, "6" = 6,
    "7" = 7, "8" = 8, "9" = 9, "10" = 10)),
   textInput("vardp", "Variable dependiente (Magnitudes):"),
   textInput("vindp", "Variable independiente (Magnitudes):"),
   textInput("kprop", "Cte. de proporcionalidad (Magnitudes):"),
   actionButton("datos1", "Guardar datos 1"),
   textInput("vardpdata", "Variable dependiente (Datos):"),
   textInput("vindpdata", "Variable independiente (Datos):"),
   actionButton("datos2", "Guardar datos 2"),
   actionButton("compile", "Graficar")
  ),
  mainPanel(
   textOutput("consola"),
   uiOutput("path")
  )
 )
)
```

Observación: Es relevante destacar que, por el momento, no se debe compilar la aplicación, ya que esto generaría un error: `Error in dots_list(...) : argument outputId is missing, with no default`. Este error se debe a que los algoritmos correspondientes aún no se han integrado en el servidor (`server`) de la aplicación.

7.1.2 Servidor

El servidor utilizará las entradas declaradas en la interfaz de usuario como argumentos y ejecutará los algoritmos en respuesta a tres eventos: "Guardar datos 1"(Evento 1), "Guardar datos 2"(Evento 2) y "Graficar"(Evento 3).

```
server <- function(input, output, session) {
 # EVENTO 1
 observeEvent(input$datos1, {
 # AQUI VA EL CODIGO
 (
 # Algoritmo guardar magnitudes, unidades y dimensiones
 # Algoritmo entradas vacias
 # Otros algoritmos
 )
 })
 # EVENTO 2
 observeEvent(input$datos2, {
 # AQUI VA EL CODIGO
 (
 # Algoritmo guardar datos experimentales
 # Algoritmo entradas vacias
 # Otros algoritmos
 )
 })
 # EVENTO 3
 observeEvent(input$compilar, {
 # AQUI VA EL CODIGO
 (
 # Algoritmo asignacion
 # Algoritmo para compilar
 # Algoritmo ejecutar
 # Otros algoritmos
 )
 })
}
```

El evento 1 permite la observación cuando el usuario presiona el botón "Guardar datos 1.º `input$datos1`. Cuando este evento ocurre, se ejecutan dos acciones específicas:

1. Algoritmo guardar magnitudes, unidades y dimensiones (Ver Flujograma 6.4)

```
# Algoritmo guardar magnitudes, unidades y dimensiones
vardp <- strsplit(input$vardp,",")[[1]]
vindp <- strsplit(input$vindp,",")[[1]]
kprop <- strsplit(input$kprop,",")[[1]]
etiquetasmg <- c("Magnitud","Unidad","Dimension")
datos1 <- data.frame(
matrix(etiquetasmg,nrow = 3,byrow = TRUE),
matrix(vindp,nrow = 3,byrow = TRUE),
matrix(vardp,nrow = 3,byrow = TRUE),
matrix(kprop,nrow = 3,byrow = TRUE)
)
```

2. Algoritmo entradas vacías (Ver Flujograma 6.7)

```
# Algoritmo entradas vacias
if (is.null(input$vardp) || input$vardp == "") {
 output$consola <- renderText({"
  > Ingresar magnitudes de la variable dependiente"
  })
} else if (is.null(input$vindp) || input$vindp == "") {
 output$consola <- renderText({"
  > Ingresar magnitudes de la variable independiente"
  })
} else if (is.null(input$kprop) || input$kprop == "") {
 output$consola <- renderText({"
  > Ingresar magnitudes de la constante de proporcionalidad"
  })
} else {
```

```
 write.table(datos1,file = "base_de_datos_1.txt", sep =" ",row.names = FALSE, col.
   names = FALSE,quote = FALSE)
 output$consola <- renderText("
  > Datos 0 guardados correctamente."
 )
}
```

Estos procedimientos son ejecutados en respuesta al evento de "Guardar datos 1z aseguran que se capturen y almacenen las magnitudes, unidades y dimensiones ingresadas por el usuario, y que se manejen apropiadamente las posibles entradas vacías en la interfaz de la aplicación.

Es importante mencionar que, en este punto usted ya puede ejecutar la aplicación *web*. Por defecto, el resultado se muestra en un ventana emergente de **RStudio**. Además, puede ingresar el valor de magnitudes de prueba o simplemente valores al azar en las entradas de texto correspondientes al evento 1 y guardar los datos. Para verificar, que se hayan guardado los datos deberá ingresar al ruta `C:\Software_de_graficacion\Graficador` y contrastar con los valores del archivo `base_de_datos_1.txt`.

El evento 2, de forma similar al evento 1 permite observar cuando el usuario presione el botón "Guardar datos 2" o `input$datos2`. Cuando este evento ocurre, se ejecutan dos acciones específicas:

1. Algoritmo guardar datos experimentales (Ver Flujograma)

```
# Algoritmo guardar datos experimentales
vardpdata <- strsplit(input$vardpdata,",")[[1]]
vindpdata <- strsplit(input$vindpdata,",")[[1]]
datos2 <- data.frame(
Independiente = matrix(vindpdata,ncol = 1,byrow = TRUE),
Dependiente = matrix(vardpdata,ncol = 1,byrow = TRUE)
)
datos2_traspuesto <- as.data.frame(t(datos2))
```

2. Algoritmo entradas vacías (Ver Flujograma 6.7)

```
# Algoritmo entradas vacias
if (is.null(input$vardpdata) || input$vardpdata == "") {
 output$consola <- renderText({"
  > Ingresar datos experimentales de la variable dependiente"
  })
} else if (is.null(input$vindpdata) || input$vindpdata == "") {
 output$consola <- renderText({"
  > Ingresar datos experimentales de la variable independiente"
  })
} else {
 write.table(datos2_traspuesto,file = "base_de_datos_2.txt", sep =" ",row.names =
   TRUE, col.names = FALSE,quote = FALSE)
 output$consola <- renderText("
  > Datos 2 guardados correctamente."
 )
}
```

Finalmente para el evento 3, se debe instalar la librería `knitr`, que permitirá convertir los archivos `Plot4.Rnw`, `Plot5.Rnw`, ..., `Plot10.Rnw` a archivos compatibles con `knitr`, que es una .evolución de Sweave".

```
> install.packages("knitr")
library("knitr")
```

El evento 3, permite observar la acción de presionar el botón "Guardar datos 2" o `input$datos2` por parte del usuario. Cuando este evento ocurre, se ejecutan tres acciones específicas:

1. Algoritmo de asignación (Ver Flujograma 6.2)

```
# Algoritmo asignacion
archivo = if(input$select == 4){
 4
} else if(input$select == 5){
5
} else if(input$select == 6){
6
} else if(input$select == 7){
7
} else if(input$select == 8){
8
} else if(input$select == 9){
9
} else {
10
}
```

2. Algoritmo para compilar (Ver Flujograma 6.3)

```
# Algoritmo para compilar
compilar <- function(archivo){
 Sweave2knitr(paste0("Plot", archivo, ".Rnw"))
 knit(paste0("Plot", archivo, "-knitr.Rnw"))
 tools::texi2pdf(paste0("Plot", archivo, "-knitr.tex"))
 output$path <- renderUI({
  tags$iframe(
  src = paste0("Graficador/Plot",archivo,"-knitr.pdf"),
  style = "height:800px; width:100%; scrolling=yes"
  )
 }
)
}
```

   Importante: Especificar la ruta en la que se deben buscar lso archivos generados por el evento 3, `Plot4-knitr.pdf`, `Plot5-knitr.pdf`, ..., `Plot10-knitr.pdf` para mostrarlos en la salida o contenedor `path` de la interfaz de usuario UI.

3. Algoritmo ejecutar (Ver Flujograma 1)

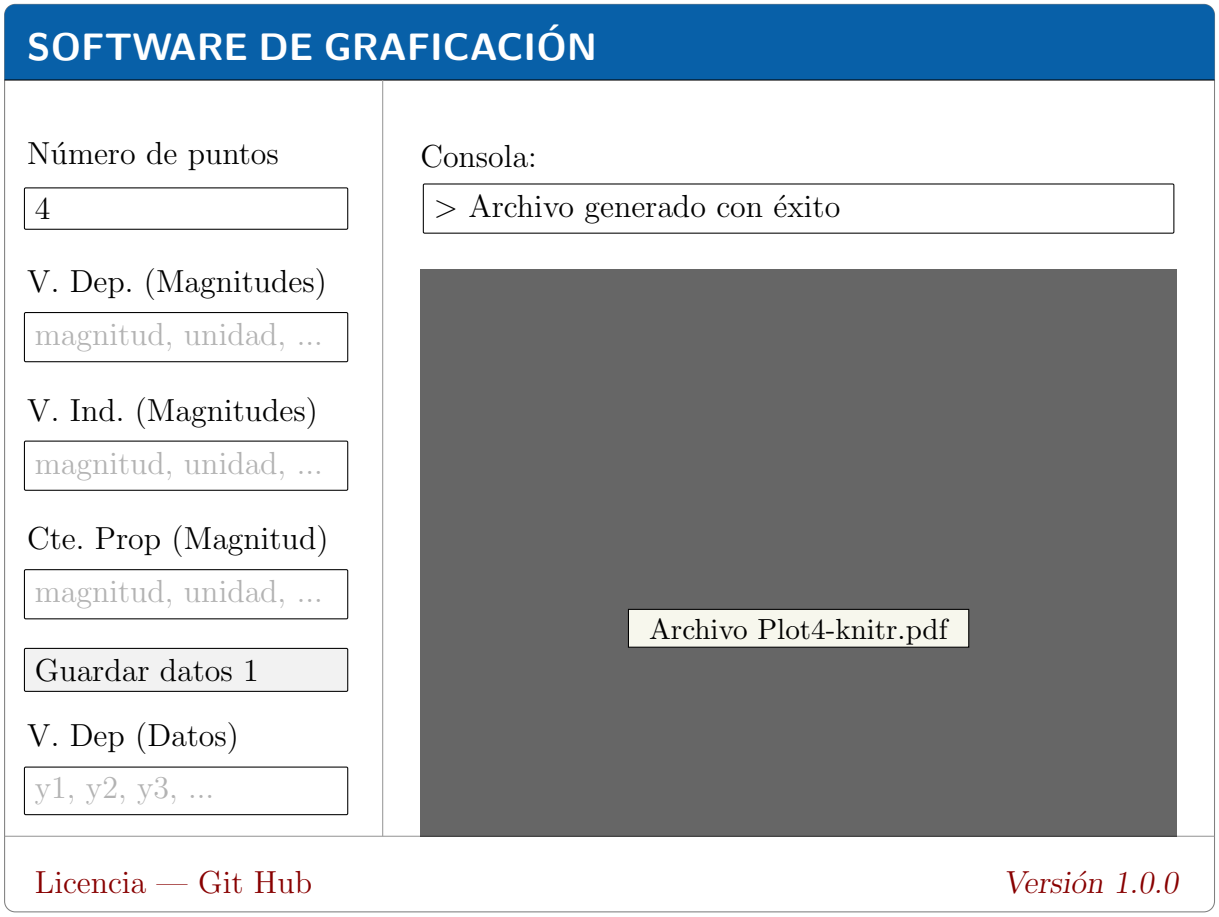
```
# Algoritmo ejecutar
compilar(archivo)
output$consola <- renderText("> Archivo PDF generado correctamente.")
```

Si se completaron correctamente todos los procesos detallados en los capítulos antecesesores, usted ya podría representar datos experimentales de forma dinámica, desde un servidor local o `localhost`. Además, estaría en la capacidad de poder realizar modificaciones en el código LaTeX, para incluir más datos experimentales, cambiar el color de los nodos, de la recta de regresión lineal, cambiar el posicionamiento de el triángulo de inspección, entre otros beneficios.

De igual manera, se recomienda llevar a cabo diversas pruebas con datos generados al azar para verificar el funcionamiento del *software de graficación* desarrollado.

Dado que hasta este punto se ha centrado exclusivamente en la funcionalidad del *software de graficación*, es esencial, en esta etapa, mejorar la parte visual de la aplicación. Además, contar con un dominio propio albergado en un alojamiento *web* permitirá evitar la necesidad de abrir **RStudio** cada vez que deseen representar, verificar o rectificar datos experimentales.

HTML y CSS




```
\begin{tikzpicture}
 \begin{scope}
  \clip [rounded corners] (-5,-7) rectangle (11,5);
  \draw [fill=backblue] (-5.5,4) rectangle (11.5,5.5);
  \draw [black!40!] (-5.5,-7.5) rectangle (11.5,-6);
 \end{scope}
 \begin{scope}
  \clip [rounded corners] (-5,-6) rectangle (.5,4);
  \draw [black!40!] (-5.5,-7) rectangle (0,5);
 \end{scope}
 \node [box] at (-2.7,3) {Numero de puntos};
 \node [box,draw,rounded corners=.5] at (-2.6,2.3) {4};
 \node [box] at (-2.7,1.3) {V. Dep. (Magnitudes)};
 \node [box,draw,rounded corners=.5] at (-2.6,0.6)  {magnitud, unidad, ...};
 \node [box] at (-2.7,-.4) {V. Ind. (Magnitudes)};
 \node [box,draw,rounded corners=.5] at (-2.6,-1.1)  {magnitud, unidad, ...};
 \node [box] at (-2.7,-2.1) {Cte. Prop (Magnitud)};
 \node [box,draw,rounded corners=.5] at (-2.6,-2.8)  {magnitud, unidad, ...};
 \node [box,draw,rounded corners=.5,fill=black!5!] at (-2.6,-3.8)  {Guardar datos 1};
 \node [box] at (-2.7,-4.7) {V. Dep (Datos)};
 \node [box,draw,rounded corners=.5] at (-2.6,-5.4)  {y1, y2, y3, ...};
 \node [box] at (-2.6,-6.6)  {\textcolor{redb}{Licencia | Git Hub}};
 \node [box] at (10.2,-6.6)  {\textcolor{redb}{\textsl{Version 1.0.0}}};
 \path [fill=black,fill opacity=.6] (.5,-6) rectangle (10.5,1.5);
 \node [box] at (2.5,3) {Consola:};
 \node [rectangle,text width=9.65cm,draw,align=left,rounded corners=.5] at (5.5,2.3) {$
   >$ Archivo generado con exito};
 \node [rectangle,text width=10cm,align=left] at (0.3,4.5) {\sffamily\large \textcolor{
   white}{\textbf{SOFTWARE DE GRAFICACION}}};
 \draw [fill=blue!30!yellow!10!white] (3.25,-3.5) rectangle (7.75,-3);
 \node at (5.5,-3.3) {\small Archivo Plot4-knitr.pdf};
\end{tikzpicture}
```

8.1 HTML y CSS

Es crucial que la interfaz de usuario (UI) del *Software de Graficación* sea simple y amigable, aunque es importante recordar que la percepción de la sencillez y la amigabilidad puede variar según las preferencias individuales de cada usuario. Por tal motivo, se desarrollará una estructura básica de HTML y CSS, para evitar mayores complicaciones.

8.1.1 Diseño HTML

Para incorporar etiquetas HTML en **R** se debe considerar la siguiente sintaxis:

Etiqueta HTML		Equivalencia en **R**
head	→	tags$head
body	→	tags$body
link	→	tags$link
script	→	tags$script
div	→	tags$div
strong	→	tags$strong
img	→	tags$img
nav	→	tags$nav
a	→	tags$a
i	→	tags$i
p	→	tags$p
...	→	...

El *Software de Graficación* se compone de dos secciones principales: los metadatos, representados por `tags$head`, y el cuerpo, que se encuentra en `tags$body`.

```
ui <- fluidPage(
 tags$head(
 # AQUI VAN LOS METADATOS
 ),
 tags$body(
 # AQUI VA LA UI
 )
)
```

En la sección de metadatos, se deben declarar las etiquetas para vincular con un ícono, con el documentos que albergan los estilos CSS, y un título.

```
ui <- fluidPage(
 tags$head(
  tags$title("Software de Graficacion"),
  tags$link(rel ="icon", href=""),
  tags$link(rel = "stylesheet", type = "text/css", href = "www/index.css")
 ),
 tags$body(
  ...
 )
)
```

En `href=" "` debe ingresar el enlace a la imagen que desea utilizar como ícono de la página *web*.

El cuerpo se subdivide en cuatro bloques fundamentales: el encabezado, el menú principal, el contenedor principal y el pie de página.

```
ui <- fluidPage(
 tags$head(
  ...
 ),
```

```
 tags$body(
  tags$div(class="cuerpo__grid",
  tags$div(class="encabezado",
  # AQUI VA EL CONTENIDO DEL ENCABEZADO
  ),
  tags$div(class="menu_principal",
    # AQUI VA EL CONTENIDO DEL MENU PRINCIPAL
  ),
  tags$div(class="contenedor_principal",
  # AQUI VA EL CONTENIDO DEL CONTENEDOR PRINCIPAL
  ),
  tags$div(class="pie_de_pagina",
  # AQUI VA EL CONTENIDO DEL PIE DE PAGINA
  )
 )
 )
)
```

En el encabezado, se incluirá el título o logotipo del *software de graficación*.

```
...
 tags$div(class="encabezado",
  tags$strong(tags$a("Software de Graficacion"))
 ),
...
```

El menú principal contendrá todas las entradas o `inputs`, permitiendo a los usuarios interactuar con la aplicación.

```
...
 tags$div(class="menu_principal",
  tags$nav(
   sidebarLayout(
    sidebarPanel(
     selectInput("select", "Numero de puntos:", choices = list("4" = 4, "5" = 5, "6" =
    6, "7" = 7, "8" = 8, "9" = 9, "10" = 10)),
     textInput("vardp", "Variable dependiente (Magnitudes):"),
     textInput("vindp", "Variable independiente (Magnitudes):"),
     textInput("kprop", "Cte. de proporcionalidad (Magnitudes):"),
     actionButton("datos1", "Guardar datos 1"),
     textInput("vardpdata", "Variable dependiente (Datos):"),
     textInput("vindpdata", "Variable independiente (Datos):"),
     actionButton("datos2", "Guardar datos 2"),
     actionButton("compilar", "Graficar")
     ),
     mainPanel()
   )
  )
 ),
...
```

El contenedor principal albergará las salidas o `outputs`, mostrando los resultados y representaciones gráficas generados.

```
...
 tags$div(class="contenedor_principal",
  tags$section(class="contenedor_principal_consola",
  textOutput("consola"),
  ),
  tags$section(class="contenedor_principal_pdf",
  uiOutput("path")
  )
 ),
...
```

Finalmente, el pie de página proporcionará enlaces a información relevante, como licencias, versiones y el repositorio del *software* en GitHub.

```
...
 tags$div(class="pie_de_pagina",
     tags$div(class="footer-left",
     tags$a(href="#","Licencia"),
     tags$a(href="#","| Git Hub")
```

```
    ),
    tags$div(class="footer-right",
    tags$em("Version 1.0.0")
    )
 )
...
```

8.1.2 Diseño CSS

Antes de comenzar a darle estilo a las diferentes estructuras de HTML, se debe especificar la ruta donde **R** deberá buscar la hoja de estilos CSS, es decir el archivo `index.css`.

```
library(shiny)
library(knitr)

addResourcePath("Graficador","C:/RStudio/Software_de_graficacion/Graficador")
addResourcePath("www","www")

...
```

Para configurar los estilos CSS, debe abrir el archivo `index.css` que se encuentra en la ruta `C:\Software_de_graficacion\Graficador\www`, con la aplicación **RStudio**.

1. Personalización de la clase `container-fluid` generada por Shiny.

```
/*CONTENEDOR GENERADO POR SHINY*/
.container-fluid {
 padding-right: 0px;
 padding-left: 0px;
 margin-right: auto;
 margin-left: auto;
}
```

2. Pesonalización de la clase `cuerpo__grid`.

```
/*CONTENEDORES GRID PARA SER RESPONSIVA*/
.cuerpo__grid {
 font-size: 12px;
 width: 100%;
 /*2560px*/
 min-height: 180vh;
 display: grid;
 grid-template-columns: 24em  1fr;
 grid-template-rows: 5em 1fr  4em;
}
```

3. Pesonalización del encabezado, clase `encabezado`.

```
.encabezado {
 grid-column: 1 / 4;
 background-color: #0860a8;
 display: flex;
 align-items: center;
 width: 100%;
 height: 5em;
 position: fixed;
 user-select: none;
 transition: all .6s;
 z-index: 5;
}
```

4. Personalización del titulo del encabezado, clase `encabezado` con elementos `strong` y `a`.

```
.encabezado strong a {
 color: #fff;
 text-decoration: none;
 font-size: 1.7em;
 padding-left: .3em;
}
```

5. Personalización de una columna generada por shiny.

```
.col-sm-4 {
 width: 100%;
}
```

6. Personalización del menú principal, clase `menu_principal`.

```
.menu_principal{
 grid-column: 1 / 2;
 grid-row: 2 / 4;
 border-right: 1px solid rgba(0, 0, 0, 0.2);
 transition: all .6s;
}
```

7. Personalización del menú principal, con elemento `nav`.

```
.menu_principal-nav{
 height: 100%;
 position: fixed;
 user-select: none;
}
```

8. Personalización del contenedor principal, clase `contenedor_principal`.

```
.contenedor_principal{
 grid-column: 2/4;
 grid-row: 2 / 4;
 display: flex;
 flex-direction: column;
 align-items: center;
 transition: all .6s;
 color: #000000;
 height: calc(100% - 4em);
 z-index: 0;
}
```

9. Personalización de la consola, clase `contenedor_principal_consola`.

```
.contenedor_principal_consola{
 width: 95%;
 display: flex;
 align-items: center;
 box-sizing: border-box;
 padding: 0em;
 margin-top: 1em;
 border: none;
 text-align: left;
}
```

10. Personalización del contenedor del archivo PDF, clase `contenedor_principal_pdf`.

```
.contenedor_principal_pdf {
 width: 95%;
 height: 100%;
 box-sizing: border-box;
 padding: 1em;
 margin-top: 1em;
 border: none;
 text-align: center;
}
```

11. Personalización del pie de página, clase `pie_de_pagina`.

```
.pie_de_pagina {
 grid-column: 1 / 3;
 grid-row: 3 / 4;
 z-index: 5;
 border-top: 1px solid rgba(0, 0, 0, 0.2);
 background-color: #fff;
 display: flex;
 justify-content: space-between;
 align-items: center;
}
```

12. Personalización del texto ubicado a la izquierda del pie de página, clase `footer-left`.

```
.footer-left{
 margin-left: 2em;
}
```

13. Personalización del texto ubicado a la derecha del pie de página, clase `footer-right`.

```
.footer-right{
 margin-right: 2em;
}
```

Es importante destacar que tanto la estructura HTML como los estilos CSS no son de carácter definitivo, ya que su diseño y organización dependen en gran medida de las preferencias y necesidades del desarrollador y del propósito del *Software de Graficación*. Cada desarrollador tiene su enfoque único y su estilo de diseño, lo que da lugar a una amplia variedad de enfoques creativos.

Algunos desarrolladores optan por interfaces más complejas, incorporando múltiples elementos y características interactivas para ofrecer una experiencia enriquecida a los usuarios. Estas interfaces pueden ser ideales para aplicaciones web avanzadas o sitios *web* con requerimientos específicos.

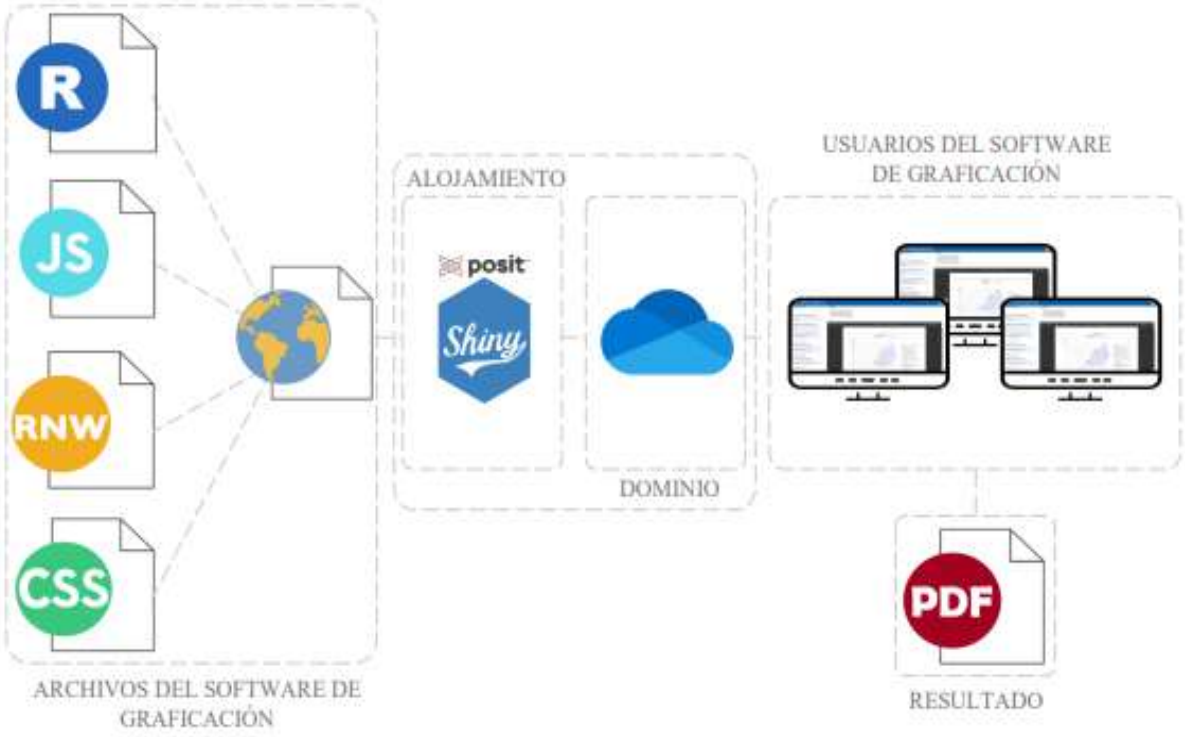
Por otro lado, hay desarrolladores que prefieren enfoques más simples y minimalistas, centrándose en la usabilidad y la accesibilidad. Estas interfaces tienden a ser limpias y fáciles de navegar, lo que las hace ideales para sitios *web* informativos o aplicaciones donde la simplicidad es clave.

A continuación, se proponen algunas ideas que podrían ser consideradas para mejorar aún más el *Software de Graficación*:

1. Implementar el uso de archivos HTML externos para una gestión más eficiente.
2. Incorporar JavaScript desde archivos externos para mejorar la interactividad.
3. Explorar la posibilidad de utilizar menús desplegables para aprovechar al máximo el espacio de visualización.
4. Agregar la opción de descarga en diversos formatos para brindar flexibilidad a los usuarios.
5. Considerar la inclusión de una función de retroalimentación o cuestionario para recopilar comentarios y detectar posibles fallos.

Lo fundamental es que tanto la estructura como el diseño se adapten eficazmente a los objetivos y requisitos específicos del *Software de Graficación*, garantizando a los usuarios una experiencia satisfactoria y coherente.

Publicación




```
\includegraphics[width=15cm,height=10cm]{Publi}
```

Antes de publicar el *Software*, es necesario realizar un leve cambio en `Graficador.R` que se encuentra en la ruta `C:\Software_de_graficacion\Graficador`. A continuación, se detallan los procesos a seguir:

1. En primer lugar, es importante instalar la librería `here` para poder construir rutas relativas. Para hacerlo, basta con ejecutar el siguiente comando en la consola de **R**:

```
> install.packages("here")
```

2. A continuación, se debe importar la librería `here` en el archivo `Graficador.R`. Para ello, se añade la siguiente declaración junto con las demás librerías utilizadas:

```
...
library(knitr)
library(here)
...
```

3. Por último, se realiza un cambio en los directorios absolutos, reemplazándolos por directorios relativos `here::here()`. Este ajuste es fundamental para poder publicar el *software*, ya que los servicios de alojamiento web, como *Shiny Apps*, requieren rutas relativas. A continuación, se muestra el cambio en el código:

```
...
addResourcePath("Graficador","here::here()")
addResourcePath("www","www")
...
```

Estos pasos aseguran que el *software* sea más portable y esté listo para su publicación en plataformas de alojamiento *web*, garantizando que las rutas de acceso sean correctas y que el *software* funcione sin problemas.

9.1 Shiny Apps

Para publicar el *Software de Graficación*, es esencial contar con una cuenta en el sitio de alojamiento de aplicaciones de Shiny. Una vez que haya configurado su cuenta, puede seguir estos pasos para llevar a cabo la publicación:

1. Ingrese a la ruta `C:\Software_de_graficacion\Graficador` y abra el archivo `Graficador.R`.

2. En la interfaz de **RStudio**, ubique y haga clic en el icono destinado a publicar la aplicación o documento, situado en la parte superior derecha de la ventana.

3. En la ventana emergente denominada `"Publish to Server"`, realice la selección de los siguientes archivos:

   a) `Graficador.R`.

   b) `Plot4.Rnw`,`Plot5.Rnw`,`Plot6.Rnw`, y así sucesivamente hasta `Plot10.Rnw`.

   c) `www/index.css`.

4. Asigne un nombre en el campo `Title` para el dominio de su página *web*.

5. Espere a que finalice el proceso de carga de archivos y, finalmente, dispondrá de su propio *software de graficación* listo y accesible en línea.

Estos pasos le permitirán publicar su *software* de manera efectiva en la plataforma de alojamiento de Shiny Apps, y evitar tener que abrir **RStudio** cada vez que desee representar datos experimentales.

Referencias Bibliográficas

Feuersänger, C. (2021). *Manual for Package pgfplots*. CTAN. https://n9.cl/8mievw

Leisch, F. (2005). Sweave User Manual. https://n9.cl/1vbur

Morales, R. (2006). *Generación automática de resportes con R y LaTeX*. CRAN. https://n9.cl/x68r0.

Paradis, E. (2002). *R para principiantes*. CRAN. https://n9.cl/edl8r.

Tantau, T. (2023). *The Tikz and PGF Packages: Manual for version 3.1.10*. CTAN. https://n9.cl/10kqu.